

基于DeepAgents深度搜索项目

开篇引入

在过去短短数年间，人工智能的形态正经历一场层层递进的深刻演化。它从最初仅能“响应问题”的大型语言模型（LLM），逐步迭代为具备“调用工具、落地执行”能力的 AI Agent，如今更朝着拥有协作意识、可驾驭复杂工作流的 Agentic AI 加速迈进。这条演进路径绝非简单的功能叠加，而是一条从“语言理解能力”到“自主行动能力”，再到“智能组织与协作能力”的质变之路——几乎构筑起未来智能应用的核心主轴。



为突破现有技术瓶颈，两个关键概念在研究与实践领域迅速崛起并成为核心驱动力——**深度代理（Deep Agents）**与**高阶提示（Higher-Order Prompts, HOPs）**。

在深度代理的架构中，模型不再是“一次性输出答案”的黑盒，而是化身具备“规划 - 执行 - 反馈 - 迭代”闭环能力的智能主体：

- 面对复杂任务时，先将其拆解为可落地的子目标；
- 为每个子目标匹配专属的子代理（sub-agent），实现专业化分工；
- 执行过程中实时监控各步骤产出，精准识别偏离目标的异常；
- 基于执行结果动态调整计划、替换策略，或生成新的子任务以补全链路；
- 当出现错误时，通过反思（reflection）机制回溯问题根源，修正执行路径。

而高阶提示（HOPs）则聚焦于“教会模型如何思考”：如果说深度代理是智能系统的“组织架构师”，负责搭建任务执行的骨架，那么高阶提示就是“认知规范师”，定义思考与推理的底层逻辑。

- 传统提示**：告诉我做什么。

分析下面这条用户评论，判断情绪是正面、负面还是中性，并给出改进建议。

评论：“这个软件经常卡顿，打开要等很久，界面也不好看，用起来很烦躁。”

- 高阶提示**：告诉我怎么想、按什么步骤、用什么逻辑。

请按以下**固定思考流程**分析用户评论：

- 先逐句提取事实：用户提到了哪些具体问题？
- 再判断情绪：根据关键词判断是正面 / 负面 / 中性，并给出理由。
- 按问题严重程度排序：从最影响体验到最轻。
- 每条问题对应给出**可落地**的改进建议，不要空泛。
- 最后用一句话总结核心痛点。

评论：“这个软件经常卡顿，打开要等很久，界面也不好看，用起来很烦躁。”

传统提示偏向“直接下达结果指令”，而高阶提示则更进一步，它向模型传递的是“思考框架与推理范式”——明确告知模型该如何分析问题、如何拆解逻辑、如何组织思考过程，从根源上提升决策的精准度与执行的可靠性。

一、DeepAgents 框架核心

1.1 DeepAgents介绍和作用

构建能够规划、使用子代理并利用文件系统处理复杂任务的代理(深度智能体)

<https://docs.langchain.com/oss/python/deepagents/overview>

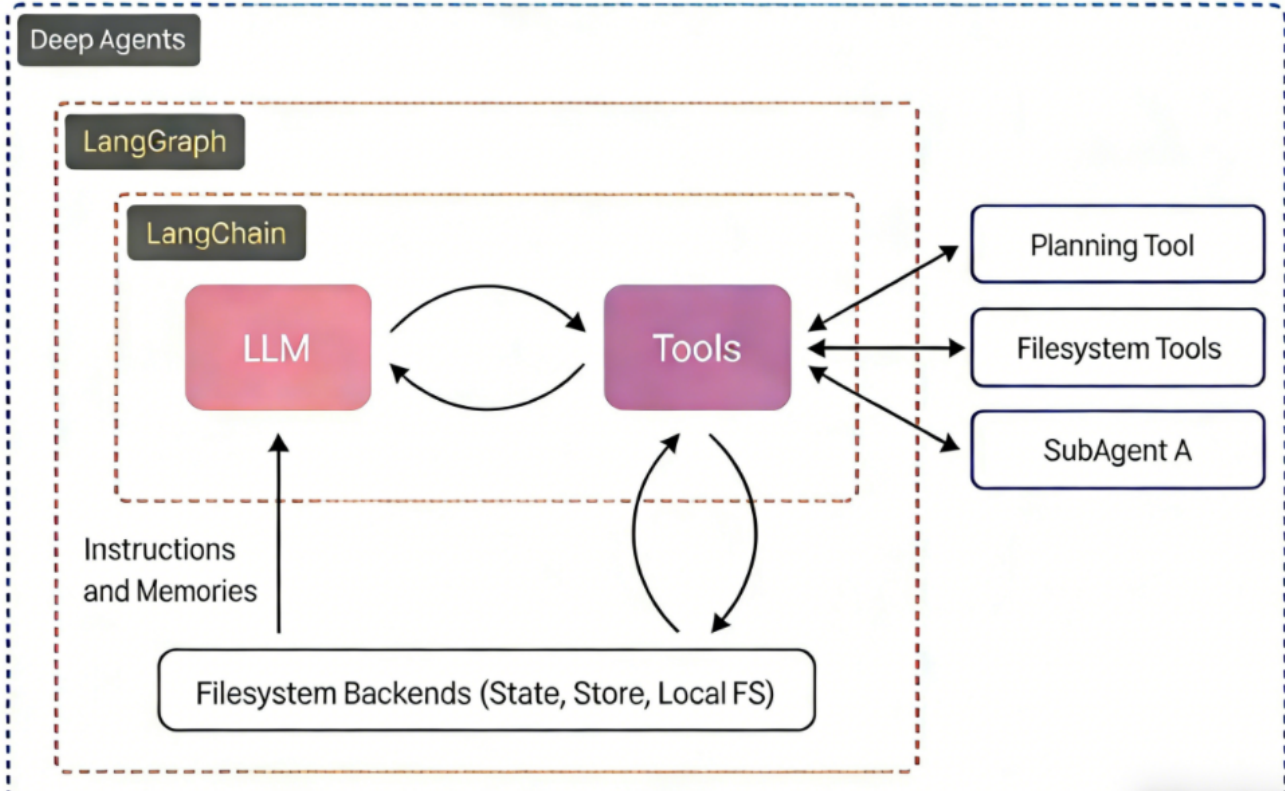
DeepAgents是一个独立库，建立在 LangChain代理核心构建模块之上（关系类似于：SpringBoot和Spring Framework）！主要实现**自主多智能体系统（Agentic AI）**！**DeepAgents**是构建由大型语言模型（LLM）驱动的代理和应用的最简单方式——内置任务规划功能、用于上下文管理的文件系统、子代理生成和长期记忆。可以用它完成复杂、多步骤、切自主规划的任务。

家族框架对比：

<https://docs.langchain.com/oss/python/concepts/products>

三者并非单向引用，有时会出现循环调用场景！！

- **LangChain (Framework)**: 做“动作”，是核心的代理框架。它封装了 LLM 与工具的交互，提供灵活的代理结构，但不包含规划、记忆或文件系统，适合开发者自己定制逻辑。
- **LangGraph (Runtime)**: 管“流程”，是外层的运行时。它把执行变成可管理的图结构，支持循环、并行和持久化，保障智能体执行的稳定与可控。
- **DeepAgents (Harness)**: 负责“组织”，是最外层的工具包。它内置规划器、子代理、文件系统和持久存储，让智能体从“能执行”升级为“能组织、能管理、能记忆”的深度智能体。



功能横向对比：

	Framework	Runtime	Harness
Value add	• Abstractions • Integrations	• Durable execution • Streaming • HITL • Persistence	• Predefined tools • Prompts • Subagents
When to use	• Getting started quickly • Standardizing how a team builds	• Low-level control • Long running, stateful workflows and agents	• More autonomous agents • Agents faced with complex, non-deterministic tasks
Options	• LangChain • Vercel's AI SDK • CrewAI • OpenAI Agents SDK • Google ADK • LlamaIndex	• LangGraph • Temporal • Inngest	• Deep Agents SDK • Claude Agent SDK • Manus

LangChain、LangGraph、Deep Agents 使用场景总结：

1. 何时使用 LangChain?
 - 你想**快速构建代理与自主应用**。
 - 你需要对**模型、工具、代理循环**提供标准抽象。（单Agent）
 - 你需要**易用且灵活**的开发框架。
 - 你在开发**简单直接的代理应用**，无复杂编排需求。
2. 何时使用 LangGraph?
 - 你需要对**代理编排做细粒度、底层控制**。
 - 你需要**持久化执行**，支持**长时间运行、有状态的代理**。
 - 你在构建**结合确定性步骤与智能代理步骤**的复杂 workflows。
 - 你需要**可直接用于生产环境**的代理部署基础设施。
3. 何时使用 Deep Agents SDK?

- 你在构建**长期运行、持续运营、自主规划**的智能代理。
- 你在构建需要处理**复杂、多步骤任务**的代理。
- 你需要使用**预定义工具**：如文件系统操作、自定义工具、自动化上下文工程等。
- 你希望直接使用**预设提示词与子代理能力**。

1.2 DeepAgents核心能力

核心能力一： 智能规划与任务分解 (最核心智能协调体现！避免传统的工作流)

DeepAgents 内置 `write_todos` 工具，使代理能够：

- 将复杂任务分解为离散的执行步骤
- 实时跟踪任务执行进度
- 根据新信息动态调整执行计划

例如：你要 “办一场生日派对”，DeepAgent 不会上来就瞎忙活，而是先帮你列个清晰的待办清单：

1. 确定派对时间 / 地点 → 2. 邀请朋友 → 3. 买食材 / 蛋糕 → 4. 布置场地 → 5. 准备游戏

执行过程中，它还会实时标记 “已完成 / 未完成”，比如发现蛋糕店没开门，会自动把 “买蛋糕” 改成 “换一家店”，甚至新增 “订外卖蛋糕” 的步骤——完全像个有经验的策划师，把复杂事拆成一步步能落地的小事，还能灵活调整。

提示: `write_todos` 不是 Python 原生的函数，而是 **DeepAgents 框架内置的一个 “工具函数”** —— 你可以把它理解成：

DeepAgent 自带的一个 “待办清单生成器”，专门让智能体把复杂任务拆成一条条可执行的待办事项，并且能把这些事项存起来、DeepAgent 还配了其他工具进行调整

- `read_todos`：智能体执行任务时，用来 “读取” 当前的待办清单，知道下一步该做啥；
- `update_todos`：执行中发现需要调整步骤（比如 “检索资料” 需要加 “筛选权威来源”）来修改清单；
- `delete_todos`：删掉不需要的步骤（比如报告写完后，删掉 “检查完整性” 的重复项）。

核心能力二： 高效上下文管理（最实用的 “内存扩容” 方案！避免上下文溢出）

DeepAgents 内置文件系统工具集（`ls`、`read_file`、`write_file`、`edit_file`），使代理能够：

- 将大型上下文信息卸载到外部存储
- 有效防止上下文窗口溢出问题
- 处理可变长度的工具执行结果

就像你要 “整理 100 页的年度工作总结”，你的大脑（对应代理的上下文窗口）只能同时记住 10 页内容，多了就会忘前忘后。DeepAgent 会帮你准备一个 “专属文件柜”：

1. 先把 100 页总结拆成 10 个文件存在柜里 → 2. 看第 1-10 页时，只把这 10 页拿在手里 → 3. 看完后放回，再取 11-20 页 → 4. 整理出的要点先写在草稿纸（临时文件）里，最后汇总成最终版本

执行过程中，它还会灵活管理文件：比如查到一份 5 万字的 AI 行业数据，不会硬塞进 “大脑”，而是用 `write_file` 存到文件柜，需要某段数据时，用 `read_file` 精准调取；想看看存了哪些文件，用 `ls` 列一下；发现数据有误，用 `edit_file` 直接修改 —— 完全像个会整理文件的助理，让 “大脑” 只装当前要用的信息，既不超载，也不丢东西。

提示: 这套文件系统工具集不是简单的 “本地文件操作”，而是 **DeepAgents 封装的跨存储工具** —— 你可以把它理解成：DeepAgent 自带的 “智能文件管家”，专门帮代理管理超出 “大脑容量” 的信息，支持本地文件、云存储（OSS/S3）等多种存储方式，还能和其他工具联动。

核心能力三： 子代理生成机制（最灵活的 “分工协作” 模式！避免主代理超负荷）

DeepAgents 内置 `task` 工具，使代理能够：

- 选择对应的子代理处理特定任务
- 实现上下文隔离，保持主代理环境整洁
- 深入执行复杂的子任务流程

就像你要 “装修一套房子”，你（主代理）不会既当设计师、又当瓦工、还当水电工，而是找专业的人干专业的事：

1. 派 “设计子代理” 出装修图纸 → 2. 派 “施工子代理” 按图纸砌墙 / 铺砖 → 3. 派 “水电子代理” 装水管 / 电路 → 4. 你只负责统筹进度、汇总结果

执行过程中，子代理还能独立工作：比如设计子代理只关注 “风格、尺寸、配色”，不会被施工细节干扰；施工子代理出了问题（比如墙面不平），只会自己调整，不会影响你和水电子代理的工作 —— 完全像个会 “招兵买马” 的项目经理，把复杂任务拆给专业的 “帮手”，自己只做全局把控。

核心能力四： 长期记忆能力（最持久的 “记忆存储” 系统！避免代理 “失忆”）

DeepAgents 利用 LangGraph 的 Store 功能，使代理能够：

- 为代理扩展跨线程(协程)持久内存
- 保存和检索历史对话信息
- 支持多会话间的知识共享

就像你要 “持续跟进一个客户的需求”，你不会每次和客户聊天都从头问 “你想要什么功能”，而是有一个 “客户档案本”：

1. 第一次聊天记录客户 “想要红色的产品、预算 5000 元” → 存进档案本 →
2. 一周后聊天，先翻档案本记住之前的需求 →
3. 新聊的 “要加定制 logo” 也补充进去 →
4. 同事跟进时，也能看这个档案本，不用你再转述

执行过程中，这份记忆还能跨场景用：比如你在电脑上和客户聊的需求，换手机登录后，代理还能查到档案本；多个同事（多代理）跟进同一个客户，都能共享这份档案 —— 完全像个带 “永久档案柜” 的客服，不管过多久、换什么设备，都能记住之前的信息。

1.3 DeepAgents快速入门

快速构建第一个 Deep Agent：**一个能够自主联网搜索并撰写报告的“AI 研究员”**会借用Tavily网络搜索工具！

步骤1：安装依赖

```
1 pip install deepagents tavily-python python-dotenv langchain-openai
```

步骤2：配置 API Key

确保你拥有 LLM 和 Tavily (搜索) 的 API Key。位置：`.env`

```
1 # OPENAI风格配置
2 OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
3 OPENAI_API_KEY=sk-6296bb4dab98463689911fd07a973c97
4 LLM_QWEN3=qwen3-32b
5 LLM_QWEN_MAX=qwen-max
6
7 #https://app.tavily.com/
8 #tavily-api-key
9 TAVILY_API_KEY=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
```

步骤3：定义搜索工具

DeepAgents 需要通过工具与外部世界交互。我们先定义一个简单的联网搜索工具。

位置：`tavily_tools.py`

```
1 from typing import Literal
2 from langchain.tools import tool
3 from tavily import TavilyClient
4 from dotenv import load_dotenv,find_dotenv
5 import os
6
7 # 加载 .env文件
8 load_dotenv(find_dotenv())
9
10 # 创建tavily_client
11 tavily_client = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))
12
13 # 定义搜索工具
14 @tool
15 def internet_search(
16     query:str,
17     max_results:int =10,
18     topic:Literal["general","news","finance"] = "general",
19     include_raw_content:bool = False):
20     """
21     互联网搜索工具！
22     :param query: 搜索关键字
23     :param max_results: 返回结果数量
24     :param topic: 主题类型
25     :param include_raw_content: False精简 True 返回详细结果
26     :return: 搜索结果列表
27     """
28     print(f"进行网络搜索！搜索条件：{query},搜索主题类别:{topic},搜索最大的条数：{max_results}")
29     return tavily_client.search(
30         query=query,
31         max_results=max_results,
```

```
32         topic=topic,
33         include_raw_content=include_raw_content
34     )
```

步骤4：创建 Deep Agent

通过 `create_deep_agent` 工厂函数，将工具和 System Prompt 组装成一个智能体。

```
1  from langchain.chat_models import init_chat_model
2  from deepagents import create_deep_agent
3  import os
4  from dotenv import load_dotenv, find_dotenv
5
6  from base.tavily_tool import internet_search
7
8  # 使用 find_dotenv() 自动查找 .env 文件，无论你在哪个目录下运行脚本都能正确加载环境变量
9  load_dotenv(find_dotenv())
10
11 # 极简初始化（自动读取OPENAI环境变量）
12 llm = init_chat_model(
13     model=os.getenv("LLM_QWEN_MAX"),
14     model_provider="openai"
15 )
16
17 # api地址 https://reference.langchain.com/python/deepagents/graph/
18 # 功能等价于langchain的 create_agent
19 deep_agent = create_deep_agent(
20     model=llm,
21     tools=[internet_search],
22     subagents=[],
23     system_prompt="""
24     你是一位专家级研究员。你的任务是进行深入研究并撰写一份精美的报告。
25     你有权使用 internet_search 工具来收集信息。
26     """
27 )
```

步骤5：运行并获取结果

```
1  # 运行代理
2  prompt = input("输入你关心的问题！")
3  result = deep_agent.invoke({
4      "messages": [
5          {"role": "user", "content": f"{prompt}"}
6      ]
7  })
8  #result = main_agent.invoke({"inout": "人工智能和机器人的热点新闻！！"})
9
10 """
11 结果数据说明
12  {
13      "messages": [
14          # 第0条：你的提问（HumanMessage）
15          HumanMessage(content='搜索宇树机器人的新闻！'),
16          # 第1条：Agent 调用工具的指令（AIMessage，内容为空，仅触发工具）
17          AIMessage(content='', tool_calls=[{'name': 'internet_search', ...}]),
18          # 第2条：工具返回的搜索结果（ToolMessage，一堆JSON数据）
19          ToolMessage(content='{"query": "宇树机器人 新闻", "results": [...]}'),
20          # 第3条：Agent 整理后的最终回复（AIMessage，这是你要的内容）
21          AIMessage(content='以下是关于宇树机器人的一些最新新闻：...')
22      ]
23  }
24  """
25  print(result['messages'][-1].content)
```

总结

- 1. `result['messages']`：定位到存储全流程对话的列表；
- 2. `[-1]`：精准抓取列表最后一条（Agent 整理后的最终回复）；
- 3. `.content`：过滤掉所有冗余属性，只取纯文本回复内容。

1.4 DeepAgents流式处理结果解析（重点）

深度代理基于 LangGraph 的流基础设施构建，提供一流的子代理流支持。当深度代理将工作委派给子代理时，你可以独立从每个子代理处流式更新——实时跟踪进展、LLM 令牌和工具调用

```
1 # 运行代理
2 # 输入：查询机器人最新的热点
3 prompt = input("输入你关心的问题！")
4 # 同步流
5 stream = deep_agent.stream({
6     "messages": [
7         {"role": "user", "content": f"{prompt}"}
8     ]
9 })
10 # =====
11 # Chunk 数据结构参考文档（Python 对象视图）
12 # =====
13 # LangGraph 流式输出（chunk）的四种核心场景示例：
14 # 1. [场景 A：Agent 思考并决定调用工具]
15 # {
16 #     "model": {
17 #         "messages": [
18 #             AIMessage(
19 #                 content="",
20 #                 tool_calls=[{
21 #                     "name": "read_file_content",
22 #                     "args": {"filename": "需求.docx"},
23 #                     "id": "call_123"
24 #                 }]
25 #             )
26 #         ]
27 #     }
28 # }
29 # 2. [场景 B：工具执行完毕，返回结果]
30 # {
31 #     "tools": {
32 #         "messages": [
33 #             ToolMessage(
34 #                 content="[文件内容]...",
35 #                 name="read_file_content",
36 #                 tool_call_id="call_123"
37 #             )
38 #         ]
39 #     }
40 # }
41 # 3. [场景 C：Agent 决定调用子 Agent（特殊工具 'task'）]
42 # {
43 #     "model": {
44 #         "messages": [
45 #             AIMessage(
46 #                 content="",
47 #                 tool_calls=[{
48 #                     "name": "task",
49 #                     "args": {
50 #                         "subagent_type": "网络搜索助手", # 目标子 Agent
51 #                         "description": "查询2024政策" # 下发的具体任务
52 #                     },
53 #                     "id": "call_456"
54 #                 }]
55 #             )
56 #         ]
57 #     }
58 # }
59 # 4. [场景 D：Agent 最终回复用户]
60 # {
61 #     "model": {
62 #         "messages": [
63 #             AIMessage(
64 #                 content="根据查询结果，2024年新政策如下...",
```

```
65 #         tool_calls=[]
66 #     )
67 # ]
68 # }
69 # }
70 # =====
71 for chunk in stream:
72     """
73     # 场景1: 单个节点更新 ( 常见 )
74     chunk = {
75         "model": {"messages": [AIMessage(content='', tool_calls=[...])]} # 仅模型节点更新
76     }
77
78     # 场景2: 多个节点同时更新 ( 少数但存在 )
79     chunk = {
80         "model": {"messages": [AIMessage(content='最终回复...')]}, # 模型节点
81         "tools": {"messages": [ToolMessage(content='工具结果...')]}, # 工具节点
82         "todos": {"todos_list": ["已完成: 搜索宇树机器人新闻"]} # 待办节点
83     }
84     """
85     for node_name, state in chunk.items():
86         print(f"本次处理的节点类型{node_name}")
87         # 有些中间节点 例如: TodoListMiddleware 没有 messages跳过!
88         if not state or "messages" not in state: continue
89         # 有的直接获取
90         messages = state["messages"]
91         # message不为null!并且是集合类型
92         if messages and isinstance(messages, list):
93             # 获取最后一条就是最终结果
94             last_msg = messages[-1]
95             # 1. 模型节点 (model): 决定下一步行动
96             if node_name == "model":
97                 # 如果有 tool_calls, 说明模型决定调用工具或子智能体
98                 if last_msg.tool_calls:
99                     for tool_call in last_msg.tool_calls:
100                         if tool_call['name'] == 'task':
101                             sub_agent = tool_call['args'].get('subagent_type')
102                             print(f"[模型决策] 呼叫子智能体: {sub_agent}")
103                         else:
104                             print(f"[模型决策] 调用工具: {tool_call['name']}, 参数为: {tool_call['args']}")
105                 # 如果没有 tool_calls 且有 content, 说明是最终回复
106                 elif last_msg.content:
107                     print(f"📄 [最终回复] {last_msg.content}")
108             # 2. 工具节点 (tools): 显示工具/子智能体的执行结果
109             elif node_name == "tools":
110                 # ToolMessage 的 content 是工具返回的原始数据 (可能是 JSON 字符串)
111                 # 建议只打印前 100 个字符, 避免刷屏
112                 # 展开成普通if-else, 更易理解
113                 content_preview = ''
114                 if len(last_msg.content) > 100:
115                     # 取前100个字符 + 省略号 ( 截断预览 )
116                     content_preview = last_msg.content[:100] + "..."
117                 else:
118                     # 内容较短, 直接完整显示
119                     content_preview = last_msg.content
120                 print(f"[执行结果] {content_preview}")
```

解读返回结果:

场景 1: 智能体前置处理 (before_agent 节点)

节点名: PatchToolCallsMiddleware.before_agent

核心含义: 接收用户输入, 格式化 / 校验消息

```
1 {
2     "PatchToolCallsMiddleware.before_agent": {
3         "messages": Overwrite( # LangChain 自定义Overwrite对象
4             value=[ # 核心数据在value字段
5                 HumanMessage( # 用户消息对象
```

```
6         content="北京今天天气怎么样？", # 用户提问内容
7         additional_kwargs={},
8         response_metadata={},
9         id="466118de-5cdb-4250-a57f-bacf28b6407a" # 消息唯一ID
10    )
11 ]
12 )
13 }
14 }
```

场景 2：模型思考（决定调用工具）（model 节点）

节点名：model

核心含义：大模型分析问题，决定调用工具 / 子代理（无直接回答）

```
1 {
2   "model": {
3     "messages": [
4       AIMessage( # 模型消息对象
5         content="", # 内容为空（因为要调用工具）
6         additional_kwargs={"refusal": None},
7         response_metadata={ # 模型元数据
8           "token_usage": {"completion_tokens": 30, "prompt_tokens": 5265, "total_tokens": 5295},
9           "model_provider": "openai",
10          "model_name": "qwen-max",
11          "finish_reason": "tool_calls" # 结束原因：调用工具
12        },
13        id="lc_run--019c6f4e-9c10-7ae0-966a-5788f78b2017-0",
14        tool_calls=[ # 模型决定调用的工具列表
15          {
16            "name": "task", # 工具/子任务名
17            "args": { # 工具参数
18              "subagent_type": "weather_helper",
19              "description": "查询北京今天的天气情况。"
20            },
21            "id": "call_232308d358d64454905543",
22            "type": "tool_call"
23          }
24        ],
25        invalid_tool_calls=[],
26        usage_metadata={"input_tokens": 5265, "output_tokens": 30}
27      ]
28    }
29  }
30 }
```

子智能体：name="task", args 含 subagent_type/description；

自定义工具：name=工具名, args 含工具自有参数（如 query）；

场景 3：模型后置钩子（after_model 节点）

节点名：TodoListMiddleware.after_model

核心含义：模型执行完成的空钩子（无实际业务数据）

```
1 {
2   "TodoListMiddleware.after_model": None
3 }
```

场景 4：工具 / 子代理执行（tools 节点）

节点名：tools

核心含义：执行工具调用，返回外部数据（如天气、搜索结果）

```
1 {
2   "tools": {
3     "messages": [
4       ToolMessage( # 工具消息对象
5         content='{"query": "DeepAgents", "results": [{"url": "...", "title": "deepagents - PyPI", "content": "..."}]}' , # 实际返回的是 JSON 字符串
6         name="internet_search", # 对应调用的工具名 (例如 internet_search)
7         id="81d0bddd-30de-4874-baac-0bca8aa38936",
8         tool_call_id="call_232308d358d64454905543" # 关联模型调用的工具ID
9       )
10    ]
11  }
12 }
```

场景 5：模型生成最终回答（model 节点）

节点名：model

核心含义：模型基于工具结果，生成自然语言最终回答

```
1 {
2   "model": {
3     "messages": [
4       AIMessage(
5         content="今天北京的天气晴朗，气温25度，非常适合出游。", # 最终回答内容
6         additional_kwargs={"refusal": None},
7         response_metadata={
8           "token_usage": {"completion_tokens": 17, "prompt_tokens": 5318, "total_tokens": 5335},
9           "finish_reason": "stop" # 结束原因：正常完成
10        },
11         id="lc_run--019c6f4e-af53-7ca3-aee6-9f386be2ac78-0",
12         tool_calls=[], # 无工具调用 (已完成回答)
13         usage_metadata={"input_tokens": 5318, "output_tokens": 17}
14       )
15    ]
16  }
17 }
```

1.5 子代理Subagents和多智能体

指南：<https://www.anthropic.com/engineering/building-effective-agents>

1.5.1 多智能体理解

多 Agent 系统（Multi-Agent System, MAS）是由多个具备**自主性、反应性、目标导向性**的智能体（Agent）组成的协作体系，通过标准化通信与协同机制，共同完成单一智能体无法独立应对的复杂任务。

简单解释，就是将复杂任务，拆解成多个子任务，分发给专长的Agent进行处理，最后综合结果！本质**分而治之**！！

维度	单体模型（注意力稀释法则）	多智能体（分而治之的极效）
核心问题	同一个模型需处理多领域知识（如医学 + 法律），不同领域信息互相污染，推理能力断崖式下跌	将任务物理拆解，由专业 Agent 分别处理独立并行子任务，多方处理独立任务，性能优势极大提升！
组织类比	一人全栈（精力分散、专业度不足）	专业敏捷团队（分工明确、各司其职）
核心逻辑	注意力资源被多领域任务稀释，导致认知过载	分布式算力 + 专业化分工，突破单体模型的物理天花板

1.5.2 多智能体弊端

多智能体系统在实现**分而治之的性能飞跃**的同时，必然伴随两大**灾难级代价**，成为落地的核心阻碍：

Token 消耗的指数级失控

单体模型仅需承担自身推理的 Token 成本，而多 Agent 系统的核心交互逻辑是**Agent 间的上下文互通**。当多个 Agent 为完成协同任务，频繁互传长文本上下文、进行多轮循环复读时，Token 消耗会呈指数级暴涨。

更致命的是，无约束的群聊式交互可能在极短时间内爆 API 额度，直接导致**成本失控**，甚至成为中小团队落地多 Agent 的核心经济门槛。应对这一代价的关键，在于建立**拦截基准线**——对简单任务直接拒绝多 Agent 协作，强制使用单 Agent 或者 标准图流程，从源头控制交互规模。

调试成为噩梦：非确定性与全链路黑箱

多 Agent 系统的“涌现性”同时带来了**不可控性**：Agent 间的交互是非线性的，系统崩溃并非单一节点故障，而是像城市交通连环撞车一样，由多步交互的连锁反应引发，且难以复现。

传统的单 Agent 日志调试方式完全失效，若未搭建**全链路追踪（Tracing）**体系，出现问题时既无法定位根因，也无法复盘交互过程，直接导致系统上线后风险极高。因此，“不建全链路追踪录像，绝不上线”成为多 Agent 落地的**硬性底线**，这也是保障系统稳定性与可维护性的核心前提。

代价类型	核心问题	具体表现	风险阈值 / 特征	应对底线 / 策略
账单被击穿	Token 消耗失控	Agent 间频繁互传长文本上下文，Token 消耗呈 指数级增长 ，无节制的交互对话极易快速耗尽 API 额度	短时间内激增	建立拦截基准线，简单任务严禁触发多智能体流程，控制交互文本长度
调试成为噩梦	系统非确定性与不可追溯	群智网络伴随“非确定性”，系统崩溃场景复杂且无规律，难以定位根因，如同城市红绿灯瘫痪引发连环事故	多 Agent 交互出现不可复现的异常、全链路无日志追踪	不建全链路追踪录像（Tracing）绝不上线，强制记录交互日志

1.5.3 用多智能体的三条铁律

这三条铁律本质上是**多智能体系统的“入场许可”**，只有满足其中至少一条，才值得我们承担多 Agent 带来的成本与复杂度代价：

- 问题极度开放**：这类任务没有标准答案或固定流程，比如“制定企业年度战略”“开放式科研探索”，单体模型容易陷入局部最优，而多智能体可以通过多角色探索不同方向，动态调整路径。
- 存在领域冲突**：当任务需要跨两个及以上专业领域时（比如“医疗 + 法律”“金融 + 工程”），单体模型的注意力会被分散，导致推理精度断崖式下跌。多智能体通过物理隔离不同领域 Agent，避免知识污染，保障各模块专业度。
- 需要多方向并行**：任务天然可以拆分为多个互不依赖的子任务（比如“多源数据采集”“多版本方案并行设计”），多智能体可以利用分布式算力并行执行，大幅缩短整体耗时，实现“1+1>2”的效率提升。

只有符合这些条件时，多智能体才是“宝贝”；否则，强行使用只会让系统陷入“毒药”般的成本与调试困境。

铁律名称	核心场景	详细说明
问题极度开放	高复杂度、无固定路径任务	问题复杂度过高，无法事前硬编码死路径，需要在执行过程中灵活转向、探索旁支
存在领域冲突	多领域混杂任务	当混淆领域达到两个以上时，单体模型会因注意力稀释导致表现下滑，必须物理隔离不同领域专家的推理上下文
需要多方向并行	天然可拆解为独立路径的任务	任务天然要求沿多条独立路径同时并行推进，采用多体架构并行处理可带来极显著的性能耗时收益

1.5.4 多智能两种架构模式

模式一：层级工作流 (Hierarchical / Orchestrator-Workers)

别名：指挥官模式、主从模式 核心逻辑：中央集权。有一个“大脑”负责思考和分派任务，其他 Agent 只是干活的手。

- 运作方式：
 - 输入：用户给出一个复杂任务（比如“写一份新产品的上市策划案”）。
 - 主脑 (Orchestrator)：主管 Agent 收到任务，它不直接干活，而是分析任务，把它拆解成几个子任务（如：市场调研、创意设计、文案撰写）。
 - 分发：主管把子任务分发给对应的 垂直领域专家 Agent（Worker）。
 - 执行：Worker Agent 并行或串行工作，产出结果返还给主管。
 - 整合：主管汇总所有结果，整合成最终报告输出。
- 优点：
 - 可控性强：主脑掌控全局，知道进度，方便纠错。
 - 逻辑清晰：上下级关系明确，就像传统的公司组织架构。
- 缺点：
 - 单点故障：主脑如果挂了或判断失误，整个任务就崩了。
 - 通信瓶颈：所有信息都要经过主脑中转。



模式二：协作 workflow (Collaborative / Network)

别名：网状模式、专家会诊模式 核心逻辑：去中心化。没有绝对的领导，大家都是平等的专家，坐在一起开会讨论，互相交换信息。

- 运作方式：
 - 输入：用户给出一个开放性问题（比如“评估这家公司的投资价值”）。
 - 共享：任务被扔到一个“共享会议室”（Shared State/Context）。
 - 自组织：不同的专家 Agent（定价、产品、财务、合规）根据自己的专长，从“会议室”里拿取信息，进行分析。
 - 交互：Agent 之间可以直接交流。比如财务 Agent 算出成本太高，直接告诉定价 Agent 调整价格，不需要经过领导批准。
 - 收敛：最后通过一个 评估者 (Evaluator) 或规则来决定什么时候讨论结束，输出最终方案。
- 优点：
 - 灵活性极高：适合解决极其复杂、没有标准答案的问题。
 - 涌现能力：不同的专家碰撞可能产生意想不到的创新解法。
- 缺点：
 - 容易失控：Agent 之间可能陷入无休止的争论（死循环）。
 - 难以调试：很难追踪到底是谁做出的关键决策。



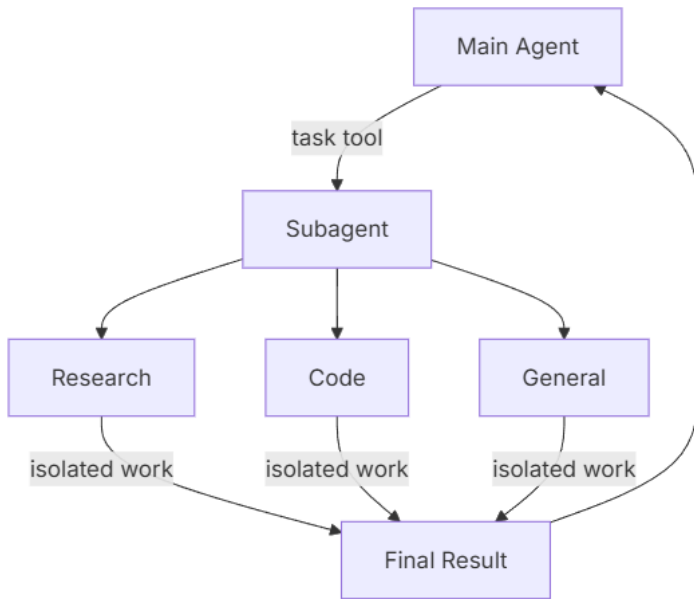
DeepAgents 是典型的**层级/指挥官模式**，而 AutoGen 是典型的**去中心化/网状协作模式**（像群聊头脑风暴），CrewAI可以构建任意一种模式。

框架	核心模式	协作形态	适用场景	复杂度
DeepAgents (MetaGPT)	层级 workflow	流水线 ：角色分工明确，按预定义顺序（如：PM->架构师->工程师）传递任务。	标准化流程 ：如软件开发全流程、长篇报告撰写。	中等
AutoGen	协作 workflow	群聊/网状 ：所有 Agent 在一个群里，根据上下文自动接话，自由交互。	开放性探索 ：多角色头脑风暴、复杂问题求解、代码自动修正。	低 (开箱即用)
CrewAI	混合模式	层级+委派 ：主要是层级任务，但允许 Agent 自主委派子任务给别人。	通用任务 ：既有流程控制，又需要一点灵活性的场景。	低

1.5.5 DeepAgents子代理入门

<https://docs.langchain.com/oss/python/deepagents/subagents#configuration>

深度代理可以创建子代理来委派工作。你可以在 `子代理` 参数中指定自定义子代理。子代理用于上下文隔离（保持主代理上下文的干净）以及提供专业指令。



子代理解决了**上下文膨胀问题**。当代理使用输出较大的工具（如网页搜索、文件读取、数据库查询）时，上下文窗口会迅速被中间结果填满。子代理将这些详细工作隔离开来——主代理只接收最终结果，而非产生该结果的数十个工具调用。

什么时候使用SubAgent：

- 多步骤任务会让主代理的上下文变得杂乱
- 有需要 “专业技能 / 专属工具” 的环节

比如主代理要做 “股票分析”，其中 “基本面分析” 需要财务工具、“技术面分析” 需要 K 线工具，给这两个环节配专属子代理（带对应工具）

- 需要不同模型能力的任务（多模态）
- 当你想让主Agent专注于高层协调时

什么时候不应使用SubAgent：

- 任务简单，一步就能干完
- 需要中间信息连贯，不能拆

比如 “读一篇文章，然后总结核心观点”，拆给子代理读、再拆给另一个子代理总结，会丢上下文，不如主代理一次性干完。

- 当运营费用超过收益时

Subagent配置方式: 子代理 配置有两种方案**词典**或**CompiledSubAgent** 对象。

将子代理定义为包含以下字段的词典：

字段名	类型	必填 / 可选	核心描述	继承规则（与主代理的关系）
name	str	必填	子代理的唯一标识；主代理调用 task() 工具时会使用该名称，也会作为 AIMessage / 流式输出的元数据，用于区分不同代理	-（无继承，需自定义）
description	str	必填	子代理的职能描述（需具体、以行动为导向）；主代理会根据此信息判断是否将任务委派给该子代理	-（无继承，需自定义）
system_prompt	str	可选	子代理的执行指令，需包含工具使用指导、输出格式要求等核心规则	不继承主代理的，需自定义
tools	list[Callable]	可选	子代理可使用的工具列表；建议极简配置，仅保留必要工具	不继承主代理的，需自定义
model	str BaseChatModel	可选	子代理使用的模型：1. 传字符串（如 openai:gpt-5）2. 传 LangChain 模型对象（如 init_chat_model("gpt-5"）省略则使用主代理的模型	默认继承主代理的模型，自定义会覆盖默认值
middleware	list[Middleware]	可选	自定义中间件，用于实现日志记录、速率限制、自定义行为等功能	不继承主代理的，需自定义
interrupt_on	dict[str, bool]	可选	为特定工具配置 “人机协作流程（HITL）”；需搭配检查点（checkpointer）使用	-
skills	list[str]	可选	技能文件的来源路径（如 ["/skills/research/"]），用于加载子代理专属技能	-

示例：创建一个主智能体，它拥有三个助手：

1. **天气助手：** 查询天气（固定返回“晴朗”）。
2. **计算助手：** 处理数学问题。
3. **翻译助手：** 负责中英互译。

代码实现：

```
1 from langchain.chat_models import init_chat_model
2 from deepagents import create_deep_agent
3 import os
4 from dotenv import load_dotenv, find_dotenv
5 import json
6
7 load_dotenv(find_dotenv())
8
9 # 极简初始化（自动读取OPENAI环境变量）
10 llm = init_chat_model(
11     model=os.getenv("LLM_QWEN_MAX"),
12     temperature=0.1, # 自定义温度（更严谨的回答）
13     model_provider="openai"
14 )
15
16 # 1. 定义子智能体：天气助手
17 weather_agent = {
18     "name": "weather_helper",
19     "description": "用于查询天气信息。当用户询问天气时，请调用此助手。",
20     "system_prompt": "你是一个天气助手。无论用户问哪个城市的天气，你都统一回答：'今日天气晴朗，气温 25 度，适合出游。'",
21     "tools": [] # 这里不需要额外工具，仅靠 prompt 回复
22 }
23
24 # 2. 定义子智能体：计算助手
25 math_agent = {
26     "name": "math_helper",
```

```
27     "description": "用于处理数学计算问题。",
28     "system_prompt": "你是一个严谨的数学助手。请帮助用户计算数学问题。",
29     "tools": []
30 }
31
32 # 3. 定义子智能体：翻译助手
33 translate_agent = {
34     "name": "translator",
35     "description": "用于中英互译任务。",
36     "system_prompt": "你是一个翻译助手。如果是中文请翻译成英文，如果是英文请翻译成中文。",
37     "tools": []
38 }
39
40 # 4. 创建主智能体，并注册子智能体
41 main_agent = create_deep_agent(
42     model=llm,
43     tools=[], # 主智能体本身不带工具，依靠子智能体
44     subagents=[weather_agent, math_agent, translate_agent],
45     system_prompt="你是一个全能管家。你会根据用户的需求，调度不同的助手来解决问题。"
46 )
47
48
49 # 5. 可视化运行（Stream）
50 # 使用 stream() 替代 invoke()，可以实时打印出智能体的“调度”过程，看到它如何分发任务
51 def test_stream(query):
52     print(f"\n>>> 提问: {query}")
53     # 遍历流式输出
54     for chunk in main_agent.stream({"messages": [{"role": "user", "content": query}]}):
55         # chunk 是一个字典，键是节点名（如 'model', 'tools'），值是该节点的状态更新
56         for node_name, state in chunk.items():
57             if not state or "messages" not in state: continue
58             messages = state["messages"]
59             if messages and isinstance(messages, list):
60                 last_msg = messages[-1]
61                 # 1. 模型节点（model）：决定下一步行动
62                 if node_name == "model":
63                     # 如果有 tool_calls，说明模型决定调用工具或子智能体
64                     if last_msg.tool_calls:
65                         for tool_call in last_msg.tool_calls:
66                             if tool_call['name'] == 'task':
67                                 sub_agent = tool_call['args'].get('subagent_type')
68                                 print(f"[模型决策] 呼叫子智能体: {sub_agent}")
69                             else:
70                                 print(f"[模型决策] 调用工具: {tool_call['name']}, 参数为: {tool_call['args']}")
71                     # 如果没有 tool_calls 且有 content，说明是最终回复
72                     elif last_msg.content:
73                         print(f"[最终回复] {last_msg.content}")
74
75                 # 2. 工具节点（tools）：显示工具/子智能体的执行结果
76                 elif node_name == "tools":
77                     content_preview = ''
78                     if len(last_msg.content) > 100:
79                         # 取前100个字符 + 省略号（截断预览）
80                         content_preview = last_msg.content[:100] + "..."
81                     else:
82                         # 内容较短，直接完整显示
83                         content_preview = last_msg.content
84                     print(f"[执行结果] {content_preview}")
85 test_stream("北京今天天气怎么样？")
86 test_stream("100 + 256 等于多少？")
```

原理解析：

- subagents 参数接收一个列表，每个元素是一个字典，定义了子智能体的配置。
- description 非常关键：主智能体通过这段描述来判断何时调用该子智能体。
- 当主智能体发现用户意图匹配某个子智能体的 description 时，会自动生成一个 task 工具调用，将任务分发下去。

特别注意：主子Agent之间上下文默认隔离

- 独立 Prompt：每个 Agent 都有自己独立的 system_prompt，定义了它是谁，负责什么。
- 独立工具集 (Skills/Tools)：子 Agent 只能使用分配给它自己的工具，通常不能直接调用父 Agent 的工具，反之亦然。
- 独立记忆 (Memory/State)：子 Agent 在执行任务时产生的临时对话历史、变量状态，通常只在它自己的生命周期内有效，执行完向父 Agent 汇报结果后，这些中间过程可能不会全部同步给父 Agent（除非通过特定的返回值传递）。
这种设计的目的：
 - 专注：防止上下文污染。比如负责写代码的 Agent 不需要知道负责写文案的 Agent 的具体指令。
 - 安全：限制工具权限。比如只有顶层 Agent 能批准发布，底层 Agent 只能提交代码。
 - 模块化：方便独立测试和复用子 Agent。

也可以调整成异步执行：

- 高并发服务**：用 FastAPI/Starlette 做接口时（比如给前端返回流式回答），`astream()` + 异步能同时处理成百上千个用户请求，不会因单个请求阻塞整个服务；
- 批量处理任务**：需要同时调用智能体处理多个查询（比如你测试的 3 个问题），`astream()` 并发执行耗时~最长单个任务，比同步 `stream()` 串行快几倍；
- 非阻塞主线程**：在 GUI 程序（如 PyQt/Tkinter）、定时任务中调用智能体，`astream()` 异步执行不会让界面卡死 / 定时任务中断。

```
1 from langchain.chat_models import init_chat_model
2 from deepagents import create_deep_agent
3 import os
4 import asyncio # 新增：导入异步库
5 from dotenv import load_dotenv, find_dotenv
6 import json
7
8 load_dotenv(find_dotenv())
9
10 # 极简初始化（自动读取OPENAI环境变量）
11 llm = init_chat_model(
12     model=os.getenv("LLM_QWEN_MAX"),
13     verbose=True, # 自定义参数
14     temperature=0.1, # 自定义温度（更严谨的回答）
15     model_provider="openai"
16 )
17
18 # 1. 定义子智能体：天气助手（和原来一致）
19 weather_agent = {
20     "name": "weather_helper",
21     "description": "用于查询天气信息。当用户询问天气时，请调用此助手。",
22     "system_prompt": "你是一个天气助手。无论用户问哪个城市的天气，你都统一回答：'今日天气晴朗，气温 25 度，适合出游。'",
23     "model": llm,
24     "tools": []
25 }
26
27 # 2. 定义子智能体：计算助手（和原来一致）
28 math_agent = {
29     "name": "math_helper",
30     "description": "用于处理数学计算问题。",
31     "system_prompt": "你是一个严谨的数学助手。请帮助用户计算数学问题。",
32     "model": llm,
33     "tools": []
34 }
35
36 # 3. 定义子智能体：翻译助手（和原来一致）
37 translate_agent = {
38     "name": "translator",
39     "description": "用于中英互译任务。",
40     "system_prompt": "你是一个翻译助手。如果是中文请翻译成英文，如果是英文请翻译成中文。",
41     "model": llm,
42     "tools": []
43 }
44
45 # 4. 创建主智能体（和原来一致）
46 main_agent = create_deep_agent(
47     model=llm,
48     tools=[],
49     subagents=[weather_agent, math_agent, translate_agent],
```

```
50     system_prompt="你是一个全能管家。你会根据用户的需求，调度不同的助手来解决问题。"
51 )
52
53
54 # 5. 异步版本：适配 astream() (核心修改)
55 async def test_astream(query): # 新增 async 定义协程函数
56     print(f"\n>>> 提问: {query}")
57     # 核心修改：同步 for → 异步 async for
58     async for chunk in main_agent.astream({"messages": [{"role": "user", "content": query}]}):
59         for node_name, state in chunk.items():
60             if not state or "messages" not in state: continue
61             messages = state["messages"]
62             if messages and isinstance(messages, list):
63                 last_msg = messages[-1]
64                 # 1. 模型节点逻辑 (和原来一致)
65                 if node_name == "model":
66                     if last_msg.tool_calls:
67                         for tool_call in last_msg.tool_calls:
68                             if tool_call['name'] == 'task':
69                                 sub_agent = tool_call['args'].get('subagent_type')
70                                 print(f"[模型决策] 呼叫子智能体: {sub_agent}")
71                             else:
72                                 print(f"[模型决策] 调用工具: {tool_call['name']}, 参数为: {tool_call['args']}")
73                         elif last_msg.content:
74                             print(f"[最终回复] {last_msg.content}")
75                 # 2. 工具节点逻辑 (和原来一致)
76                 elif node_name == "tools":
77                     content_preview = ''
78                     if len(last_msg.content) > 100:
79                         content_preview = last_msg.content[:100] + "..."
80                     else:
81                         content_preview = last_msg.content
82                     print(f"[执行结果] {content_preview}")
83
84 # 6. 执行异步函数 (新增)
85 if __name__ == "__main__":
86     # 执行单个查询
87     # asyncio.run(test_astream("北京今天天气怎么样?"))
88     # 也可以并发执行多个查询 (协程核心优势)
89     async def batch_run():
90         # 并发执行2个查询
91         task1 = test_astream("北京今天天气怎么样?")
92         task2 = test_astream("100 + 256 等于多少?")
93         task3 = test_astream("将 你好 翻译成 英文?")
94         await asyncio.gather(task1, task2, task3)
95
96     # # 执行包装后的协程
97     asyncio.run(batch_run())
```

了解扩展：DeepAgents 框架支持配置子代理（Subagent）的嵌套，即子代理可以拥有自己的子代理。

1. 无限嵌套结构：

- 你可以创建一个 Main Agent。
- 给它配置一个 Subagent A。
- 而这个 Subagent A 本身也可以配置它自己的 Subagent B。
- 这种结构理论上支持多层嵌套（Agent -> Subagent -> Sub-Subagent）。

2. 配置方式：

- 在创建代理时，通过 subagents 参数传入子代理列表。
- 如果某个子代理也需要下级，就在定义该子代理时，同样配置它的 subagents 参数。

代码示例 (Python)：假设你要构建一个 公司层级 的代理系统：CEO -> 技术总监 (CTO) -> 工程师 (Coder)。

```
1 import os
2 from langchain.chat_models import init_chat_model
3 from deepagents import create_deep_agent
4 from dotenv import load_dotenv, find_dotenv
5
```

```
6 # 加载环境变量
7 load_dotenv(find_dotenv())
8
9 llm = init_chat_model(
10     model=os.getenv("LLM_QWEN_MAX"),
11     model_provider="openai"
12 )
13
14 # -----
15 # 强制层级委派的关键配置
16 # -----
17
18 # 1. 底层 Coder 配置
19 # 职责明确：只有他能写代码
20 coder_config = {
21     "name": "Coder",
22     "description": "高级Python工程师，他是唯一有权限编写具体代码的人。",
23     "system_prompt": "你是一个高级Python工程师。你的职责是接收具体的编码任务并实现它。请使用 write_file 工具编写代码。",
24     "tools": [], # Coder 拥有默认的文件操作工具
25 }
26
27 # 2. 中间层 CTO 配置
28 # 职责明确：承上启下，必须指挥 Coder
29 cto_config = {
30     "name": "CTO",
31     "description": "技术总监，负责将战略需求转化为技术任务并分配给工程师。",
32     # 关键修改：明确告诉 CTO 不要自己写代码，必须找 Coder
33     "system_prompt": """"你是技术总监。
34 注意：你没有编写代码的权限！
35 你的职责是：
36 1. 分析 CEO 的需求。
37 2. 设计技术方案。
38 3. 调用 'Coder' 子代理来完成具体的代码编写工作。
39 """,
40     "tools": [],
41     "subagents": [coder_config]
42 }
43
44 # 3. 顶层 CEO 配置
45 # 职责明确：只负责战略，禁止干具体的活
46 ceo_agent = create_deep_agent(
47     model=llm,
48     name="CEO",
49     # 关键修改：明确告诉 CEO 不要自己动手，必须找 CTO
50     system_prompt=""""你是CEO，负责公司战略决策。
51 注意：你严禁直接编写代码或操作文件！
52 你必须将所有技术相关的开发任务委派给 'CTO' 处理。
53 你的工作是验收 CTO 提交的结果。
54 """,
55     subagents=[cto_config]
56 )
57
58 # 运行 CEO 代理
59 print(">>> 开始执行任务链...")
60 stream = ceo_agent.stream({
61     "messages": [
62         {"role": "user", "content": "帮我开发一个贪吃蛇游戏，要求用Python实现"}
63     ]
64 })
65
66 # 打印最终结果
67 print("\n>>> 最终结果：")
68 for chunk in stream:
69     print(chunk)
```

注意事项：虽然支持无限嵌套，但层级过深会导致调试困难和延迟增加。一般建议 2-3 层即可满足业务需求即可。

1.6 兼容Langgraph和Langchain

LangChain 生态下的其他 Agent（比如 AgentExecutor、ReActAgent、StructuredChatAgent 等）可以挂载为 DeepAgents 的子代理，但不能直接挂 —— 需要先把这些 Agent 封装成「符合 LangGraph StateGraph 规范的图」（核心是状态里包含 messages 键），再用 CompiledSubAgent 封装。

简单说：DeepAgents 认的是「LangGraph 格式的图」，不是直接认 LangChain Agent；但所有 LangChain Agent 都能转成 LangGraph 图，所以最终都能挂。

兼容Langgraph编译图

1. DeepAgents 子代理的“准入门槛”

DeepAgents 调度子代理的核心要求只有一个：子代理的执行逻辑必须是「带有 messages 键的状态图」（不管这个图是直接用 StateGraph 写的，还是从其他 Agent 转来的）。这个 messages 键就是你之前看 result 结构时的核心字段 ——DeepAgents 靠它传递对话、工具调用、执行结果，没有这个键就无法和主代理通信。

2. 演示LangGraph格式图挂载

```
1 import os
2 from typing import TypedDict, Annotated
3 from dotenv import load_dotenv, find_dotenv
4 from langchain_openai import ChatOpenAI
5 from langchain_core.messages import AIMessage, HumanMessage
6 from langgraph.graph import StateGraph, END
7 from langgraph.graph.message import add_messages
8 from deepagents import create_deep_agent, CompiledSubAgent
9 from langchain.chat_models import init_chat_model
10
11 # 加载环境变量
12 load_dotenv(find_dotenv())
13
14 # 我们将之前形式的agent 包裹到subagents [deepagent]
15 # 1.定义的state必须包含 messages属性 【deepagents】
16 # 2.
17
18 # --- 1. 定义子智能体（基于 StateGraph） ---
19
20 # 定义 State（必须包含 messages）
21 class SubState(TypedDict):
22     messages: Annotated[list, add_messages]
23
24
25 # 定义节点逻辑（增加打印语句，证明它被触发了）
26 def processing_node(state: SubState):
27     print("\n    >>> [子智能体内部] 收到任务，正在处理...")
28
29     # 获取主智能体传来的任务描述
30     last_msg = state["messages"][-1]
31     print(f"    >>> [子智能体内部] 输入内容：{last_msg.content}")
32
33     # 模拟处理逻辑
34     result_text = f"【已由Graph处理】经核查，业务逻辑处理完毕。原始内容：{last_msg.content}"
35
36     print(f"    >>> [子智能体内部] 处理完成，准备返回。\\n")
37     return {"messages": [AIMessage(content=result_text)]}
38
39
40 # 构建图
41 workflow = StateGraph(SubState)
42 workflow.add_node("worker", processing_node)
43 workflow.set_entry_point("worker")
44 workflow.add_edge("worker", END)
45 compiled_graph = workflow.compile()
46
47 # 封装为 CompiledSubAgent
48 sub_agent_config = CompiledSubAgent(
49     name="complex_worker",
50     description="处理复杂业务逻辑、核查任务的子智能体。当用户提到'复杂业务'或'核查'时调用。",
51     runnable=compiled_graph
```

```
52 )
53
54 # --- 2. 创建主智能体 ---
55
56 llm = init_chat_model(
57     model=os.getenv("LLM_QWEN_MAX"),
58     model_provider="openai"
59 )
60
61 deep_agent = create_deep_agent(
62     model=llm,
63     subagents=[sub_agent_config],
64     system_prompt="你是一个协调员。遇到复杂任务时，必须调用 complex_worker 子智能体处理。"
65 )
66
67 # --- 3. 运行测试（带漂亮的日志） ---
68
69 if __name__ == "__main__":
70     query = "请帮我处理这个复杂业务：核对用户 ID 9527 的数据。"
71     print(f"User: {query}")
72     print("=" * 60)
73
74     # 使用 stream 并解析结果
75     for chunk in deep_agent.stream({"messages": [HumanMessage(content=query)]}):
76         print(f"chunk结果:: {chunk}")
```

兼容Langchain单智能体

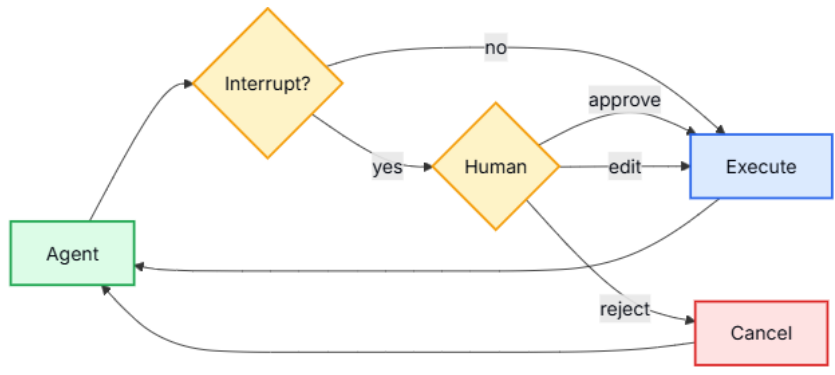
```
1 import os
2 from langchain.chat_models import init_chat_model
3 from langchain.agents import create_agent
4 from langchain_core.tools import tool
5 from deepagents import create_deep_agent, CompiledSubAgent
6 from dotenv import load_dotenv, find_dotenv
7
8 # 加载环境变量
9 load_dotenv(find_dotenv())
10
11 # 1. 初始化模型
12 llm = init_chat_model(
13     model=os.getenv("LLM_QWEN_MAX"),
14     model_provider="openai"
15 )
16
17
18 @tool
19 def get_weather(city: str) -> str:
20     """查询指定城市的天气"""
21     return f"{city}的天气是晴朗，25度"
22
23 # Create a custom agent
24 agent = create_agent(
25     model=llm,
26     tools=[get_weather]
27 )
28
29 # Use it as a custom subagent
30 custom_subagent = CompiledSubAgent(
31     name="subagent",
32     description="子任务，可以调用天气工具，查询天气信息！",
33     runnable=agent
34 )
35
36 subagents = [custom_subagent]
37
38 deep_agent = create_deep_agent(
39     model=llm,
40     tools=[],
41     system_prompt="你是一个智能助手，主要调用子代理实现功能，你只做任务分配，可以调用subagent实现功能！！",
42     subagents=[custom_subagent]
```

```
43 )
44
45 result = deep_agent.invoke({
46     "messages": [
47         {"role": "user", "content": "查询北京的天气！"}]
48 })
49
50 print(f"最终结果{result['messages'][-1].content}")
```

1.7 流程人机交互 (HITL)

<https://docs.langchain.com/oss/python/deepagents/human-in-the-loop>

有些工具作可能比较敏感，需要人工批准才能执行。深度代理通过 LangGraph 的中断功能支持人机参与的工作流程。您可以使用 `interrupt_on` 参数配置哪些工具需要批准。



1.7.1 交互步骤说明

步骤1：设置tool是否进行人工互动

根据不同工具的风险等级配置

```
1 # create_deep_agent的属性，配置工具是否需要人工互动
2 deep_agent = create_deep_agent(
3     model = ""
4     tools = [a,b,c,d]
5     subagents = []
6     interrupt_on = {
7         # 工具名：{需要人工审核动作！同意，编辑（同意但是修改工具方法参数），拒绝}
8         "delete_file": {"allowed_decisions": ["approve", "edit", "reject"]},
9         "a": True,
10        "write_file": {"allowed_decisions": ["approve", "reject"]},
11        # 不需要审批，默认就是False可以不写
12        "read_file": True,
13        "list_files": False,
14    } )
```

步骤2：配置检查点

人工互动需要一个检查点，在中断和恢复之间保持代理状态：

```
1 from langgraph.checkpoint.memory import MemorySaver
2
3 checkpointer = MemorySaver()
4 agent = create_deep_agent(
5     tools=[...],
6     interrupt_on={...},
7     checkpointer=checkerpointer # 必须，否则没有进度！
8 )
```

步骤3：设置相同的thead_id

恢复时，必须使用相同的配置和相同的 `thread_id`

```
1 # 第一次调用，不会立即执行，会判断是否有中断动作
2 config = {"configurable": {"thread_id": "my-thread"}}
3 result = agent.invoke(input, config=config)
4
5 # 第二次调用，配置相同的线程id以及审批行为，最终执行
6 # 必须是相同的线程id才能确保同一个agent线程状态执行
7 result = agent.invoke(Command(resume={...}), config=config)
```

步骤4：行动决策配置

决策列表必须符合以下 `action_requests` 顺序：

```
1 # 获取结果中是否存在打断状态！存在就获取人工参与再执行！
2 if result.get("__interrupt__"):
3     interrupts = result["__interrupt__"][0].value
4     action_requests = interrupts["action_requests"]
5
6     # 根据tool顺序，处理设置决策，是否执行还是编辑
7     decisions = []
8     for action in action_requests:
9         decision = get_user_decision(action) # 我们定义得逻辑方法！
10        decisions.append(decision)
11
12    result = agent.invoke(
13        Command(resume={"decisions": decisions}),
14        config=config
15    )
```

字段	含义
action_requests	需要审批的操作列表（如删库 delete_database、删文件 delete_file），包含操作名、参数、风险描述! {'action_requests': [{'name': 'delete_table', 'args': {'tablename': 'users'}, 'description': "描述"}
review_configs	允许的审批操作（approve 同意 / reject 拒绝 / edit 编辑参数)
id	中断会话唯一标识（确保恢复时匹配同一个会话)

1.7.2 中断交互

当代理调用多个需要批准的工具时，所有中断都会被批量处理成一个中断。必须按顺序为每个动作提供决策

```
1 # -*- coding: utf-8 -*-
2 """
3 DeepAgents 中断审批机制示例
4 核心功能：演示高危工具调用前的人工审批流程，支持删除数据库表/文件的审批控制
5 """
6 import os
7 from langchain.chat_models import init_chat_model
8 from langchain.tools import tool
9 from deepagents import create_deep_agent
10 from langgraph.checkpoint.memory import InMemorySaver # 内存检查点，用于保存中断状态
11 from langgraph.types import Command # 恢复执行的指令类型
12 from dotenv import load_dotenv, find_dotenv
13
14 # 加载环境变量（DASHSCOPE_API_KEY等），优先查找当前目录的.env文件
15 load_dotenv(find_dotenv())
16
17
18 # ===== 1. 定义工具函数 =====
19 # 装饰器@tool将普通函数转为LangChain可调用工具，函数文档字符串会作为工具描述给Agent
20 @tool
21 def delete_database(table_name: str):
22     """
23     高危操作：删除数据库表
24     :param table_name: 要删除的表名
25     :return: 操作结果提示
26     """
27     print(f"[工具执行] 删除表: {table_name}")
28     return f"已成功删除表: {table_name}"
29
30
31 @tool
32 def select_data(table_name: str):
33     """
34     普通操作：查询指定表名的数据（无需审批）
35     :param table_name: 要查询的表名
36     :return: 操作结果提示
37     """
38     print(f"[工具执行] 查询指定表名数据: {table_name}")
39     return f"查询数据成功: {table_name}"
40
41
```

```
42 @tool
43 def delete_file(file_name: str):
44     """
45     高危操作：删除文件
46     :param file_name: 要删除的文件路径/名称
47     :return: 操作结果提示
48     """
49     print(f"[工具执行] 删除文件: {file_name}")
50     return f"已成功删除文件: {file_name}"
51
52
53 # ===== 2. 核心配置 =====
54 # 配置检查点（必须）：保存Agent中断时的状态，确保恢复执行时能衔接上下文
55 # 注意：InMemorySaver仅用于测试，生产环境建议使用RedisCheckpointter等持久化方案
56 checkpointer = InMemorySaver()
57
58 # 初始化大模型（通义千问）
59 llm = init_chat_model(
60     model=os.getenv("LLM_QWEN_MAX"),          # 模型名称（从环境变量读取）
61     model_provider="openai"                   # 兼容OpenAI格式的接口
62 )
63
64 # 创建DeepAgents智能体（核心配置）
65 deep_agent = create_deep_agent(
66     model=llm,                                # 绑定大模型
67     tools=[delete_database, delete_file, select_data], # 注册可用工具
68     # 中断配置：指定调用以下工具前触发人工审批（高危操作管控）
69     interrupt_on={"delete_database": True, "delete_file": True},
70     checkpointer=checkpointer,                # 绑定检查点（中断恢复必备）
71     system_prompt="所有的回答都使用中文！！"  # 系统提示词，规范Agent输出语言
72 )
73
74 # ===== 3. 执行流程 =====
75 # 会话配置：通过thread_id绑定会话，确保中断/恢复在同一个会话中执行
76 thread_config = {"configurable": {"thread_id": "safe_thread_1"}}
77
78 print("\n=== 第一阶段：触发工具调用（规划阶段）===")
79 # 第一次调用：Agent会规划操作序列，但触发中断后不会执行任何工具
80 result_1 = deep_agent.invoke(
81     {
82         "messages": [
83             {
84                 "role": "user",
85                 "content": "先把用户表(users)删了！在查询product表数据！最后把 user.txt 文件也删除了！"
86             }
87         ]
88     },
89     config=thread_config # 绑定会话ID，确保状态可追溯
90 )
91 print("当前状态：智能体已暂停，等待人工确认。")
92
93 # 提取中断信息（核心：审批操作列表）
94 interrupts = result_1.get("__interrupt__")
95
96 if interrupts:
97     # 解析中断数据：Interrupt对象 → value字典 → action_requests列表
98     action_requests = interrupts[0].value['action_requests']
99     # 打印需要审批的操作数量和名称（用于人工审批界面展示）
100     print(f"需要审核动作数量{len(action_requests)}，输出结果: {[action_request['name'] for action_request in
    action_requests]} ")
101
102     # 模拟人工审批决策（生产环境需替换为人工交互/审批系统接口）
103     # 注意：decisions顺序必须和action_requests一致
104     decisions = [
105         {"type": "approve"}, # 审批结果1：同意删除数据库表（delete_database）
106         {"type": "reject"}   # 审批结果2：拒绝删除文件（delete_file）
107     ]
108
109     # 第二次调用：恢复执行，Agent会按审批结果执行操作
```

```
110     result = deep_agent.invoke(  
111         # Command(resume)是DeepAgents专用的恢复指令  
112         Command(resume={  
113             "decisions": decisions # 传入人工审批结果  
114         }),  
115         config=thread_config # 必须使用同一个thread_id, 否则无法恢复状态  
116     )  
117  
118     # 打印最终执行结果 (Agent的最终回复)  
119     print("\n=== 执行结果 ===")  
120     print(result["messages"][-1].content)
```

1.7.3 编辑参数

结合之前的删库 / 删文件场景，编写完整的「edit 审批处理」代码，包含**参数编辑**、**多操作编辑**、**执行验证**等核心逻辑，注释清晰可直接运行：

```
1  # -*- coding: utf-8 -*-  
2  """  
3  DeepAgents 中断审批-EDIT操作示例  
4  核心功能：演示人工编辑工具参数后恢复执行的完整流程  
5  """  
6  import os  
7  from langchain.chat_models import init_chat_model  
8  from langchain.tools import tool  
9  from deepagents import create_deep_agent  
10 from langgraph.checkpoint.memory import InMemorySaver  
11 from langgraph.types import Command  
12 from dotenv import load_dotenv, find_dotenv  
13  
14 load_dotenv(find_dotenv())  
15  
16  
17 # ===== 1. 定义工具函数 =====  
18 @tool  
19 def delete_database(table_name: str):  
20     """危险操作：删除数据库表"""  
21     print(f"[工具执行] 删除表: {table_name}")  
22     return f"已成功删除表: {table_name}"  
23  
24  
25 @tool  
26 def select_data(table_name: str):  
27     """查询指定表名的数据"""  
28     print(f"[工具执行] 查询指定表名数据: {table_name}")  
29     return f"查询数据成功: {table_name}"  
30  
31  
32 @tool  
33 def delete_file(file_name: str):  
34     """危险操作：删除文件"""  
35     print(f"[工具执行] 删除文件: {file_name}")  
36     return f"已成功删除文件: {file_name}"  
37  
38  
39 # ===== 2. 核心配置 =====  
40 checkpointer = InMemorySaver()  
41  
42 # 初始化LLM  
43 llm = init_chat_model(  
44     model=os.getenv("LLM_QWEN_MAX"),  
45     model_provider="openai"  
46 )  
47  
48 # 创建Agent  
49 deep_agent = create_deep_agent(  
50     model=llm,  
51     tools=[delete_database, delete_file, select_data],  
52     interrupt_on={"delete_database": True, "delete_file": True}, # 高危操作触发审批  
53     checkpointer=checkerpoint, # 使用内存检查点器
```

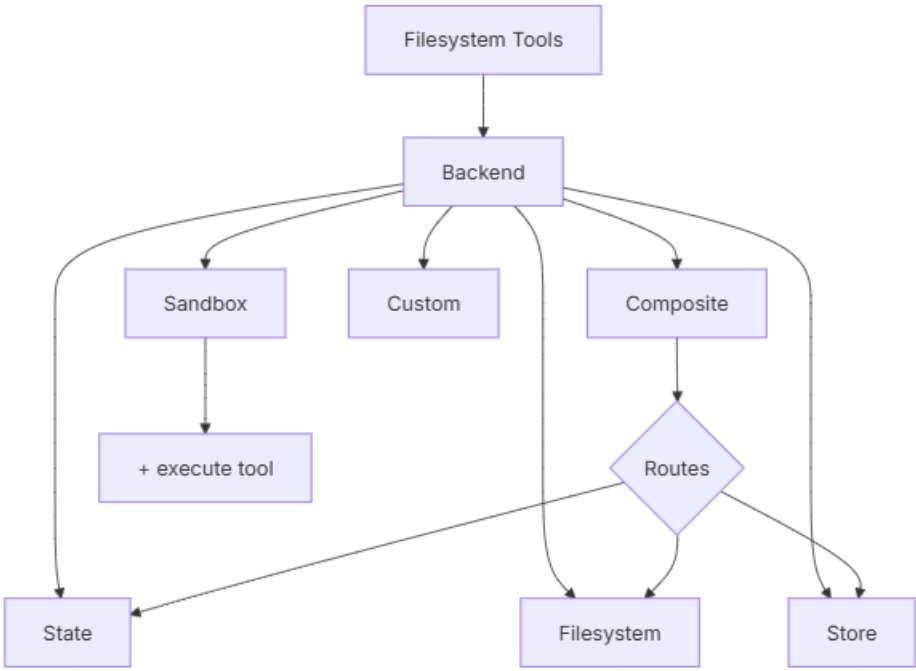
```
54     system_prompt="所有的回答都使用中文！严格按照审批后的参数执行工具操作！"
55 )
56
57 # ===== 3. EDIT审批核心逻辑 =====
58 # 会话配置
59 thread_config = {"configurable": {"thread_id": "edit_safe_thread_1"}}
60
61 print("\n=== 第一阶段：触发中断（获取原始操作参数）===")
62 # 第一次调用：触发中断，获取Agent规划的原始操作参数
63 result = deep_agent.invoke(
64     {
65         "messages": [
66             {
67                 "role": "user",
68                 "content": "删除users表！删除/user.txt文件！"
69             }
70         ]
71     },
72     config=thread_config
73 )
74
75 # 检测中断并处理EDIT审批
76 if result.get("__interrupt__"):
77     # 1. 解析中断数据（提取原始操作参数）
78     interrupts = result["__interrupt__"][0].value
79     action_requests = interrupts["action_requests"]
80
81     print(f"\n=== 待审批操作列表 ===")
82     for idx, action in enumerate(action_requests):
83         print(f"操作{idx + 1} - 工具名: {action['name']}, 原始参数: {action['args']}")
84
85     # 2. 模拟人工编辑参数（核心：EDIT操作）
86     # 场景：
87     # - delete_database：原始参数users → 编辑为test_users（避免删正式表）
88     # - delete_file：原始参数/user.txt → 编辑为/tmp/test.txt（避免删核心文件）
89     decisions = []
90     for action in action_requests:
91         if action["name"] == "delete_database":
92             # 编辑删库参数：仅删除测试表
93             decisions.append({
94                 "type": "edit", # 审批类型：编辑参数
95                 "edited_action": {
96                     "name": action["name"], # 必须保留工具名
97                     "args": {"table_name": "test_users"} # 编辑后的参数
98                 }
99             })
100         elif action["name"] == "delete_file":
101             # 编辑删文件参数：仅删除临时文件
102             decisions.append({
103                 "type": "edit",
104                 "edited_action": {
105                     "name": action["name"],
106                     "args": {"file_name": "/tmp/test.txt"}
107                 }
108             })
109
110     print(f"\n=== 人工编辑后的审批决策 ===")
111     print(f"审批结果: {decisions}")
112
113     # 3. 恢复执行（使用编辑后的参数）
114     print("\n=== 第二阶段：恢复执行（使用编辑后的参数）===")
115     result = deep_agent.invoke(
116         Command(resume={"decisions": decisions}), # 传入编辑后的决策
117         config=thread_config # 必须使用相同的thread_id
118     )
119
120     # 4. 输出最终结果
121     print("\n=== 执行完成 ===")
122     print(f"Agent最终回复: {result['messages'][-1].content}")
```

```
123 else:
124     # 无中断时直接输出结果
125     print("无需要审批的操作，执行结果:", result["messages"][-1].content)
```

1.8 后端存储 (Backends)

<https://docs.langchain.com/oss/python/deepagents/backends>

DeepAgents 的 **Backend** 系统是为 Agent 构建的“虚拟文件系统”，核心作用是定义 Agent 生成文件的最终存储位置，也是实现跨线程数据共享、落地长期记忆能力的核心载体。



核心机制：

- 被动触发逻辑：Backend 仅在 Agent 主动调用文件操作工具（如 `write_file`、`edit_file`、`read_file`）时才会被激活。需注意的是，Agent 的思考过程、对话上下文等临时状态仅存储在内存（State）中，不会自动写入 Backend，只有显式执行文件操作的内容才会进入该系统。
- 路径映射规则：Agent 操作的所有文件均基于“虚拟路径”（如 `/report.txt`、`/store/memory.txt`），Backend 会按照预设规则将这些虚拟路径映射到实际物理存储介质——比如本地硬盘、Redis 数据库、内存等，实现“虚拟路径”到“物理存储”的无感转换。

存储行为对照表：

行为	Backend 是否存储	存储位置
Agent 说: "你好"	否	仅在当前对话内存 (State)
Agent 思考过程	否	仅在当前对话内存 (State)
Agent 调用 <code>write_file("a.txt", "内容")</code>	是	Backend (硬盘/数据库)

1.8.1 后端类型概览

DeepAgents 提供了四种标准的后端实现，适用于不同的开发和生产场景：

后端类型	存储介质	适用场景	类比
StateBackend (默认)	内存 (State)	临时文件、中间运算结果。会话结束即销毁。	浏览器的“无痕模式”
FilesystemBackend	本地硬盘	本地开发、调试、需要直接查看生成文件的场景。	电脑的本地磁盘
StoreBackend	数据库 (KV Store)	生产环境、跨 Agent 共享数据、持久化记忆 (Redis/Postgres)。	云盘 (iCloud/OneDrive)
CompositeBackend	混合存储	生产环境最佳实践。区分“临时文件”和“重要记忆”。	系统盘 (C盘) + 数据盘 (D盘)

1.8.2 本地文件存储 (FilesystemBackend)

场景描述：

在本地开发或调试时，我们希望 Agent 生成的文件直接出现在项目文件夹中，方便开发者查看和验证。`FilesystemBackend` 将 Agent 的虚拟路径直接映射到宿主机的物理文件系统。

功能特点：

- 直观可见**：生成的文件可以直接在 IDE 或文件管理器中打开。
- 安全隔离**：推荐开启 `virtual_mode=True`，将 Agent 限制在指定的工作目录（`root_dir`）内，防止越权访问系统敏感文件。

代码示例：

```
1 from pathlib import Path # 导入Path类
2 from deepagents import create_deep_agent
3 from deepagents.backends import FilesystemBackend
4 from langchain.chat_models import init_chat_model
```

```

5 from dotenv import load_dotenv, find_dotenv
6 import os
7
8 load_dotenv(find_dotenv())
9
10 # 1. 准备本地工作目录（用Path改写）
11 workspace_dir = Path("./agent_workspace").resolve() # resolve() 等价于 os.path.abspath(), 获取绝对路径
12 if not workspace_dir.exists(): # 等价于 os.path.exists()
13     workspace_dir.mkdir(parents=True, exist_ok=True) # 等价于 os.makedirs()
14
15 print(f"Agent 的工作目录已设置为: {workspace_dir}")
16
17 # 2. 配置本地文件系统后端
18 # virtual_mode=True 开启安全沙箱模式, 限制 Agent 只能访问 workspace_dir
19 backend = FilesystemBackend(root_dir=workspace_dir, virtual_mode=True)
20
21 llm = init_chat_model(
22     model=os.getenv("LLM_QWEN_MAX"),
23     model_provider="openai"
24 )
25
26 # 3. 创建 Agent
27 # System Prompt 提示 Agent 按需创建文件
28 agent = create_deep_agent(
29     model=llm,
30     backend=backend,
31     system_prompt="你是一个智能助手。你可以使用文件工具来读写文件, 但只有在用户明确要求时才创建文件。"
32 )
33
34 # 4. 运行并验证
35 print("\n=== Case 1: 普通问答（不应该生成文件） ===")
36 result1 = agent.invoke({"messages": [{"role": "user", "content": "请告诉我 Python 是什么时候发明的?"}]}))
37 print("Agent 回复:", result1["messages"][-1].content)
38
39 # === Case 1: 验证没有生成文件 ===
40 # 替换 os.listdir → Path.iterdir() + 判断是否有文件
41 files = list(workspace_dir.iterdir()) # 列出目录下所有条目（文件/子目录）
42 if not files:
43     print("Case 1 成功: 目录下没有生成任何文件。")
44 else:
45     # 提取文件名/目录名, 保持输出格式一致
46     file_names = [f.name for f in files]
47     print(f"Case 1 失败: 目录下生成了文件: {file_names}")
48
49 print("\n=== Case 2: 明确要求生成文件 ===")
50 result2 = agent.invoke({"messages": [{"role": "user", "content": "帮我写一份关于 Java 的简短介绍, 并保存为 java_intro.md"}]}))
51 print("Agent 回复:", result2["messages"][-1].content)
52
53 # 5. 验证 Case 2 文件是否真实存在
54 # 替换 os.path.join → Path / 运算符
55 file_path = workspace_dir / "java_intro.md" # Path 拼接更直观
56 if file_path.exists(): # 替换 os.path.exists → Path.exists()
57     print(f"\nCase 2 成功! 文件已生成在: {file_path}")
58     with open(file_path, "r", encoding="utf-8") as f: # Path 对象可直接传入 open
59         print(f" 文件内容预览:\n{f.read()[:100]}...")
60 else:
61     print("\nCase 2 失败: 文件未生成。")

```

1.8.3 数据库/内存存储 (StoreBackend)

场景描述:

在生产环境或分布式系统中, 文件不适合存储在本地磁盘。StoreBackend 利用 LangGraph 的 Store 机制, 将文件内容作为 Key-Value 数据存储在数据库（如 Redis、Postgres）或内存中。这对于实现跨线程记忆共享至关重要。

功能特点:

- **持久化:** 配合 RedisStore 可实现数据持久保存。
- **共享性:** 不同线程 (Thread) 甚至不同 Agent 可以通过访问同一个 Store 来共享数据。

- **适配器模式**： `StoreBackend` 充当适配器，将文件操作转换为 KV 存储操作。

代码示例：

```
1 from deepagents import create_deep_agent
2 from deepagents.backends import StoreBackend, StateBackend
3 from langgraph.store.memory import InMemoryStore
4 from dotenv import load_dotenv, find_dotenv
5 from langchain.chat_models import init_chat_model
6 import os
7 load_dotenv(find_dotenv())
8
9 # 生产环境建议使用 RedisStore: from langgraph.store.redis import RedisStore
10
11 # 1. 准备 Store (模拟数据库)
12 # InMemoryStore 是轻量级内存存储，重启后数据丢失。
13 store = InMemoryStore()
14
15 # 2. 配置 Store 后端
16 llm = init_chat_model(
17     model=os.getenv("LLM_QWEN_MAX"),
18     model_provider="openai"
19 )
20
21 # StoreBackend 将 Agent 的文件操作转换为对 Store 的读写
22 # 默认情况下，它将文件存储在 ("filesystem", filename) 的 key 下
23 agent = create_deep_agent(
24     model=llm,
25     store=store,          # 传入数据仓库
26     backend=StoreBackend, # 注意：此处主 Backend 设为 StateBackend, StoreBackend 通常作为辅助或通过 Composite 使用
27     tools=[],
28     system_prompt="请把用户的重要信息保存到 user_profile.txt"
29 )
30
31 # 注意：为了让 Agent 直接使用 StoreBackend 存储文件，
32 # 通常会直接将 backend 设置为 StoreBackend，或者在 CompositeBackend 中路由。
33 # 下面的示例主要演示 Store 的跨线程数据读取能力。
34
35 # 3. 运行 Agent (Thread A) - 写入记忆
36 print("\n=== 写入记忆 (Thread A) ===")
37 config_a = {"configurable": {"thread_id": "thread_a"}}
38 # 假设 Agent 内部逻辑会将信息写入 Store (需配合正确的 Backend 配置，此处简化演示 Store 交互)
39 # 在实际使用 StoreBackend 时，Agent 调用 write_file("user_profile.txt") 会被存入 Store
40 result = agent.invoke({
41     "messages": [{"role": "user", "content": "我叫大风子，我的幸运数字是 7。"}]
42 }, config=config_a)
43
44 print("Agent 回复:", result["messages"][-1].content)
45
46 # 4. 运行 Agent (Thread B) - 跨线程读取
47 print("\n=== 读取记忆 (Thread B) ===")
48 # 使用不同的 thread_id，模拟另一个会话
49 config_b = {"configurable": {"thread_id": "thread_b"}} # 注意这里是 thread_b
50
51 # 这里的关键点是：Store 是共享的。Thread B 可以读取 Thread A 写入的数据。
52 result_b = agent.invoke({
53     "messages": [{"role": "user", "content": "请读取 user_profile.txt 告诉我，我叫什么名字？我的幸运数字是多少？"}]
54 }, config=config_b)
55
56 print("Agent (Thread B) 回复:", result_b["messages"][-1].content)
57
58 # 验证：直接检查 Store 数据
59 print("\n=== 验证 Store 数据 ===")
60 # 所有文件的创建、修改、读取操作，都会自动关联 ("filesystem",) 顶级命名空间；
61 items = store.search(("filesystem",))
62 for item in items:
63     print(f"Key: {item.key}")
64     print(f"Value: {item.value}")
```

1.8.4 混合存储策略 (CompositeBackend)

场景描述：

这是最灵活且推荐的生产环境配置。`CompositeBackend` 允许你根据**文件路径的前缀**，将文件路由到不同的后端。例如，将临时文件存本地，将重要记忆存数据库。

配置逻辑：

- **默认路由 (Default)**: 处理普通路径，通常映射到 `FilesystemBackend` (本地) 或 `StateBackend` (临时)。
- **特定路由 (Routes)**: 处理特定前缀路径 (如 `/store/`)，映射到 `StoreBackend` (数据库)。

代码示例：

```
1 from deepagents import create_deep_agent
2 from deepagents.backends import StoreBackend, FilesystemBackend, CompositeBackend
3 from langgraph.store.memory import InMemoryStore
4 from dotenv import load_dotenv, find_dotenv
5 from langchain.chat_models import init_chat_model
6 import os
7 from pathlib import Path # 新增导入 Path 类
8
9 load_dotenv(find_dotenv())
10
11 # 1. 准备 Store
12 store = InMemoryStore()
13
14 # 2. 配置 LLM
15 llm = init_chat_model(
16     model=os.getenv("LLM_QWEN_MAX"),
17     model_provider="openai"
18 )
19
20 # 3. 定义混合后端工厂函数
21 def create_composite_backend(runtime):
22     # 后端 A: 本地文件系统 (存普通文件)
23     # 1. 准备本地工作目录 (用Path改写)
24     workspace_dir = Path("./agent_workspace").resolve() # resolve() 等价于 os.path.abspath(), 获取绝对路径
25     if not workspace_dir.exists(): # 等价于 os.path.exists()
26         workspace_dir.mkdir(parents=True, exist_ok=True) # 等价于 os.makedirs()
27     fs_backend = FilesystemBackend(root_dir=workspace_dir, virtual_mode=True)
28
29     # 后端 B: 数据库存储 (存重要记忆)
30     store_backend = StoreBackend(runtime)
31
32     # 组合后端: 配置路由规则
33     return CompositeBackend(
34         default=fs_backend, # 默认走本地文件系统
35         routes={
36             "/store/": store_backend # 以 /store/ 开头的路径走数据库存储
37         }
38     )
39
40
41 agent = create_deep_agent(
42     model=llm,
43     store=store,
44     backend=create_composite_backend, # 传入工厂函数
45     tools=[],
46     system_prompt="""你是一个智能助手。
47     - 普通文件：直接写入文件名 (如 `report.txt`)，保存到本地 workspace。
48     - 重要记忆：写入 `/store/` 目录 (如 `/store/profile.txt`)，保存到store指定的存储方式中。
49     """
50 )
51
52 # 4. 运行 Agent
53 print("\n=== 测试混合存储 ===")
54 config = {"configurable": {"thread_id": "thread_composite"}}
55
56 # 任务：同时触发两种存储路径
57 user_input = "1. 创建本地文件 local.txt，内容'本地文件'。\\n2. 创建记忆文件 /store/memory.txt，内容'重要记忆'。"
```

```
58 print(f"用户指令: {user_input}")
59
60 result = agent.invoke({
61     "messages": [{"role": "user", "content": user_input}]
62 }, config=config)
63
64 print("Agent 回复:", result["messages"][-1].content)
65
66 # 5. 验证结果
67 print("\n=== 验证本地文件 (Filesystem) ===")
68 # 替换 os.path.join + os.path.exists 为 Path 写法
69 local_path = Path("agent_workspace") / "local.txt" # Path 拼接路径
70 if local_path.exists(): # Path 内置 exists 方法
71     print(f"本地文件存在: {local_path}")
72 else:
73     print("本地文件缺失")
74
75 print("\n=== 验证数据库存储 (Store) ===")
76 # CompositeBackend 会自动剥离路由前缀, 所以 /store/memory.txt 在 Store 中的 key 为 /memory.txt
77 items = store.search(("filesystem",))
78 for item in items:
79     print(item)
```

1.9 Middleware 核心概念

Middleware 是 DeepAgents 的「流程拦截器」，可以在 Agent 执行的**关键生命周期节点**（如工具调用前 / 后、思考完成后、回复生成前）插入自定义逻辑，实现：

- 操作日志记录
- 权限校验 / 参数过滤
- 工具调用结果修改
- 异常捕获 / 兜底处理
- 自定义监控 / 告警

案例实现一个中间件，进行日志输出：

1. **工具调用前日志**：记录调用的工具名和参数；
2. **工具调用后日志**：记录工具执行结果和耗时。

@wrap_tool_call实现

```
1 # -*- coding: utf-8 -*-
2 """
3 DeepAgents Middleware 极简案例
4 核心：实现工具调用的日志监控中间件
5 """
6 import os
7 import time
8
9 from langchain.agents.middleware import wrap_tool_call
10 from langchain.agents.middleware.types import AgentMiddleware, ToolCallRequest
11 from langchain.chat_models import init_chat_model
12 from langchain.tools import tool
13 from deepagents import create_deep_agent
14 from langgraph.checkpoint.memory import InMemorySaver
15 from dotenv import load_dotenv, find_dotenv
16
17 # 加载环境变量
18 load_dotenv(find_dotenv())
19
20
21 # ===== 1. 定义测试工具 =====
22 @tool
23 def add_numbers(a: int, b: int):
24     """计算两个数字的和"""
25     time.sleep(0.5) # 模拟耗时操作
26     result = a + b
27     print(f"[工具执行] {a} + {b} = {result}")
```

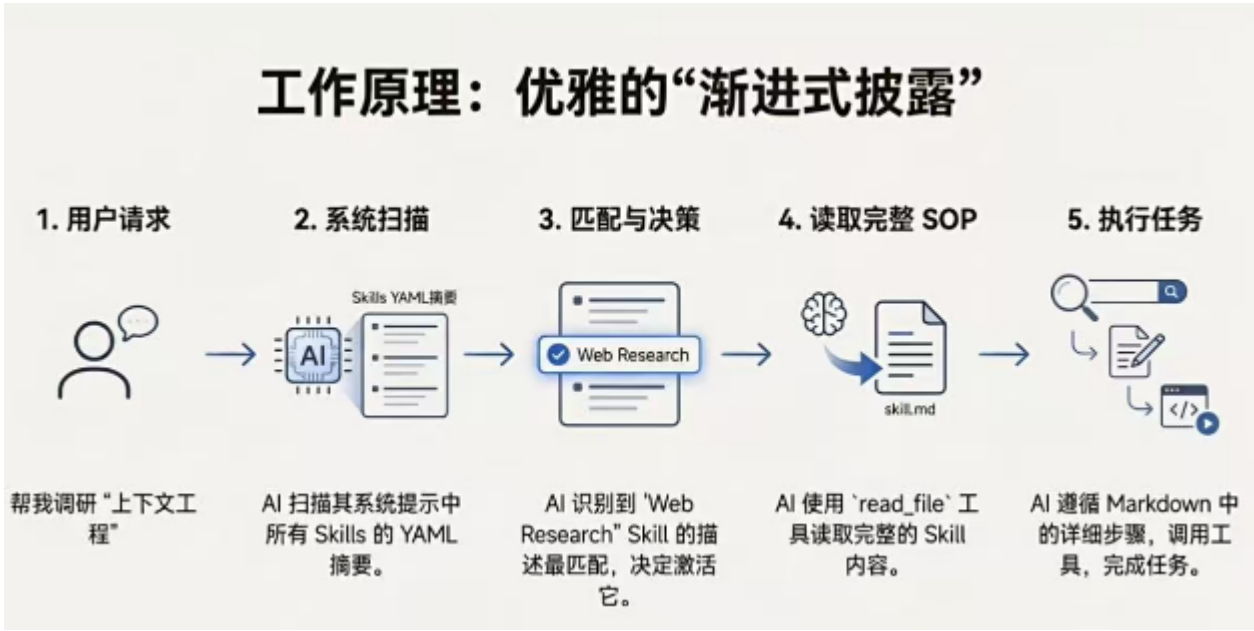
```
28     return result
29
30
31 @wrap_tool_call
32 def log_tool_call(request, handler):
33     tool_name = request.tool_call["name"]
34     tool_args = request.tool_call["args"]
35
36     # 1. 前置逻辑
37     print(f"\n[前置中间件] 工具调用开始 - 工具名: {tool_name}, 参数: {tool_args}, 时间: {time.strftime('%Y-%m-%d
%H:%M:%S')}}")
38
39     start_time = time.time()
40
41     # 2. 执行工具 (调用 handler)
42     result = handler(request)
43
44     end_time = time.time()
45     duration = end_time - start_time
46
47     # 3. 后置逻辑
48     # 先尝试从 result 对象中获取 content 属性; 如果 result 没有 content 属性 (比如不是 ToolMessage 类型), 就把 result 转成字符串作为兜底值。
49     content = getattr(result, "content", str(result))
50     print(f"[后置中间件] 工具调用完成 - 工具名: {tool_name}, 结果: {content}, 耗时: {duration:.2f}秒")
51
52     return result
53
54 # ===== 3. 配置Agent并绑定Middleware =====
55 # 初始化LLM
56 llm = init_chat_model(
57     model=os.getenv("LLM_QWEN_MAX"),
58     model_provider="openai"
59 )
60
61 # 创建Agent, 绑定中间件
62 deep_agent = create_deep_agent(
63     model=llm,
64     tools=[add_numbers],
65     checkpointer=InMemorySaver(),
66     # 绑定中间件: 传入 Middleware 实例列表
67     middleware=[log_tool_call],
68     system_prompt="你是一个计算器助手, 使用add_numbers工具完成加法计算, 回答仅返回计算结果。"
69 )
70
71 # ===== 4. 执行测试 =====
72 if __name__ == "__main__":
73     # 会话配置
74     thread_config = {"configurable": {"thread_id": "middleware_test_1"}}
75
76     # 调用Agent
77     result = deep_agent.invoke(
78         {
79             "messages": [
80                 {"role": "user", "content": "帮我计算 100 + 200 的结果"}
81             ]
82         },
83         config=thread_config
84     )
85
86     # 输出最终结果
87     print("\n=== 最终回复 ===")
88     print(result["messages"][-1].content)
```

1.10 Agent Skills (技能扩展)

DeepAgents 提供的 **Skills (技能)** 机制, 是为智能体 (Agent) 注入领域知识与专业能力的核心方式。Skills 本质是可复用、可插拔的“能力包”, 核心由指令文档 (SKILL.md) 及配套资源构成, 能让 Agent 在运行过程中, 根据任务场景的实际需求动态加载、调用对应的技能知识, 无需修改 Agent 核心逻辑即可快速扩展专业能力。

核心概念：

- **SKILL.md**：技能的核心描述文件，是 Agent 学习和使用该技能的“说明书”。文件整体分为两部分 —— 头部的**元数据（Frontmatter）**（以 YAML 格式定义 定义name 和 description）和正文的**具体指令**（以 Markdown 格式编写），Agent 会通过解析该文件掌握技能的使用方法、适用场景及操作流程。
- **渐进式披露**：Skills 机制的核心优化策略，用于解决大模型上下文窗口有限的问题。Agent 启动时仅读取所有技能的元数据（轻量信息，占用极少上下文），仅记录“技能名称、适用场景、触发关键词”等基础信息；只有当用户任务匹配某一技能的触发条件时，Agent 才会加载该技能的详细指令内容，有效避免无关信息占用上下文，提升任务执行效率。



标准技能目录结构：

一个完整的 DeepAgents 技能包遵循标准化的文件目录结构，不同文件各司其职，确保技能可复用、易维护，典型结构如下：

1	skill-xxx/	# 技能根目录（命名规范：skill-技能名，小写+短横线）
2	├─ SKILL.md	# 核心：技能描述文件（必选）
3	├─ requirements.txt	# 依赖声明文件（可选）
4	├─ resources/	# 配套资源目录（可选）
5	├─ template/	# 模板文件（如报表模板、代码模板）
6	├─ examples/	# 示例文件（如技能使用的输入/输出示例）
7	└─ config/	# 配置文件（如工具调用的默认参数、规则配置）
8	└─ scripts/	# 辅助脚本目录（可选）
9	└─ helper.py	# 技能配套的辅助脚本（如复杂逻辑的封装、数据预处理）

各文件 / 目录的具体功能：

1. SKILL.md（必选）

技能的核心载体，是 Agent 唯一需要解析的文件，典型结构如下：

```
1  ---
2  # 元数据（Frontmatter，YAML 格式，Agent 启动时读取）
3  name: 数据清洗          # 技能名称（唯一标识）
4  version: 1.0            # 技能版本
5  trigger: ["清洗数据", "处理csv", "缺失值填充"]  # 触发关键词（匹配用户指令时加载技能）
6  tools: ["pandas", "read_csv", "write_csv"]      # 依赖工具（Agent 需提前注册）
7  author: xxx             # 技能作者
8  description: 用于CSV/Excel数据的去重、缺失值处理、格式标准化等操作  # 技能简介
9  ---
10 # 具体指令（Agent 触发技能时读取）
11 ## 技能说明
12 本技能适用于结构化数据清洗，支持CSV/Excel格式，包含基础清洗和高级规整两类操作。
13
14 ## 操作步骤
15 1. 调用 read_csv 工具读取数据，指定编码为 utf-8；
16 2. 执行去重操作：df.drop_duplicates(subset=["主键列"], keep="first")；
17 3. 缺失值处理：数值列用均值填充，文本列用空字符串填充；
18 4. 调用 write_csv 工具保存清洗后的数据，关闭索引输出。
19
20 ## 注意事项
21 - 若文件编码异常，尝试切换为 gbk 编码；
22 - 缺失值占比超50%的列建议直接删除。
```

2. requirements.txt（可选）

声明技能运行所需的第三方依赖包及版本，例如：

```
1 pandas>=2.0.0
2 openpyxl>=3.1.0 # 支持Excel文件处理
```

作用：部署技能时可一键安装依赖，避免因环境缺失导致技能执行失败。

3. resources/ (可选)

存放技能配套的静态资源，按用途细分：

- `template/`：存放各类模板文件，如 “数据清洗报告模板.md”“财务报表模板.xlsx”，Agent 可调用模板快速生成标准化输出；
- `examples/`：存放技能使用示例，如 “原始数据示例.csv”“清洗后数据示例.csv”，帮助 Agent 理解技能的预期输入 / 输出；
- `config/`：存放配置文件（如 JSON/YAML 格式），如 “数据清洗规则.json”，定义固定规则（如日期格式、字段映射），避免硬编码在 SKILL.md 中。

4. scripts/ (可选)

存放技能配套的辅助脚本，封装复杂逻辑或工具调用细节，例如：

- `helper.py`：编写 `fill_missing_value()` 函数封装缺失值填充逻辑，SKILL.md 中只需调用该函数，无需写完整代码；
- 脚本可被 Agent 调用的工具函数引用，简化 SKILL.md 中的指令复杂度，提升技能执行效率。

补充说明

- 技能包的核心是 `SKILL.md`，其余文件均为辅助，可根据技能复杂度选择是否添加；
- 所有文件需遵循 “轻量化” 原则，尤其是 SKILL.md 的详细指令部分，避免内容过长导致上下文超限；
- 技能包支持动态加载 / 卸载，可通过 DeepAgents 的 API 将技能注册到 Agent，也可在运行时移除无需使用的技能。

SKILL.md 标准格式示例：

文件路径：`base/skills/code-reviewer/SKILL.md`

```
1  ---
2  name: code-reviewer
3  description: 当用户请求进行代码审查(Code Review)或寻找代码Bug时，使用此技能。
4  ---
5  # Code Reviewer Skill (代码审查专家技能)
6
7  ## 角色定义
8  你是一位拥有10年经验的资深架构师，以严谨、犀利著称。
9
10 ## 审查标准 (Instructions)
11 在审查用户提供的代码时，必须严格遵循以下步骤：
12
13 1.  **安全性检查**：
14     - 检查是否有 SQL 注入、硬编码密钥、路径遍历等安全风险。
15     - 如果发现，必须用【高危】标签醒目标注。
16
17 2.  **性能优化**：
18     - 检查是否有重复计算、无效循环或过大的内存占用。
19     - 给出具体的优化代码建议。
20
21 3.  **代码风格 (PEP 8)**：
22     - 检查变量命名是否规范。
23     - 检查是否缺少必要的注释。
24
25 4.  **输出格式**：
26     - 使用 Markdown 表格列出所有问题。
27     - 评分：给代码打分（0-100）。
```

代码示例：加载外部 Skills 文件

```
1  import os
2  from pathlib import Path
3  from langchain.chat_models import init_chat_model
4  from deepagents import create_deep_agent
5  from deepagents.backends import FilesystemBackend
6  from langgraph.checkpoint.memory import MemorySaver
7  from dotenv import load_dotenv, find_dotenv
8
9  # 加载环境变量
10 load_dotenv(find_dotenv())
11
12 # ===== 1. 设置 Backend =====
13 # 使用 FilesystemBackend 连接到本地文件系统
14 # 假设 skills 目录在当前脚本同级目录下的 skills/ 中
15 current_dir = Path(__file__).parent.resolve()
```

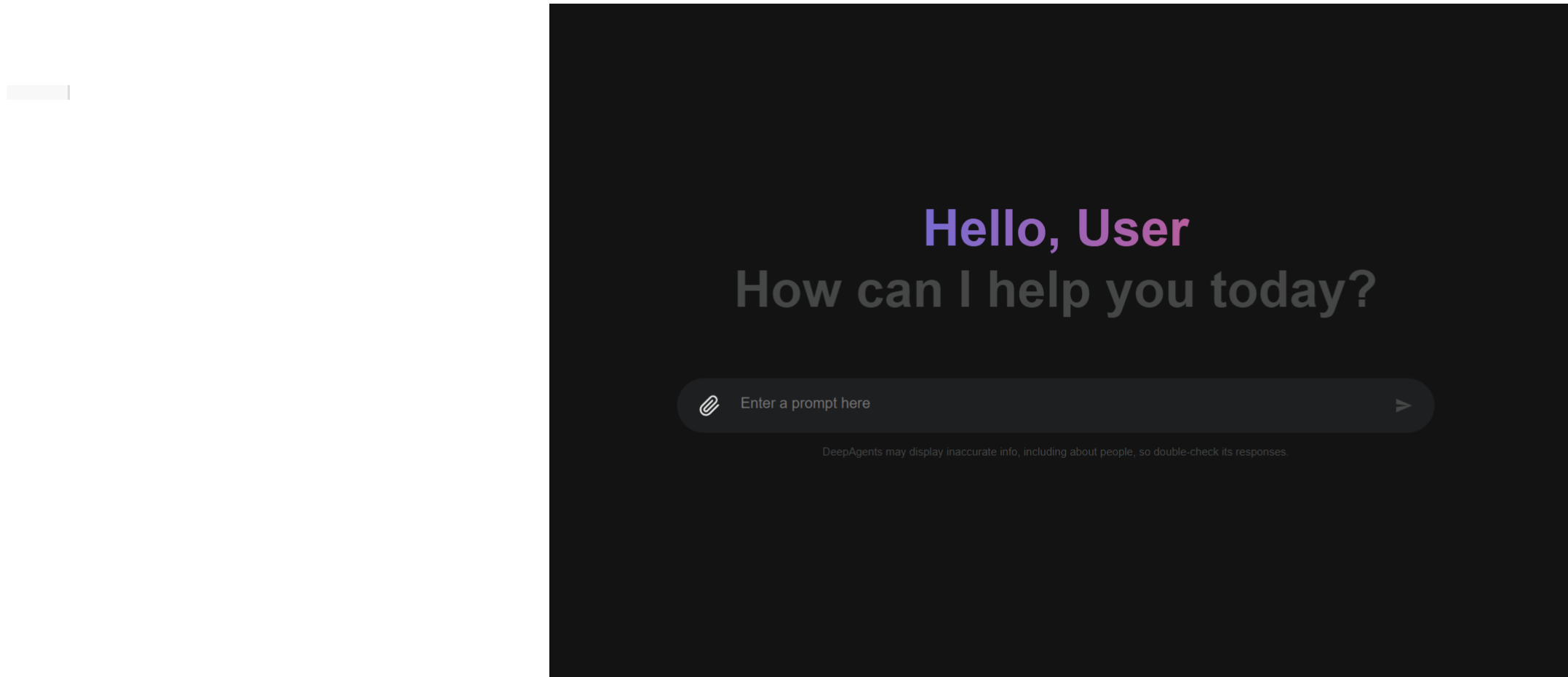
```
16 # 我们将 root_dir 设置为当前目录 (base)
17 # 注意:FilesystemBackend 的 root_dir 是物理路径的根
18 fs_backend = FilesystemBackend(root_dir=current_dir)
19
20 # ===== 2. 初始化 Agent =====
21 llm = init_chat_model(
22     model="qwen-max",
23     model_provider="openai"
24 )
25
26 # 创建带 Skill 的 Agent
27 agent = create_deep_agent(
28     model=llm,
29     # 关键点1:注入文件系统后端
30     backend=fs_backend,
31     # 关键点2:告诉 Agent 在 /skills/ 目录下查找技能
32     # 这里的 /skills/ 是相对于 backend root_dir 的路径
33     # 物理路径为: base/skills/
34     skills=["skills"],
35     checkpointer=MemorySaver(),
36     # System Prompt 可以很通用,具体的专业指令由 Skill 提供
37     system_prompt="你是一个有用的 AI 助手。"
38 )
39
40 # ===== 3. 运行演示 =====
41
42 def run_demo():
43     print("\n=== 场景:用户提供一段有问题的代码求审查 ===")
44
45     bad_code = """
46         def get_user(user_id):
47             # 连接数据库
48             import sqlite3
49             conn = sqlite3.connect('test.db')
50             cursor = conn.cursor()
51             # 直接拼接 SQL,有注入风险!
52             sql = "SELECT * FROM users WHERE id = " + user_id
53             cursor.execute(sql)
54             return cursor.fetchall()
55     """
56
57     print(f"用户代码片段:\n{bad_code}\n")
58     print(">>> Agent 正在思考并匹配技能...\n")
59
60     # 用户的提问触发了 SKILL.md 中的 description ("当用户请求进行代码审查...")
61     # Agent 会自动读取 SKILL.md 的内容,并按里面的步骤执行。
62     result = agent.invoke({
63         "messages": [
64             {"role": "user", "content": f"请使用 code-reviewer 技能帮我 Review 一下这段代码:\n{bad_code}"}
65         ],
66     }, config={"configurable": {"thread_id": "skill_demo_v3"}})
67
68     print("=== Agent 回复 (基于 code-reviewer 技能) ===")
69     print(result["messages"][-1].content)
70
71 if __name__ == "__main__":
72     run_demo()
73
```

关键点:

- 物理存储:** 将 `SKILL.md` 存放在实际的文件目录中 (`base/skills/code-reviewer/`)
- FilesystemBackend:** 使用 `FilesystemBackend` 将本地目录挂载到 Agent 的虚拟文件系统中。
- Skills 路径映射:** `skills=["skills"]` 指向的是虚拟路径, Agent 会通过 Backend 自动映射到物理路径。

二、深度搜索项目介绍

本项目是 DeepAgents 框架的一个典型最佳实践，旨在构建一个"深度搜索研究员"。

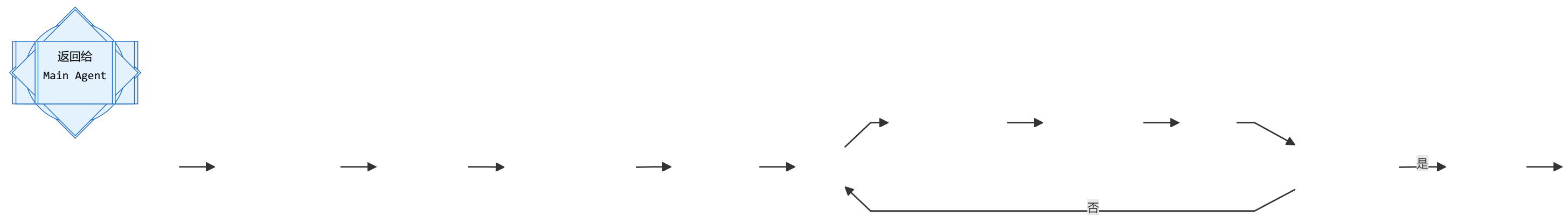


2.1 项目目标

利用DeepAgent 来**模拟人类高级研究员思维**的多路组合智能体系统，以「主智能体统筹 + 多专家子智能体并行协作」为核心架构，突破传统 RAG 单次检索局限，通过**搜索 - 阅读 - 反思 - 再搜索**多轮迭代，深度挖掘海量信息背后的隐藏逻辑，实现**广覆盖、高精度、强可靠**的复杂信息处理与文档生成。

2.2 Agent Design

系统采用**1 主 + N 专**多路组合模式，主智能体统一调度，三类专家子智能体各司其职、并行工作：



主智能体（Main Agent）是整个智能团队的 "项目经理" (Leader)。它不直接执行具体的搜索或查询任务，而是专注于理解需求、拆解任务、调度资源、交付结果。

- **Main Agent (主智能体):**
 - **职责:** 项目经理。负责理解用户意图、拆解任务步骤、生成子任务、以及最终汇总报告。
 - **能力:** 拥有全局视野，管理整个会话的状态和记忆。
- **Sub Agent (子智能体):**
 - 网络搜索助手：负责公开知识广域检索，支持由浅入深、多轮递进搜索，最多执行 5 次精准检索，覆盖 3 个以上信息维度。
 - 数据库查询助手：对接企业业务数据库，支持表结构读取、数据预览、自定义 SQL 查询，精准提取商品 / 业务明细数据。
 - RAGFlow 助手：对接企业私有知识库，先获取可用助手列表，再分层提问、深度检索，确保内部专有信息安全可用。

2.3 工具体系

我们为 Agent 装备了硬核的工具库：

主智能体工具

1. **generate_markdown** —— 生成标准 Markdown 文档
2. **convert_md_to_pdf** —— 将 Markdown 转为 PDF 文件

网络搜索助手

1. **internet_search (tavily)** —— 互联网公开信息多轮、多角度检索

数据库查询助手

1. **list_sql_tables** —— 列出数据库所有表结构
2. **get_table_data** —— 读取表数据预览
3. **execute_sql_query** —— 执行自定义 SQL 查询

RAGFlow 知识库助手

- 1. `get_assistant_list` —— 获取知识库可用助手列表
- 2. `create_ask_delete` —— 向知识库发起提问查询

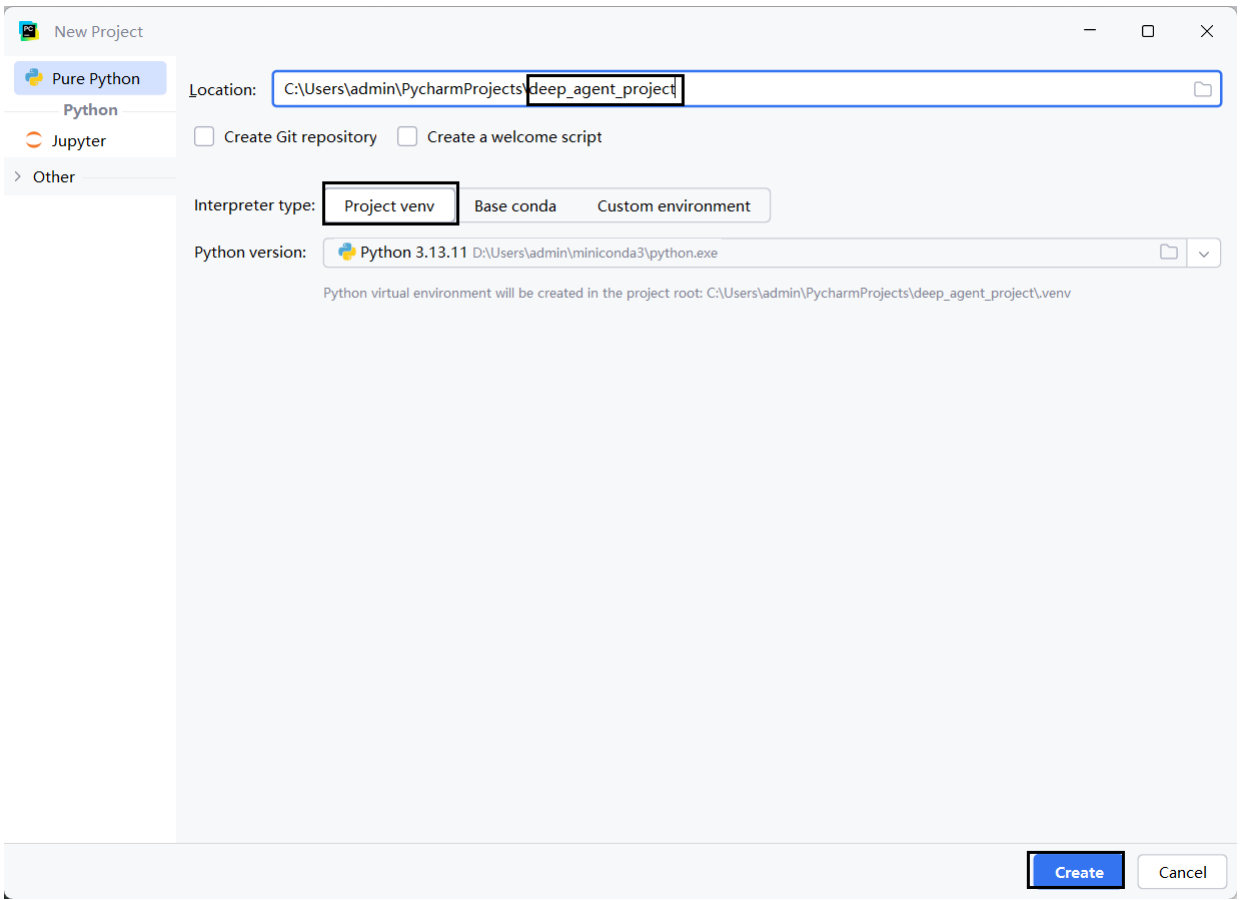
2.4 技术栈

本项目采用了全异步的高性能架构：

- LangChain / LangGraph / DeepAgent:
 - 介绍：项目的“神经中枢”。LangGraph 负责构建 有状态的循环工作流 (StateGraph)，让 Agent 具备记忆、规划和自我修正能力，打破了传统 LLM 单次问答的限制。
- OpenAI SDK：
 - 介绍：调用 GPT-4o / DeepSeek 等大模型的官方接口。通过 `bind_tools` 实现 Tool Calling（函数调用）。
- Pydantic：
 - 介绍：数据校验基石。用于定义 Agent 的状态结构 (AgentState) 和工具的输入参数模型，确保数据流转的类型安全。
- FastAPI：
 - 介绍：高性能异步 Web 框架。提供 RESTful API 接口，支持文件上传与静态资源托管。
- WebSocket：
 - 介绍：全双工通信。用于将 Agent 的思考过程 (Thinking Process) 和工具执行结果实时推送到前端，提升用户体验。
- Uvicorn：
 - 介绍：ASGI 服务器，FastAPI 的启动引擎。
- Tavily Search API (`tavily_tools.py`):
 - 介绍：专为 AI 设计的搜索引擎。相比 Google，它返回的内容更结构化，不仅包含链接，还包含清洗后的网页正文，极大减少了 Agent 的 Token 消耗。
- RAGFlow (`ragflow_tools.py`):
 - 介绍：企业级 RAG 引擎。用于连接本地知识库，支持 PDF、Word 等文档的深度解析与语义检索。
- PyMuPDF (fitz) (`pdf_tools.py`):
 - 介绍：高性能 PDF 处理库。用于精准提取 PDF 中的文本和表格，辅助 Agent 阅读文献。
- PyMySQL / SQLAlchemy (`mysql_tools.py`):
 - 介绍：数据库连接器。赋予 Agent 操作结构化数据（SQL）的能力，使其能查询业务报表。
- Markdown / File IO (`markdown_tools.py` , `upload_file_read_tool.py`):
 - 介绍：文件读写能力。支持 Markdown 格式的报告生成，以及读取用户上传的任意文本文件。
- Asyncio：
 - 介绍：Python 异步编程标准库。通过 `async/await` 实现非阻塞 IO，让 Agent 在等待网络请求（如搜索、LLM 生成）时，服务器仍能响应其他请求。
- ContextVars：
 - 介绍：线程上下文变量。在异步并发环境中，像“隐形传态”一样传递 `thread_id` 和 `user_id`，确保日志和工具能正确归属到对应用户，防止数据串线。
- Pathlib：
 - 介绍：面向对象的文件路径处理。解决了 Windows/Linux 路径分隔符差异的痛点。
- Shutil：
 - 介绍：高级文件操作。用于高效地复制、移动和归档用户上传的文件。

三、项目搭建指南

3.1 创建项目



3.2 导入依赖

```
1 # --- [核心依赖] Deep Agents 框架 ---
2 deepagents>=0.1.0          # 核心工厂方法 create_deep_agent
3
4 # --- LangChain 生态（基础支撑） ---
5 langchain>=0.2.0           # LangChain 核心库
6 langchain-core>=0.2.0      # 基础组件（Prompt, Message, Tool）
7 langchain-community>=0.2.0 # 社区工具集成
8 langgraph>=0.1.0           # 智能体工作流编排
9
10 # --- 大模型服务（LLM Services） ---
11 langchain-openai>=0.1.0    # OpenAI 接口适配
12 openai>=1.0.0              # OpenAI 官方 SDK
13
14 # --- Web 服务（API & WebSocket） ---
15 fastapi>=0.100.0           # 异步 Web 框架
16 uvicorn[standard]>=0.20.0   # 服务器引擎（含 WebSocket）
17 python-multipart>=0.0.6     # 文件上传支持
18
19 # --- 数据库与存储（Database） ---
20 mysql-connector-python>=8.0.0 # [修正] MySQL 官方驱动（用于 mysql_tools）
21 # 注意：代码中使用的是 mysql.connector，对应这个包，而非 pymysql
22
23 # --- 文件处理（File Processing - upload_file_read_tool） ---
24 python-docx>=1.0.0          # word (.docx) 读取
25 pypdf>=3.0.0                # [修正] PDF 读取（代码用的是 pypdf）
26 pandas>=2.0.0               # Excel 数据处理
27 openpyxl>=3.1.0             # Excel 读取引擎（Pandas 依赖）
28 aiofiles>=23.0.0            # 异步文件操作
29
30 # --- 外部工具集成（Integrations） ---
31 tavily-python>=0.3.0        # 联网搜索 API
32 ragflow-sdk>=0.1.0          # 知识库 SDK
33 requests>=2.31.0            # HTTP 请求库
34
35 # --- 基础工具（Utilities） ---
36 PyYAML>=6.0                 # [新增] YAML 配置文件解析（prompts.py）
37 pydantic>=2.0.0             # 数据校验
38 python-dotenv>=1.0.0        # 环境变量管理
39 typing-extensions>=4.5.0    # 类型兼容性支持
```

安装依赖：

- 1. 将资料中requirements.txt文件粘贴到项目根路径下
- 2. 使用命令进行批量安装

```
1 pip install -r requirements.txt
```

3.3 项目包结构

请按照以下结构创建目录和空文件

```
1 deep_agent_project/
2   └─ agent/
3     └─ sub_agents          # [核心] 存储subagents数据
4       └─ main_agent.py     # [核心] 智能体组装与执行逻辑
5   └─ api/
6     └─ __init__.py
7     └─ context.py          # [核心] ContextVars 会话隔离
8     └─ logger.py           # [核心] 分布式日志系统
9     └─ monitor.py          # [核心] WebSocket 监控单例
10    └─ server.py            # [入口] FastAPI 服务端
11  └─ prompt/
12    └─ prompts.yaml         # [核心] 所有 Prompt 配置文件
13  └─ tools/
14    └─ __init__.py          # 工具导出
14    └─ tavily_tools.py      # 搜索工具
15    └─ mysql_tools.py       # 数据库工具
16    └─ ragflow_tools.py     # RAG 工具
17    └─ markdown_tools.py    # 文件生成
18    └─ pdf_tools.py         # PDF 转换
19    └─ upload_file_read_tool.py # 文件读取
20  └─ ui/                    # 存储前端项目 node环境运行
21  └─ output/                # 自动生成，存放会话产物
22  └─ updated/               # 用户上传文件存放区
23  └─ .env                   # 环境变量
```

3.4 基础导入

在开始写智能体之前，先导入一些基础类和前端工程：**日志、监控、会话隔离以及UI前端先导入。**

3.4.1 导入前端项目

环境准备

- 安装 Node.js, **推荐版本 20.19.0** (可通过 `nvm` 或 Node 官网安装包配置)
- 验证安装: 执行 `node -v`, 确认输出版本为 v20.19.0 及以上

项目文件部署

将 `资料/ui` 目录完整复制到当前项目根目录下

启动服务

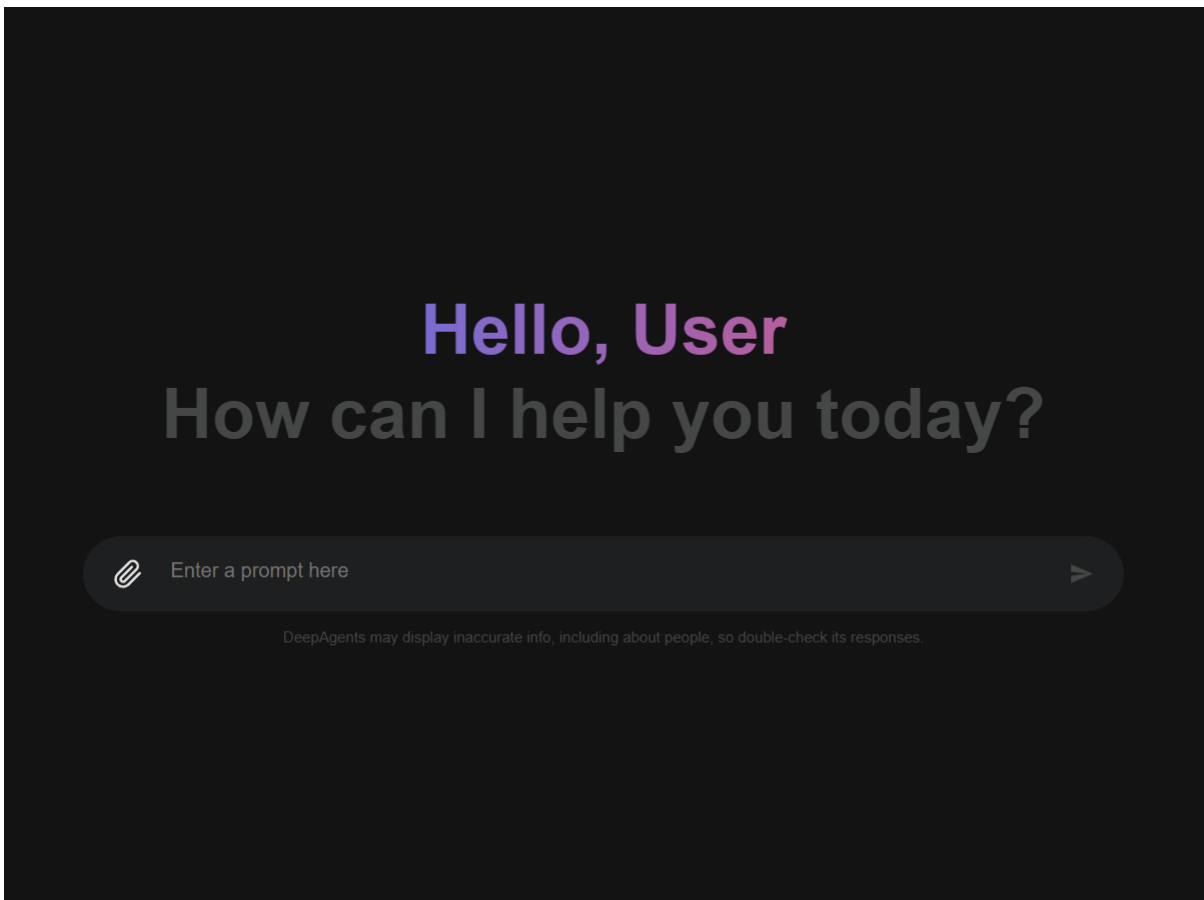
打开终端，执行以下命令：

```
1 # 进入UI目录
2 cd ui
3 # 启动开发服务
4 npm run dev
```

访问测试

服务启动成功后，在浏览器访问：

```
1 http://localhost:5173
```



补充说明

- 若启动报错，优先检查：Node 版本是否匹配、ui 目录下 `package.json` 依赖是否完整（可执行 `npm install` 安装依赖）
- 端口 5173 被占用时，可修改 ui 目录下的配置文件（如 `vite.config.js`）调整端口

3.4.2 api通信相关

以下是项目中需导入的四个核心类:

1. `server.py` —服务入口

接收客户请求（API 接口）、分配唯一标识（Thread ID），将任务派给后台异步处理，同时通过 WebSocket 实时反馈进度。

2. `context.py` —数据隔离

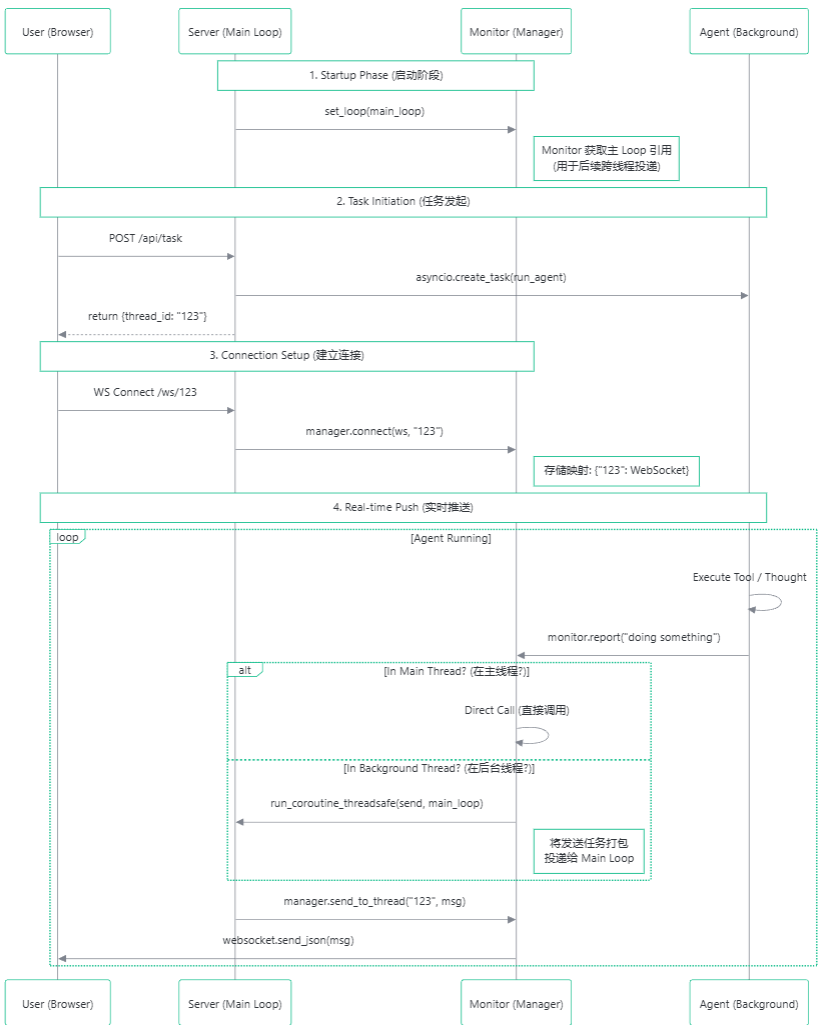
为任务打上专属标识，支持任意环节快速获取任务身份，核心实现多任务(请求)数据隔离，避免信息串混。

3. `monitor.py` —实时反馈

收集后台任务执行状态，结合任务标识精准推送消息，解决后台与前台的跨线程通信问题。

4. `logger.py` —— 全程记录

记录任务全流程操作（含思考、工具调用、结果），按任务标识分类归档日志，方便故障排查与复盘。



3.4.2.1 会话数据（`api/context.py`）

说明：使用 `ContextVars` 确保不同用户的请求（协程）在服务器内部是完全隔离的，不会混淆文件路径。

```
1 from contextvars import ContextVar
2 from typing import Optional
3
4
5 # 存储当前会话对应的文件夹
6 _session_dir_ctx: ContextVar[Optional[str]] = ContextVar("session_dir", default=None)
7
8 # 存储当前会话对应的websocket
9 _thread_id_ctx: ContextVar[Optional[str]] = ContextVar("thread_id", default=None)
10
11
12 def set_session_context(path: str):
13     """
```

```
14     设置当前请求链路的会话目录。
15     通常在 Agent 开始执行任务前调用。
16
17     Returns:
18         Token: 返回一个 Token 对象，后续可用它来恢复(reset)变量状态。
19     """
20     return _session_dir_ctx.set(path)
21
22 def get_session_context() -> Optional[str]:
23     """
24     获取当前请求链路的会话目录。
25     可以在任何深层调用的工具函数中直接使用，无需层层传递参数。
26     """
27     return _session_dir_ctx.get()
28
29 def set_thread_context(thread_id: str):
30     """
31     设置当前请求链路的 Thread ID。
32     """
33     return _thread_id_ctx.set(thread_id)
34
35 def get_thread_context() -> Optional[str]:
36     """
37     获取当前请求链路的 Thread ID。
38     """
39     return _thread_id_ctx.get()
40
41 def reset_session_context(session_token, thread_token=None):
42     """
43     清理/重置上下文。
44     通常在请求处理结束（finally 块）中调用，防止内存泄漏或污染后续请求。
45     """
46     _session_dir_ctx.reset(session_token)
47     if thread_token:
48         _thread_id_ctx.reset(thread_token)
49
50
51 if __name__ == "__main__":
52     import asyncio
53     import random
54
55     # =====
56     # 模拟案例：张三和李四同时来办理业务
57     # =====
58     async def process_user_request(user_name: str, user_dir: str):
59         print(f"[{user_name}] 1. 请求开始，设置环境 -> {user_dir}")
60
61         # 1. 【进门】设置上下文，拿到凭证（Token）
62         # 注意：这里不需要把 token 传给 deep_function，它自己能取到 context
63         dir_token = set_session_context(user_dir)
64         id_token = set_thread_context(f"thread_{user_name}")
65
66         try:
67             # 2. 【办事】模拟进入深层函数调用（中间可能隔了十层八层）
68             await deep_nested_function(user_name)
69
70         finally:
71             # 3. 【出门】业务办完，必须拿着凭证销户（恢复现场）
72             print(f"[{user_name}] 4. 请求结束，清理环境")
73             reset_session_context(dir_token, id_token)
74
75             # 验证清理结果（应该变回 None 或初始值）
76             current_dir = get_session_context()
77             print(f"[{user_name}] 5. 清理后检查：{current_dir}（应该是 None）")
78
79     async def deep_nested_function(user_name):
80         """
81         这是一个深层调用的函数，它没有接收任何 path 参数。
82         但它可以通过 ContextVar "隔空取物"。
```

```
83         """
84         # 模拟耗时操作，让张三和李四的任务交织在一起
85         await asyncio.sleep(random.uniform(0.1, 0.5))
86
87         # 核心验证：直接从 Context 取值
88         # 如果没有隔离，李四可能会取到张三的目录
89         current_dir = get_session_context()
90         current_thread = get_thread_context()
91
92         print(f"[{user_name}] 2. 在深层函数中获取上下文:")
93         print(f"    - 目录: {current_dir}")
94         print(f"    - 线程: {current_thread}")
95
96         if user_name in current_thread and user_name in current_dir:
97             print(f"[{user_name}] 3. ✅ 验证成功！数据是正确的！")
98         else:
99             print(f"[{user_name}] 3. ❌ 验证失败！数据串台了！")
100
101     async def main():
102         print("--- 开始并发测试 ---\n")
103         # 同时启动两个任务，模拟并发
104         task1 = asyncio.create_task(process_user_request("张三", "/data/zhangsan"))
105         task2 = asyncio.create_task(process_user_request("李四", "/data/lisi"))
106
107         await asyncio.gather(task1, task2)
108         print("\n--- 测试结束 ---")
109
110     asyncio.run(main())
```

3.4.2.2 实时监控 (api/monitor.py)

说明：这是一个单例（Singleton）类，负责将智能体的内部思考过程实时推送到前端 WebSocket。

```
1  import datetime
2  import asyncio
3  from typing import Any, Dict, Optional
4  from fastapi import WebSocket
5  from api.context import get_thread_context
6
7  # 尝试导入全局运行时（用于脚本模式下的流式输出）
8  try:
9     import builtins
10 except ImportError:
11     builtins = None
12
13 class ToolMonitor:
14     """
15     工具监控类，用于在工具执行过程中上报进度和状态。
16     设计为单例模式，可在任何工具中直接导入使用。
17     兼容 FastAPI WebSocket 和 脚本运行时的 stream_writer。
18
19     使用示例：
20     from api.monitor import monitor
21
22     def my_tool(arg1):
23         monitor.report_start("my_tool", {"arg1": arg1})
24         ...
25         monitor.report_running("my_tool", "正在处理数据...", progress=0.5)
26         ...
27         monitor.report_end("my_tool", result)
28     """
29     _instance = None
30
31     def __new__(cls):
32         if cls._instance is None:
33             cls._instance = super(ToolMonitor, cls).__new__(cls)
34             cls._instance.websocket_manager = None # 预留给 FastAPI WebSocketManager
35         return cls._instance
36
37     def set_websocket_manager(self, manager):
```

```
38     """设置 FastAPI 的 WebSocket 管理器"""
39     self.websocket_manager = manager
40
41 def _emit(self, event_type: str, message: str, data: Optional[Dict[str, Any]] = None):
42     """内部发送方法"""
43     payload = {
44         "type": "monitor_event",
45         "event": event_type,
46         "message": message,
47         "data": data or {},
48         "timestamp": datetime.datetime.now().isoformat()
49     }
50
51     # 1. 优先尝试通过 FastAPI WebSocket 发送（定向推送）
52     if self.websocket_manager:
53         try:
54             # 获取当前线程 ID
55             thread_id = get_thread_context()
56
57             # 确保 loop 已加载
58             manager_loop = self.websocket_manager.get_loop()
59
60             if manager_loop:
61                 if thread_id:
62                     # 检查当前是否在同一个事件循环中
63                     try:
64                         current_loop = asyncio.get_running_loop()
65                     except RuntimeError:
66                         current_loop = None
67
68                     if current_loop and current_loop == manager_loop:
69                         # 如果在同一个循环中（例如在 create_task 中运行），直接创建任务
70                         current_loop.create_task(
71                             self.websocket_manager.send_to_thread(payload, thread_id)
72                         )
73                     else:
74                         # 如果在不同线程，使用 threadsafe 方法
75                         asyncio.run_coroutine_threadsafe(
76                             self.websocket_manager.send_to_thread(payload, thread_id),
77                             manager_loop
78                         )
79                 else:
80                     # 如果没有 thread_id，说明可能是系统级消息，或者未上下文环境
81                     pass
82             except Exception as e:
83                 print(f"[Monitor] WebSocket send failed: {e}")
84
85     # 2. 尝试通过全局 runtime 输出（DeepAgents 脚本模式）
86     # 这使得 simple_agents.py 中的 MockRuntime 能接收到数据
87     if builtins and hasattr(builtins, 'runtime') and hasattr(builtins.runtime, 'stream_writer'):
88         try:
89             builtins.runtime.stream_writer(payload)
90         except Exception:
91             pass
92
93     # 3. 控制台保底输出（方便调试）
94     # 加上特殊前缀，方便肉眼识别
95     print(f"\n[Monitor:{event_type}] {message}")
96
97 def report_tool(self, tool_name: str, args: Dict[str, Any] = None):
98     """报告工具开始执行"""
99     self._emit("tool_start", f"开始执行工具: {tool_name}", {"tool_name": tool_name, "args": args})
100
101 def report_assistant(self, assistant_name: str, args: Dict[str, Any] = None):
102     """报告正在调用的子智能体进度"""
103     self._emit("assistant_call", f"正在调用助手: {assistant_name}", {"assistant_name": assistant_name, "args":
args})
104
105 def report_task_result(self, result: str):
```

```
106         """报告任务最终结果"""
107         self._emit("task_result", "任务执行完成", {"result": result})
108
109     def report_session_dir(self, path: str):
110         """报告任务工作目录"""
111         self._emit("session_created", f"工作目录已创建: {path}", {"path": path})
112
113 # 全局单例实例
114 monitor = ToolMonitor()
115
116 class ConnectionManager:
117     def __init__(self):
118         self.active_connections: Dict[str, WebSocket] = {}
119         # 延迟绑定 loop, 防止初始化时 loop 不一致
120         self.loop = None
121
122     def get_loop(self):
123         """懒加载获取当前运行的事件循环"""
124         if self.loop is None:
125             try:
126                 self.loop = asyncio.get_running_loop()
127                 # 同时设置 monitor 的 manager (确保双向绑定)
128                 monitor.set_websocket_manager(self)
129                 print(f"[Monitor] ConnectionManager auto-bound to loop: {id(self.loop)}")
130             except RuntimeError:
131                 print("[Monitor] warning: No running event loop found yet.")
132         return self.loop
133
134     async def connect(self, websocket: WebSocket, thread_id: str):
135         # 每次连接时尝试获取/更新 loop
136         self.get_loop()
137
138         await websocket.accept()
139         self.active_connections[thread_id] = websocket
140         print(f"Client connected: {thread_id}")
141
142     def disconnect(self, websocket: WebSocket, thread_id: str):
143         if thread_id in self.active_connections:
144             del self.active_connections[thread_id]
145             print(f"Client disconnected: {thread_id}")
146
147     async def send_personal_message(self, message: str, websocket: WebSocket):
148         await websocket.send_text(message)
149
150     async def send_to_thread(self, message: dict, thread_id: str):
151         if thread_id in self.active_connections:
152             websocket = self.active_connections[thread_id]
153             await websocket.send_json(message)
154
155 manager = ConnectionManager()
```

3.4.3 配置文件

创建文件：根路径/.env

```
1 #RAGFLOW_API_URL=http://121.4.54.247
2 #RAGFLOW_API_KEY=ragflow-g3YmVhYzEyNGNlNDExZjBhMWEwNzZjYT
3 RAGFLOW_API_URL=http://129.211.218.165
4 # 你的 RAGFlow 服务地址
5 RAGFLOW_API_KEY=ragflow-gyMTY2NmZMTA1ZDExZjE4OWZkNWUwNj
6 # 你的 API 密钥
7 # LLM配置
8 OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
9 OPENAI_API_KEY=sk-6296bb4dab98463689911fd07a973c97
10 LLM_QWEN2.5=qwen2.5-14b-instruct
11 LLM_QWEN3=qwen3-32b
12 LLM_QWEN_MAX=qwen-max
13
14 #tavily-api-key
```

```
15 TAVILY_API_KEY=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
16
17 # 数据库相关配置
18 MYSQL_USER=root
19 MYSQL_PASSWORD=root
20 MYSQL_DATABASE=pharma_db
21 MYSQL_HOST=localhost
22 MYSQL_PORT=3306
```

3.4.4 导入工具

文件: `utils/word_converter.py`

负责将md转成pdf的工具类

```
1 import logging
2 import os
3 from pathlib import Path
4 import time
5
6 try:
7     import markdown
8     import win32com.client
9     import pythoncom
10 except ImportError:
11     pass
12
13 def convert_md_to_pdf_via_word(md_abs_path: Path, pdf_abs_path: Path) -> str:
14     """
15     使用 Microsoft word COM 接口将 Markdown 转换为 PDF。
16     依赖: pywin32, markdown
17     """
18     temp_html_path = md_abs_path.with_suffix('.temp.html')
19     word_app = None
20
21     try:
22         # 1. MD 转 HTML
23         with open(md_abs_path, 'r', encoding='utf-8') as f:
24             md_content = f.read()
25
26             html_body = markdown.markdown(md_content, extensions=['tables', 'fenced_code'])
27             html_content = f"""
28             <html>
29             <head>
30                 <meta charset="UTF-8">
31                 <style>
32                     body {{ font-family: "Microsoft YaHei", "SimHei", sans-serif; }}
33                     table {{ border-collapse: collapse; width: 100%; }}
34                     th, td {{ border: 1px solid black; padding: 8px; }}
35                     pre {{ background-color: #f5f5f5; padding: 10px; border-radius: 4px; }}
36                     code {{ font-family: "Consolas", "Monaco", monospace; }}
37                 </style>
38             </head>
39             <body>
40                 {html_body}
41             </body>
42             </html>
43             """
44
45             with open(temp_html_path, 'w', encoding='utf-8') as f:
46                 f.write(html_content)
47
48             # 2. 调用 word COM
49             pythoncom.CoInitialize()
50             word_app = win32com.client.Dispatch('Word.Application')
51             word_app.Visible = False
52             word_app.DisplayAlerts = False
53
54             doc = word_app.Documents.Open(str(temp_html_path.resolve()))
55             doc.SaveAs(str(pdf_abs_path.resolve()), FileFormat=17) # wdFormatPDF = 17
```

```
56         doc.Close(SaveChanges=0)
57
58         if pdf_abs_path.exists():
59             return f"成功转换: {pdf_abs_path} (word引擎)"
60         else:
61             return f"转换完成但未生成文件: {pdf_abs_path}"
62
63     except ImportError:
64         return "缺少依赖库, 请安装: pip install pywin32 markdown"
65     except Exception as e:
66         logging.error(f"Word转换PDF失败: {e}", exc_info=True)
67         return f"转换失败: {str(e)}"
68
69     finally:
70         # 3. 资源清理
71         if word_app:
72             try:
73                 word_app.Quit()
74             except:
75                 pass
76
77         if temp_html_path.exists():
78             try:
79                 temp_html_path.unlink()
80             except:
81                 pass
82
83         try:
84             pythoncom.CoUninitialize()
85         except:
86             pass
```

文件: `utils/path_utils.py`

处理路径工具，确保路径处于当前会话下！并且去掉大模型生成的虚拟路径！

```
1  import os
2  from pathlib import Path
3  from typing import Optional
4
5
6  def resolve_path(filename: str, session_dir: Optional[str] = None) -> str:
7      import os
8      from pathlib import Path
9      from typing import Optional
10
11  def resolve_path(filename: str, session_dir: Optional[str] = None) -> str:
12      """
13      统一的文件路径解析工具方法。
14
15      核心功能：
16      1. 清洗虚拟路径前缀 (/workspace, /mnt/data, /home/user)
17      2. 识别 updated/ 目录，优先相对于项目根目录解析
18      3. 结合 session_dir 处理相对/绝对路径，保证路径隔离
19      4. 防止路径嵌套 (session_id/session_id)
20
21      场景示例（基于测试环境：windows系统、session_dir=D:/Project/output/session_123、CWD=D:/Project）：
22      | 输入场景 | filename | session_dir | 核心操作
23      |-----|-----|-----|-----|
24      | 虚拟路径清洗 | /workspace/report.md | D:/Project/output/session_123 | 剥离/workspace
25      → 拼接到会话目录 | D:/Project/output/session_123/report.md |
26      | updated/ 特殊处理 | abc/updated/upload/file.pdf | D:/Project/output/session_123 | 提取updated/后
27      路径 → 解析到CWD | D:/Project/updated/upload/file.pdf |
28      | 无会话目录 | sub/test.md | None | 直接解析为CWD下
29      绝对路径 | D:/Project/sub/test.md |
30      | 绝对路径（会话内） | D:/Project/output/session_123/sub/report.md | D:/Project/output/session_123 | 验证在会话
31      内 → 无嵌套 → 直接返回 | D:/Project/output/session_123/sub/report.md |
```

```
28 | 绝对路径（会话外） | D:/OtherDir/file.md | D:/Project/output/session_123 | 验证不在会话内 →
保留原路径 | D:/OtherDir/file.md |
29 | windows Unix风格绝对路径 | /sub/test.md | D:/Project/output/session_123 | /开头无盘符 →
拼接到会话目录 | D:/Project/output/session_123/sub/test.md |
30 | 路径嵌套防护 | D:/Project/output/session_123/session_123/report.md | D:/Project/output/session_123 |
检测连续session_123 → 修正路径 | D:/Project/output/session_123/report.md |
31 | 相对路径（含session名） | session_123/report.md | D:/Project/output/session_123 | 含session名 → 防
止嵌套 → 会话目录+文件名 | D:/Project/output/session_123/report.md |
32 | 相对路径（output前缀） | output/report.md | D:/Project/output/session_123 | 含output前缀 →
会话目录+文件名 | D:/Project/output/session_123/report.md |
33 | 普通相对路径 | sub1/sub2/test.md | D:/Project/output/session_123 | 无特殊标识 → 拼
接到会话目录 | D:/Project/output/session_123/sub1/sub2/test.md |
34 | 虚拟路径+updated | /mnt/data/updated/doc.md | D:/Project/output/session_123 | 剥离/mnt/data
→ 触发updated处理 | D:/Project/updated/doc.md |
35 | Linux系统绝对路径 | /home/user/test.md | /data/session_123 (Linux) | 剥离/home/user
→ 拼接到Linux会话目录 | /data/session_123/test.md |

36
37 Args:
38     filename (str): 输入的文件名或路径
39     session_dir (str, optional): 会话上下文目录
40
41 Returns:
42     str: 解析后的绝对路径
43     ""
44
45 path = Path(filename)
46 path_str = filename.replace("\\", "/") # 统一处理字符串匹配
47
48 # 1. 虚拟路径清洗
49 virtual_prefixes = ["/workspace", "/mnt/data", "/home/user"]
50 for prefix in virtual_prefixes:
51     if path_str.startswith(prefix):
52         # 去掉前缀
53         cleaned = path_str[len(prefix):].lstrip("/")
54         path = Path(cleaned)
55         path_str = str(path).replace("\\", "/")
56         break
57
58 # 2. 特殊处理：updated/（用户上传文件）
59 # 只要路径中包含 updated/，就提取其后半部分，并相对于 CWD 解析
60 if "updated/" in path_str:
61     idx = path_str.find("updated/")
62     relative_part = path_str[idx:]
63     return str(Path(relative_part).resolve())
64
65 if not session_dir:
66     return str(path.resolve())
67
68 session_path = Path(session_dir).resolve()
69 session_name = session_path.name
70
71 # 3. 结合 Session Context
72
73 # 检测 Unix 风格绝对路径（以 / 开头）
74 is_unix_abs = path_str.startswith("/")
75
76 # 如果是绝对路径（Windows带盘符 或 Unix/开头）
77 if path.is_absolute() or (os.name == 'nt' and is_unix_abs):
78     # windows 特殊情况：以 / 开头但无盘符，视为相对路径
79     if os.name == 'nt' and is_unix_abs and not path.drive:
80         full_path = session_path / path_str.lstrip("/")
81     else:
82         full_path = path.resolve()
83
84 # 检查是否在 session 目录下
85 try:
86     # 判断 full_path 是否是 session_path 的子路径
87     if session_path in full_path.parents or full_path == session_path:
88         # 检查嵌套（例如 ../session_abc/session_abc/file.txt）
89         # 检查路径部分中是否有连续重复的 session_name
```

```
89         parts = full_path.parts
90         for i in range(len(parts) - 1):
91             if parts[i] == session_name and parts[i + 1] == session_name:
92                 # 发现嵌套,修正为 session_dir / filename
93                 return str(session_path / full_path.name)
94         return str(full_path)
95     except Exception:
96         pass
97
98     # 绝对路径但不在 session_dir 下 -> 保持原样
99     return str(full_path)
100
101 else:
102     # 相对路径处理
103     parts = path.parts
104
105     # 检查是否包含 session_name (避免重复) 或 output/ 前缀
106     if session_name in parts:
107         return str(session_path / path.name)
108
109     if parts and parts[0] == "output":
110         return str(session_path / path.name)
111
112     # 默认: 拼接到 session_dir
113     return str(session_path / path)
114
```

四、功能开发&测试

4.1 大模型准备

4.1.1 检查配置文件

文件: `.env`

```
1 # LLM配置
2 # 统一使用前文
3 OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
4 OPENAI_API_KEY=sk-key
5 LLM_QWEN2.5=qwen2.5-14b-instruct
6 LLM_QWEN3=qwen3-32b
7 LLM_QWEN_MAX=qwen-max
```

4.1.2 创建大模型对象

文件: `agent/llm.py`

```
1 from dotenv import load_dotenv, find_dotenv
2 import os
3 from langchain.chat_models import init_chat_model
4
5 load_dotenv(find_dotenv())
6
7 model = init_chat_model(
8     model= os.getenv("LLM_QWEN_MAX"),
9     model_provider="openai"
10 )
```

4.2 提示词配置&加载

4.2.1 创建提示词配置文件 (yaml)

文件: `prompt/prompts.yml` 【简化, 独立智能体进行优化提示词】

```
1 # Main Agent Configuration
2 main_agent:
3   system_prompt: |
4     你是一个沃华医药公司的智能团队负责人，负责协调三个专家助手完成复杂任务
5     先占位~~~~~
```

```
6 # Sub Agents Configuration
7 sub_agents:
8   tavily:
9     name: "网络搜索助手"
10    description: |
11      负责进行网络知识搜索的智能体助手
12    system_prompt: |
13      你是一个专业的网络信息查询助手
14  db:
15    name: "数据库查询助手"
16    description: |
17      负责进行数据库查询的智能体助手。
18    system_prompt: |
19      你是一个专业的数据库查询助手
20  ragflow:
21    name: "RAGFlow助手"
22    description: |
23      负责与RAGFlow知识库进行交互的智能体助手
24    system_prompt: |
25      你是一个专业的RAGFlow知识库助手
```

❏ 被称为**块折叠标量**（**Literal Block Scalar**）

它是YAML里专门用来处理**多行文本**的语法（用❏开头），核心作用是：让你能把需要换行的内容（比如prompt、描述、长文本）清晰地分行写，YAML解析时会**完整保留换行格式**，不会把所有内容挤成一行，特别适合写大段的文本配置

4.2.2 读取提示词文件

文件：agent/prompts.py

```
1 import yaml
2 from pathlib import Path
3
4
5 # 加载YAML格式的提示词配置文件
6 def load_prompt(file_path):
7     """
8     读取并加载YAML格式的提示词配置文件
9     Args:
10         file_path (str/Path): YAML配置文件的路径
11     Returns:
12         dict: 解析后的YAML配置字典，包含主智能体和子智能体的提示词配置
13     """
14     # 以UTF-8编码打开文件，避免中文乱码
15     with open(file_path, 'r', encoding="utf-8") as f:
16         """
17         这是 safe_load 区别于 load 的核心（也是为什么必须用它）：
18         yaml.load(): 不安全，会解析 YAML 中的「自定义对象 / 执行代码」，如果加载的 YAML 文件被恶意篡改（比如插入了执行系统命令的代码），会导致服务器被攻击、数据泄露；
19         yaml.safe_load(): 仅解析 YAML 标准数据类型（字符串、数字、字典、列表、布尔值等），完全禁止解析 / 执行任何自定义对象、函数、代码，从根源避免安全风险。
20         """
21         # 使用safe_load保证加载安全，加载成字典类型
22         return yaml.safe_load(f)
23
24
25 # 获取当前脚本文件的父级目录的上一级（项目根目录）
26 # Path(__file__): 当前脚本文件的绝对路径
27 # parents[1]: 向上追溯两级目录，定位到项目根目录
28 root_path = Path(__file__).parents[1]
29
30 # 拼接提示词配置文件的完整路径（根目录/prompt/prompts.yml）
31 prompt_file_path = root_path / "prompt" / "prompts.yml"
32
33 # 加载YAML配置文件内容
34 prompt_config_content = load_prompt(prompt_file_path)
35 # 打印完整配置内容，用于调试验证加载是否成功
36 print(f"prompt_config_content: {prompt_config_content}")
37
38 # 从总配置中提取主智能体的配置（对应prompts.yml中的main_agent节点）
```

```
39 main_agent_config = prompt_config_content["main_agent"]
40 # 从总配置中提取子智能体的配置（对应prompts.yml中的sub_agents节点）
41 sub_agents_config = prompt_config_content["sub_agents"]
42
43 # 打印拆分后的配置，验证核心配置节点是否正确提取
44 print(f"main_agent_config: {main_agent_config} ， \nsub_agents_config: {sub_agents_config}")
```

4.3 Subagents实现

4.3.1 网络搜索助手（network_search_agent）

4.3.1.1 完善信息

• Agent描述：

- 1

负责进行网络知识搜索的智能体助手，当需要从网络中查询数据的时候，可以执行数据检索，在检索后会返回一段检索结果。
- 2

在需要进行非内部信息（不是数据库数据和rag的数据）的公开信息查询时务必使用此助手进行查询。

• 工具涵盖：

internet_search: 根据问题进行网络查询，当需要获取外部互联网的公开信息、最新新闻或特定主题数据时使用此工具。

• 提示词思路：

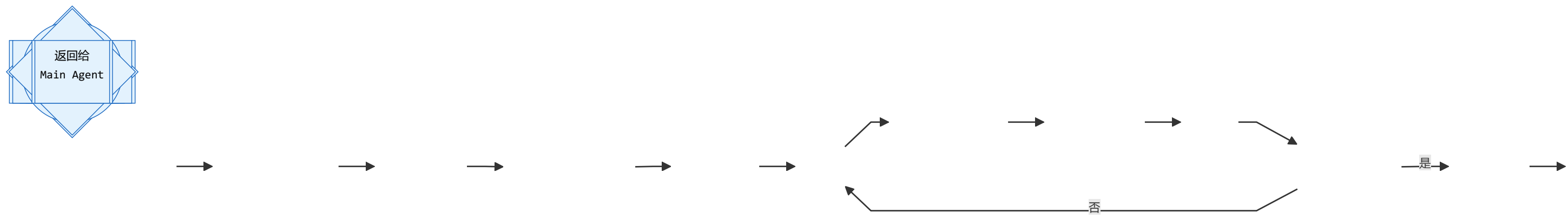
- 强制多维视角：Prompt 中明确要求 "至少检索3个角度"，防止模型只搜一次就草草了事，强迫其进行发散性思维。
- 防止死循环：设置 "最多进行5次检索" 的硬性约束，防止模型在找不到答案时无限重试，消耗 Token。
- 广度优先：强调检索 "非内部的公开信息"，明确了其边界，不与数据库和 RAG 助手冲突。
- 提示词参考：

- 1

你是一个专业的网络信息查询助手，你可以根据用户的问题，从互联网中检索相关信息，你掌握的工具包括 internet_search 工具，此工具可以根据用户的问题，从互联网中检索非内部的公开信息。
- 2

在检索网络知识的时候，至少检索3个角度的该问题，一共最多进行5次检索，如果超过5次，则不允许继续检索

• 执行策略：Agent 接收任务 -> 思考拆解搜索关键词 -> 调用搜索工具 -> 观察结果 -> 决定是否需要补充搜索 -> 汇总信息。



```
1 sub_agents:
2   tavily:
3     name: "网络搜索助手"
4     description: |
5       负责进行网络知识搜索的智能体助手，当需要从网络中查询数据的时候，可以执行数据检索，在检索后会返回一段检索结果。
6       在需要进行非内部信息的公开信息查询时务必使用此助手进行查询。
7     system_prompt: |
8       你是一个专业的网络信息查询助手，你可以根据用户的问题，从互联网中检索相关信息，你掌握的工具包括 internet_search 工具，此工具可以根据用户的问题，从互联网中检索非内部的公开信息。
9       在检索网络知识的时候，至少检索3个角度的该问题，一共最多进行5次检索，如果超过5次，则不允许继续检索
```

4.3.1.2 tavily搜索工具

步骤1：定义tavily访问api_key

文件: .env

直接打开官网：<https://app.tavily.com/> 免费版：每月 1000 次请求，完全够用学习 / 开发

```
1 #tavily-api-key
2 TAVILY_API_KEY=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
```

步骤2：tavily_tools定义&实现

位置: tools/tavily_tools.py

```
1 # ===== 导入核心依赖 =====
2 # 类型注解：增强代码提示和静态检查能力
3 from typing import Literal
4 # LangChain工具装饰器：将普通函数转为 Agent 可调用的工具
```

```
5 from langchain_core.tools import tool
6 # Tavily 官方客户端：实现网络搜索核心功能
7 from tavily import TavilyClient
8
9 # 系统/第三方依赖
10 import os # 系统路径/环境变量处理
11 from dotenv import load_dotenv # 加载 .env 文件中的环境变量
12
13 # 自定义模块：工具调用埋点监控（需确保 api 模块可导入）
14 from api.monitor import monitor
15
16 # ===== 初始化配置 =====
17 # 加载项目根目录的 .env 文件，读取环境变量（如 TAVILY_API_KEY）
18 load_dotenv()
19
20 # 初始化 Tavily 客户端（安全读取环境变量中的 API Key）
21 # 注：TavilyClient 是导入的类，判断是否存在仅为防御性编程，避免导入失败导致的异常
22 if TavilyClient:
23     # 从环境变量读取 API Key，避免硬编码泄露密钥
24     tavily_client = TavilyClient(api_key=os.getenv('TAVILY_API_KEY'))
25 else:
26     # 客户端初始化失败时置为 None，后续调用时会返回明确错误
27     tavily_client = None
28
29 #定义网络搜索工具
30 @tool
31 def internet_search(
32     query: str,
33     max_results: int = 5,
34     topic: Literal["general", "news", "finace"] = "general",
35     include_raw_content: bool = False
36 ):
37     """
38     根据问题进行网络查询，当需要获取外部互联网的公开信息、最新新闻或特定主题数据时使用此工具
39     核心用途：
40         当 AI Agent 需要获取外部互联网的公开信息、时效性数据（如新闻、金融动态）时调用，
41         替代传统搜索引擎，返回更适配大模型的结构化结果。
42     参数说明：
43         query: 搜索的核心问题/关键词，例如 "2026年AI行业政策"
44         max_results: 控制返回结果数量，免费版建议不超过5
45         topic: 限定搜索内容类型，提升结果相关性
46         include_raw_content: 是否返回详细新闻，False简略版本 True详细版本
47     返回值：
48         dict: Tavily API 返回的结构化结果，包含以下核心字段：
49             - query: 原始搜索词
50             - results: 搜索结果列表，每个元素包含 url、content（摘要）、raw_content（原始内容，可选）等
51         str: 初始化失败时返回错误提示字符串
52     异常处理：
53         捕获搜索过程中的所有异常并重新抛出，确保 Agent 能感知到搜索失败并处理
54     """
55     if not tavily_client:
56         return "Error: 'tavily-python' library is not installed."
57     monitor.report_tool("网络搜索工具", {"网络搜索工具": query})
58     try:
59         results = tavily_client.search(
60             query,
61             max_results=max_results,
62             include_raw_content=include_raw_content,
63             topic=topic,
64         )
65         return results
66     except Exception as e:
67         raise e
```

4.3.1.3 定义network_search_agent

文件： agent/sub_agents/network_search_agent.py

```
1
2 from agent.prompts import sub_agents_config
3 from tools.tavily_tools import internet_search
4
5 network_search_agent = {
6     "name":sub_agents_config["tavily"].get("name",""),
7     "description":sub_agents_config["tavily"].get("description",""),
8     "system_prompt":sub_agents_config["tavily"].get("system_prompt",""),
9     "tools": [internet_search]
10 }
```

4.3.2 数据库查询助手（database_query_agent）

4.3.2.1 完善信息

- **核心职责：**负责查询企业内部结构化数据（如商品库存、销售记录），解决"具体是多少"的精度问题。
- **技术栈：**Text-to-SQL (LLM 生成 SQL) + MySQL Connector。
- **Agent描述：**

- 1

负责进行数据库查询的智能体助手。它可以查看数据库中的表结构，读取表数据，并执行自定义SQL查询以获取精确的业务数据。
- 2

数据库中包含了企业的空调商品具体数据，可以看到所有特定商品的一切详细信息。但是数据库中不包含概括性的知识，只包含具体商品的信息。

- **工具描述：**
 - list_sql_tables: 列出配置的 MySQL 数据库中所有可用的表，这是了解数据库结构的第一步。
 - get_table_data: 读取指定 MySQL 表的前 100 行数据，用于快速预览数据内容。
 - execute_sql_query: 执行自定义 SQL 查询，当需要复杂的筛选、联接或聚合时使用此工具。
- **提示词思路：**
 - **防幻觉机制：** Prompt 强制规定了 "Step 1: list_tables" 的动作。LLM 只有知道了真实的表名，生成的 SQL 才是可执行的，避免了臆造表名的常见幻觉。
 - **数据理解：** 要求 "Step 2: get_table_data" 预览数据。这让 LLM 理解字段的具体格式（如日期是 '2026-01' 还是 '2026/01'），保证 WHERE 条件的准确性。
 - **只读权限：** Prompt 强调 "检索信息"，隐含了不进行 UPDATE/DELETE 操作的安全边界。
 - **提示词参考：**

- 1

你是一个专业的数据库查询助手。你可以直接与MySQL数据库交互来检索信息。
- 2

你掌握的工具包括：
- 3

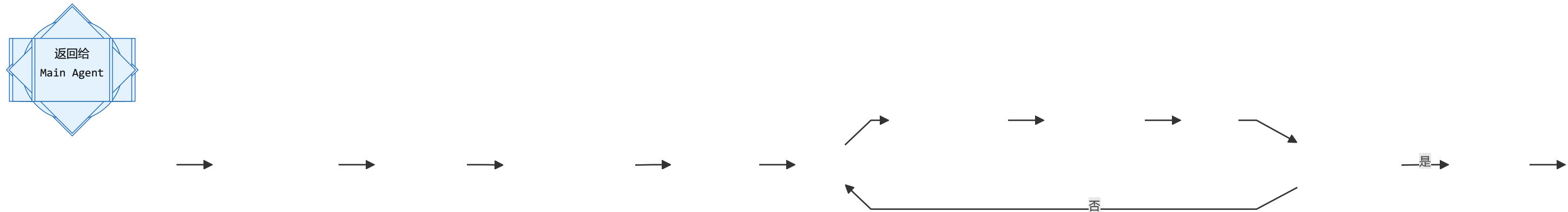
1. list_sql_tables：列出数据库中所有可用的表，这是了解数据库结构的第一步。
- 4

2. get_table_data：读取指定表的前100行数据，用于快速预览数据内容。
- 5

3. execute_sql_query：执行自定义SQL查询。当需要复杂的筛选、联接或聚合时使用此工具。
- 6

通常的工作流程是：先列出可用表，确认表名；如果需要，预览表数据了解字段；最后编写并执行SQL查询以回答用户问题。

- **执行策略** (三步走)：查表结构 -> 预览数据 -> 执行查询。
- **流程图：**



```
1 sub_agents:
2   db:
3     name: "数据库查询助手"
4     description: |
5       负责进行数据库查询的智能体助手。它可以查看数据库中的表结构，读取表数据，并执行自定义SQL查询以获取精确的业务数据。
6       数据库中包含了企业的空调商品具体数据，可以看到所有特定商品的一切详细信息。但是数据库中不包含概括性的知识，只包含具体商品的信息。
7     system_prompt: |
8       你是一个专业的数据库查询助手。你可以直接与MySQL数据库交互来检索信息。
9       你掌握的工具包括：
10      1. list_sql_tables：列出数据库中所有可用的表，这是了解数据库结构的第一步。
11      2. get_table_data：读取指定表的前100行数据，用于快速预览数据内容。
12      3. execute_sql_query：执行自定义SQL查询。当需要复杂的筛选、联接或聚合时使用此工具。
13      通常的工作流程是：先列出表，确认表名；如果需要，预览表数据了解字段；最后编写并执行SQL查询以回答用户问题。
```

4.3.2.2 数据库搜索工具

步骤1：准备数据库数据

脚本位置： 项目/sql/company_data.sql

```
1  -- 制药公司核心业务数据库设计
2  -- 包含：药品信息、库存管理、销售记录
3  -- 适用于 DeepAgents 结构化数据检索
4
5  CREATE DATABASE IF NOT EXISTS pharma_db;
6  USE pharma_db;
7
8  -- 1. 药品信息表 (Drug Details)
9  -- 记载每一种药品的详情信息
10 CREATE TABLE drugs (
11     drug_id INT PRIMARY KEY AUTO_INCREMENT,
12     generic_name VARCHAR(100) NOT NULL,      -- 通用名 (如 布洛芬缓释胶囊)
13     brand_name VARCHAR(100),                  -- 商品名 (如 芬必得)
14     approval_number VARCHAR(50),              -- 批准文号 (国药准字H...)
15     specifications VARCHAR(100),             -- 规格 (如 0.3g*24粒/盒)
16     dosage_form VARCHAR(50),                  -- 剂型 (胶囊/片剂/注射液)
17     manufacturer VARCHAR(100),                -- 生产厂家
18     therapeutic_area VARCHAR(50),            -- 治疗领域 (如 解热镇痛, 心血管)
19     description TEXT,                          -- 药品详情/适应症描述
20     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
21 );
22
23 -- 2. 库存表 (Inventory)
24 -- 记录药品的库存量, 通过 drug_id 关联
25 CREATE TABLE inventory (
26     inventory_id INT PRIMARY KEY AUTO_INCREMENT,
27     drug_id INT NOT NULL,
28
29     batch_number VARCHAR(50) NOT NULL,        -- 生产批号 (医药库存核心字段)
30     quantity_on_hand INT DEFAULT 0,           -- 当前库存量 (盒/瓶)
31     warehouse_location VARCHAR(50),           -- 仓库位置 (如 A区-01架)
32
33     production_date DATE,                      -- 生产日期
34     expiry_date DATE,                          -- 有效期至 (用于预警)
35     last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
36
37     FOREIGN KEY (drug_id) REFERENCES drugs(drug_id) ON DELETE CASCADE
38 );
39
40 -- 3. 药品销售情况表 (Sales Records)
41 -- 记录每个药品的销售情况, 通过 drug_id 关联
42 CREATE TABLE sales_records (
43     sale_id INT PRIMARY KEY AUTO_INCREMENT,
44     drug_id INT NOT NULL,
45
46     sale_date DATE NOT NULL,                   -- 销售日期
47     quantity_sold INT NOT NULL,                -- 销售数量
48     unit_price DECIMAL(10, 2),                 -- 销售单价
49     total_amount DECIMAL(15, 2),              -- 销售总额
50
51     customer_name VARCHAR(100),                -- 客户名称 (如 xx第一人民医院, xx大药房)
52     region VARCHAR(50),                        -- 销售区域 (用于区域分析)
53     sales_rep VARCHAR(50),                     -- 销售代表
54
55     FOREIGN KEY (drug_id) REFERENCES drugs(drug_id) ON DELETE CASCADE
56 );
57
58 -- --- 插入模拟数据 (Mock Data) ---
59
60 -- 1. 插入 10 种药品信息 (全中文, 假设均为我司生产)
61 INSERT INTO drugs (generic_name, brand_name, approval_number, specifications, dosage_form, manufacturer,
62     therapeutic_area, description)
63 VALUES
```

```
63 ('阿莫西林胶囊', '阿莫仙', '国药准字H20051234', '0.25g*24粒', '胶囊剂', '本公司制药', '抗生素', '用于治疗敏感菌引起的上呼吸道感染、泌尿生殖道感染等。'),
64 ('布洛芬缓释胶囊', '芬必得', '国药准字H10900089', '0.3g*20粒', '胶囊剂', '本公司制药', '解热镇痛', '用于缓解轻至中度疼痛如头痛、关节痛、偏头痛、牙痛、肌肉痛、神经痛、痛经。也用于普通感冒或流行性感冒引起的发热。'),
65 ('盐酸二甲双胍片', '格华止', '国药准字H20023345', '0.5g*48片', '片剂', '本公司制药', '糖尿病', '首选用于单纯饮食控制不满意的2型糖尿病患者，尤其是肥胖者。'),
66 ('阿托伐他汀钙片', '立普妥', '国药准字H20055567', '20mg*7片', '片剂', '本公司制药', '心血管', '原发性高胆固醇血症患者，包括家族性高胆固醇血症（杂合子型）或混合性高脂血症。'),
67 ('磷酸奥司他韦胶囊', '达菲', '国药准字H20090123', '75mg*10粒', '胶囊剂', '本公司制药', '抗病毒', '用于成人和1岁及1岁以上儿童的甲型和乙型流感治疗。'),
68 ('注射用头孢曲松钠', '罗氏芬', '国药准字H10920012', '1.0g/支', '注射剂', '本公司制药', '抗生素', '用于敏感致病菌所致的下呼吸道感染、尿路、胆道感染，以及腹腔感染、盆腔感染、皮肤软组织感染、骨和关节感染、败血症、脑膜炎等。'),
69 ('蒙脱石散', '思密达', '国药准字H20000456', '3g*10袋', '散剂', '本公司制药', '消化系统', '用于成人及儿童急、慢性腹泻。'),
70 ('硝苯地平控释片', '拜新同', '国药准字H20100345', '30mg*30片', '片剂', '本公司制药', '高血压', '1.高血压。2.冠心病。慢性稳定型心绞痛（劳累性心绞痛）。'),
71 ('阿司匹林肠溶片', '拜阿司匹灵', '国药准字J20130078', '100mg*30片', '片剂', '本公司制药', '心血管', '降低急性心肌梗死疑似患者的发病风险；预防心肌梗死复发。'),
72 ('连花清瘟胶囊', '连花清瘟', '国药准字Z20040063', '0.35g*24粒', '胶囊剂', '本公司制药', '中成药/感冒', '清瘟解毒，宣肺泄热。用于治疗流行性感冒属热毒袭肺证，症见：发热或高热，恶寒，肌肉酸痛，鼻塞流涕，咳嗽，头痛，咽干咽痛。');
73
74 -- 2. 插入库存数据
75 -- 规则：每种药 3 个批次（2501批，2506批，2511批）
76 -- 时间：全部平移至 2025 年生产，有效期至 2027 年
77
78 INSERT INTO inventory (drug_id, batch_number, quantity_on_hand, warehouse_location, production_date, expiry_date)
79 VALUES
80 -- 1. 阿莫西林
81 (1, 'MY-250101-A', 5000, '北京一号库-A区', '2025-01-01', '2027-01-01'),
82 (1, 'MY-250615-B', 8000, '北京二号库-B区', '2025-06-15', '2027-06-14'),
83 (1, 'MY-251120-C', 12000, '天津一号库-A区', '2025-11-20', '2027-11-19'),
84
85 -- 2. 布洛芬
86 (2, 'MY-250101-A', 2000, '天津二号库-冷藏区', '2025-01-01', '2027-01-01'),
87 (2, 'MY-250615-B', 15000, '北京一号库-C区', '2025-06-15', '2027-06-14'),
88 (2, 'MY-251120-C', 30000, '天津一号库-A区', '2025-11-20', '2027-11-19'),
89
90 -- 3. 二甲双胍
91 (3, 'MY-250101-A', 3000, '北京二号库-B区', '2025-01-01', '2027-01-01'),
92 (3, 'MY-250615-B', 4500, '天津二号库-B区', '2025-06-15', '2027-06-14'),
93 (3, 'MY-251120-C', 6000, '北京一号库-A区', '2025-11-20', '2027-11-19'),
94
95 -- 4. 阿托伐他汀
96 (4, 'MY-250101-A', 1000, '天津一号库-贵重药区', '2025-01-01', '2027-01-01'),
97 (4, 'MY-250615-B', 2500, '北京二号库-贵重药区', '2025-06-15', '2027-06-14'),
98 (4, 'MY-251120-C', 4000, '天津二号库-贵重药区', '2025-11-20', '2027-11-19'),
99
100 -- 5. 奥司他韦
101 (5, 'MY-250101-A', 500, '北京一号库-急救药区', '2025-01-01', '2027-01-01'),
102 (5, 'MY-250615-B', 5000, '天津一号库-急救药区', '2025-06-15', '2027-06-14'),
103 (5, 'MY-251120-C', 20000, '北京二号库-急救药区', '2025-11-20', '2027-11-19'),
104
105 -- 6. 头孢曲松
106 (6, 'MY-250101-A', 2000, '天津二号库-阴凉库', '2025-01-01', '2027-01-01'),
107 (6, 'MY-250615-B', 3500, '北京二号库-阴凉库', '2025-06-15', '2027-06-14'),
108 (6, 'MY-251120-C', 5000, '天津一号库-阴凉库', '2025-11-20', '2027-11-19'),
109
110 -- 7. 蒙脱石散
111 (7, 'MY-250101-A', 4000, '北京一号库-普药区', '2025-01-01', '2027-01-01'),
112 (7, 'MY-250615-B', 8000, '天津二号库-普药区', '2025-06-15', '2027-06-14'),
113 (7, 'MY-251120-C', 12000, '北京二号库-普药区', '2025-11-20', '2027-11-19'),
114
115 -- 8. 硝苯地平
116 (8, 'MY-250101-A', 1500, '天津一号库-慢病区', '2025-01-01', '2027-01-01'),
117 (8, 'MY-250615-B', 3000, '北京一号库-慢病区', '2025-06-15', '2027-06-14'),
118 (8, 'MY-251120-C', 5000, '天津二号库-慢病区', '2025-11-20', '2027-11-19'),
119
120 -- 9. 阿司匹林
121 (9, 'MY-250101-A', 2000, '北京二号库-常温区', '2025-01-01', '2027-01-01'),
122 (9, 'MY-250615-B', 4500, '天津一号库-常温区', '2025-06-15', '2027-06-14'),
```

```
123 (9, 'MY-251120-C', 7000, '北京一号库-常温区', '2025-11-20', '2027-11-19'),
124
125 -- 10. 连花清瘟
126 (10, 'MY-250101-A', 10000, '天津二号库-防疫专区', '2025-01-01', '2027-01-01'),
127 (10, 'MY-250615-B', 50000, '北京二号库-防疫专区', '2025-06-15', '2027-06-14'),
128 (10, 'MY-251120-C', 100000, '天津一号库-防疫专区', '2025-11-20', '2027-11-19');
129
130 -- 3. (可选) 为这 10 种药品初始化销售记录 - 待下一步生成
131
132 INSERT INTO sales_records (drug_id, sale_date, quantity_sold, unit_price, total_amount, customer_name, region,
133 sales_rep)
134 VALUES
135 -- 1. 阿莫西林
136 (1, '2025-02-15', 200, 25.00, 5000.00, '北京朝阳医院', '华北区', '北京朝阳销售部'),
137 (1, '2025-08-10', 500, 24.50, 12250.00, '天津大药房', '华北区', '天津南开销售分部'),
138 -- 2. 布洛芬
139 (2, '2025-01-20', 1000, 15.00, 15000.00, '海王星辰连锁', '华东区', '杭州滨江销售部'),
140 (2, '2025-12-05', 5000, 15.00, 75000.00, '上海华山医院', '华东区', '上海静安销售总部'),
141
142 -- 3. 二甲双胍
143 (3, '2025-03-10', 300, 35.00, 10500.00, '广州中山医院', '华南区', '广州越秀销售部'),
144 (3, '2025-09-22', 400, 35.00, 14000.00, '深圳人民医院', '华南区', '深圳罗湖销售分部'),
145
146 -- 4. 阿托伐他汀
147 (4, '2025-04-05', 100, 45.00, 4500.00, '成都华西医院', '西南区', '成都武侯销售部'),
148 (4, '2025-10-18', 150, 45.00, 6750.00, '重庆大药房', '西南区', '重庆渝中销售部'),
149
150 -- 5. 奥司他韦
151 (5, '2025-01-15', 2000, 100.00, 200000.00, '北京协和医院', '华北区', '北京东单销售部'),
152 (5, '2025-11-01', 5000, 100.00, 500000.00, '黑龙江省医院', '东北区', '哈尔滨香坊销售部'),
153
154 -- 6. 头孢曲松
155 (6, '2025-05-20', 500, 12.00, 6000.00, '武汉同济医院', '华中区', '武汉汉口销售部'),
156 (6, '2025-07-15', 600, 12.00, 7200.00, '长沙湘雅医院', '华中区', '长沙开福销售部'),
157
158 -- 7. 蒙脱石散
159 (7, '2025-06-01', 1000, 18.00, 18000.00, '杭州第一医院', '华东区', '杭州上城销售部'),
160 (7, '2025-08-25', 2000, 18.00, 36000.00, '南京鼓楼医院', '华东区', '南京鼓楼销售部'),
161
162 -- 8. 硝苯地平
163 (8, '2025-02-28', 200, 30.00, 6000.00, '西安西京医院', '西北区', '西安新城销售部'),
164 (8, '2025-11-11', 500, 30.00, 15000.00, '兰州大学第一医院', '西北区', '兰州城关销售部'),
165
166 -- 9. 阿司匹林
167 (9, '2025-03-15', 1000, 10.00, 10000.00, '济南中心医院', '华东区', '济南历下销售部'),
168 (9, '2025-09-09', 1200, 10.00, 12000.00, '青岛市立医院', '华东区', '青岛市北销售部'),
169
170 -- 10. 连花清瘟
171 (10, '2025-01-10', 10000, 20.00, 200000.00, '石家庄以岭药业', '华北区', '石家庄高新销售部'),
172 (10, '2025-12-20', 50000, 20.00, 1000000.00, '全国连锁大药房总仓', '全国', '公司大客户部');
```

步骤2：准备数据库配置

文件: `.env`

```
1 # 数据库相关配置
2 MYSQL_USER=root
3 MYSQL_PASSWORD=root
4 MYSQL_DATABASE=deepagents_database
5 MYSQL_HOST=localhost
6 MYSQL_PORT=3306
```

步骤3：mysql_tools定义&实现

```
1 import os
2 from dotenv import load_dotenv
3 from api.monitor import monitor
4 from mysql.connector import connect,Error
5 from typing import Annotated,List
```

```
6 from langchain_core.tools import tool
7
8 load_dotenv()
9
10 # 加载配置文件方便后续使用
11 def get_db_config():
12     """Get database configuration from environment variables."""
13     config = {
14         "host": os.getenv("MYSQL_HOST", "localhost"),
15         "port": int(os.getenv("MYSQL_PORT", "3306")),
16         "user": os.getenv("MYSQL_USER"),
17         "password": os.getenv("MYSQL_PASSWORD"),
18         "database": os.getenv("MYSQL_DATABASE"),
19         "charset": os.getenv("MYSQL_CHARSET", "utf8mb4"),
20         "collation": os.getenv("MYSQL_COLLATION", "utf8mb4_unicode_ci"),
21         "autocommit": True,
22         "sql_mode": os.getenv("MYSQL_SQL_MODE", "TRADITIONAL")
23     }
24     # 移除 None 值（核心必要操作）
25     config = {k: v for k, v in config.items() if v is not None}
26
27     # 补充：校验核心配置是否存在（可选但推荐）
28     required_keys = ["user", "password", "database"]
29     missing_keys = [k for k in required_keys if k not in config]
30     if missing_keys:
31         raise ValueError(f"缺失数据库核心配置：{'', '.join(missing_keys)}")
32
33     return config
34
35
36 # 定义查看数据库表的工具
37 """
38 【mysql.connector 核心 API 说明（针对 connect/cursor）】
39 1. connect 函数：
40     - 作用：建立与 MySQL 数据库的连接，返回一个 Connection 对象；
41     - 使用方式：connect(**config)，config 为包含 host/user/password 等的字典；
42     - 上下文管理器：推荐用 with 语句（with connect(**config) as conn），自动关闭连接，避免资源泄露；
43     - 核心属性/方法：
44         - conn.cursor()：创建游标对象（执行 SQL 的核心）；
45         - conn.commit()：提交事务（autocommit=True 时无需手动调用）；
46         - conn.close()：关闭连接（with 语句自动执行）。
47 2. cursor 游标对象：
48     - 作用：执行 SQL 语句、获取查询结果的核心对象；
49     - 创建方式：conn.cursor()；
50     - 上下文管理器：with conn.cursor() as cursor，自动关闭游标；
51     - 核心方法：
52         - cursor.execute(sql)：执行单条 SQL 语句（如 SHOW TABLES/SELECT/INSERT）；
53         - cursor.executemany(sql, params)：批量执行 SQL 语句（如批量插入）；
54         - cursor.close()：关闭游标（with 语句自动执行）。
55 3. 【重点】cursor 执行 DQL/DML 后的结果解析：
56     ► DQL（数据查询语言，如 SELECT/SHOW）：查询类操作，返回「数据结果集」
57     - 核心方法：
58         1. cursor.fetchall()：获取所有结果（返回列表，每个元素是元组，如 [(1, '张三'), (2, '李四')]）；
59         2. cursor.fetchone()：获取一条结果（返回元组，如 (1, '张三')，多次调用可遍历所有结果）；
60         3. cursor.fetchmany(n)：获取前 n 条结果（返回列表）；
61         4. cursor.column_names：获取查询结果的列名（列表，如 ['id', 'name']）；
62     - 解析技巧：将「列名 + 元组结果」转为字典（更易读），如 {'id': 1, 'name': '张三'}。
63     ► DML（数据操作语言，如 INSERT/UPDATE/DELETE）：修改类操作，无「数据结果集」
64     - 核心属性：
65         1. cursor.rowcount：返回受影响的行数（整数，如 INSERT 1 条返回 1，UPDATE 3 条返回 3）；
66         2. cursor.lastrowid：INSERT 操作后，返回新增记录的自增 ID（仅对有自增主键的表有效）；
67     - 解析技巧：通过 rowcount 判断操作是否生效，lastrowid 获取新增数据的主键。
68 4. 异常处理：
69     - Error：mysql.connector 专属异常类，捕获所有数据库操作异常（如连接失败、SQL 语法错误）；
70     - 推荐方式：try-except Error as e 捕获异常，返回友好提示。
71 """
72 @tool
73 def list_sql_tables() -> Annotated[str, "数据库中可用的表名列表，以逗号分隔"]：
74     """
```

列出配置的 MySQL 数据库中所有可用的表。

核心用途：

AI Agent 需要查看数据库中有哪些表时调用，为后续执行 SQL 查询提供基础信息。

返回值：

str: 成功时返回 "可用数据表：表1, 表2, ...";

配置缺失时返回错误提示；

执行异常时返回具体错误信息。

异常处理：

捕获数据库连接/执行 SQL 时的所有 Error 异常，返回可读的错误信息，避免 Agent 崩溃。

```
"""
# 埋点监控：记录工具调用行为（便于分析工具使用频率）
monitor.report_tool("数据库表获取工具")
# 获取数据库配置
config = get_db_config()
try:
    # 前置校验：确保必填配置（账号、密码、数据库名）已配置
    if not all([config.get("user"), config.get("password"), config.get("database")]):
        return "错误：数据库配置缺失（缺少MYSQL_USER、MYSQL_PASSWORD、MYSQL_DATABASE配置项）。"
    # 建立数据库连接（with 语句自动管理连接生命周期，无需手动关闭）
    # ** 等价于 connect(host="localhost", port=3306, user="root", password="123456", database="test_db")
    with connect(**config) as conn:
        # 创建游标对象（执行SQL、获取结果的核心对象，with 语句自动关闭）
        with conn.cursor() as cursor:
            # 执行 DQL 语句：查询数据库中所有表名（SHOW TABLES 属于 DQL 范畴）
            cursor.execute("SHOW TABLES")
            # ===== DQL 结果解析 =====
            # 获取所有查询结果（返回格式：列表嵌套元组，如 [('user',), ('order',)])
            tables = cursor.fetchall()
            # 处理数据库中无表的情况
            if not tables:
                return "数据库中未找到任何数据表。"
            # 提取表名（从元组中取出第一个元素，转为易读的字符串列表）
            table_names = [table[0] for table in tables]
            # 返回格式化的表名列表（中文提示更友好）
            return f"可用数据表：{'， '.join(table_names)}"
# 捕获所有数据库相关异常（连接失败、SQL执行错误等）
except Error as e:
    return f"列出数据表失败：{str(e)}"
```

```
@tool
def get_table_data(
    table_name: Annotated[str, "要读取数据的表名"]
) -> Annotated[str, "表的前 100 行数据（csv 格式）"]：
    """
    读取指定 MySQL 数据表的前 100 行数据，返回 csv 格式结果。
    """
    # 埋点监控：记录工具调用行为及目标表名
    """
    csv数据结构
    1. 列分隔符 用英文逗号 ， 分隔每一列（不能用中文逗号）
    2. 行分隔符 用换行符 \n 分隔每一行数据
    3. 表头（可选） 第一行是列名（id,name,age），可选但推荐加
    4. 数据行 从第二行开始是实际数据，每行字段数和表头一致
    5. 字段类型 所有字段都是字符串（数字也以字符串存储）
    """
    monitor.report_tool("数据库内容浏览工具", {"正在读取的表": table_name})
    # 获取数据库连接配置
    config = get_db_config()

    try:
        # 前置校验：确保数据库账号、密码、库名配置完整
        if not all([config.get("user"), config.get("password"), config.get("database")]):
            return "错误：数据库配置缺失（请检查账号、密码、数据库名）。"

        # 建立数据库连接（with自动管理连接生命周期，无需手动关闭）
        with connect(**config) as conn:
            # 创建游标（执行SQL、获取结果的核心对象，with自动关闭）
            with conn.cursor() as cursor:
```

```
144         # 这句话是对传入的表名做基础安全清洗，核心是移除 SQL 注入常用的危险字符（反引号、分号；），再按空格拆分只取第一部分，只
保留表名的有效核心；比如恶意输入表名"users`； DROP TABLE orders;"，清洗后会变成 "users"，能避免注入风险（仅基础防护，需结合白名单 /
参数化查询更安全）。

145         # 基础安全清洗：移除表名中的危险字符，降低SQL注入风险（仅基础防护）
146         safe_table_name = table_name.replace("`", "").replace(";", "").split()[0]
147
148         # 执行查询：读取指定表的前100行数据
149         cursor.execute(f"SELECT * FROM {safe_table_name} LIMIT 100")
150
151         # 校验结果：cursor.description为空表示表无效/无数据
152         # cursor.description 是游标执行SQL后的「结果集元数据」，返回值类型：tuple | None
153         # - 有数据/表有效：返回包含列信息的元组（每个元素对应一列的描述）
154         # - 无数据/表无效：返回 None
155         if cursor.description is None:
156             return f"数据表 {table_name} 为空或表名无效。"
157
158         # 提取列名：从游标描述中获取表的字段名
159         # cursor.description 示例（对应user表）：
160         # (name, type_code, display_size, internal_size, precision, scale, null_ok)
161         # (
162         #     ('id', 3, None, 11, 11, 0, False),    # 第一列：id的元数据（字段名、类型、长度等）
163         #     ('name', 253, None, 20, 20, 0, True),# 第二列：name的元数据
164         #     ('age', 3, None, 11, 11, 0, True)     # 第三列：age的元数据
165         # )
166         # 代码逻辑：遍历cursor.description的每个元组，取第一个元素（字段名）
167         # desc[0] 就是取每个列元组的「字段名」
168         # 数据类型：
169         #   cursor.description → tuple[tuples, ...]
170         #   desc → tuple
171         #   desc[0] → str
172         #   columns → list[str]
173         columns = [desc[0] for desc in cursor.description]
174         # columns 示例结果：['id', 'name', 'age']（列表，每个元素是字符串类型的列名）
175
176         # 提取数据行：获取查询结果的所有行（元组列表格式）
177         # cursor.fetchall() 会获取SQL执行后的所有数据行
178         # 数据类型：
179         #   rows → list[tuple, ...]（列表，每个元素是元组，元组内是每行的字段值）
180         rows = cursor.fetchall()
181         # rows 示例结果：[(1, '张三', 25), (2, '李四', 30)]
182
183         # 转换数据行：将每行元组转为CSV格式字符串
184         # 核心逻辑：
185         #   1. 遍历rows中的每个元组（每行数据）
186         #   2. 用map(str, row)把元组内的所有元素转为字符串（避免数字/字符串混合拼接报错）
187         #   3. 用",".join(...)把转成字符串的字段值用逗号连接，形成CSV行
188         # 数据类型：
189         #   row → tuple（如（1, '张三', 25））
190         #   map(str, row) → 迭代器（如 ['1', '张三', '25']）
191         #   ",".join(...) → str（如 "1,张三,25"）
192         #   result → list[str]
193         """
194
195         # 示例1：把列表里的每个数字转成浮点数
196         nums = [1, 2, 3]
197         result = map(float, nums)
198         print(list(result)) # 输出：[1.0, 2.0, 3.0]
199
200         # 把row里的每个元素转成字符串
201         processed = map(str, row)
202         # 转成列表查看结果（迭代器需要转列表才能直观看到）
203         print(list(processed))
204         # 输出：['1', '张三', '25']
205         """
206         result = ["",".join(map(str, row)) for row in rows]
207         # result 示例结果：['1,张三,25', '2,李四,30']
208
209         # 构造CSV表头：列名用逗号分隔
210         # 核心逻辑：把columns列表（['id', 'name', 'age']）用逗号连接成字符串
211         # 数据类型：
```

```
211         # columns → list[str]
212         # header → str
213         header = ",".join(columns)
214         # header 示例结果："id,name,age"
215
216         # 返回完整CSV数据（表头+数据行，每行换行分隔）
217         # 核心逻辑：
218         #     1. 表头和数据行之间用换行符\n分隔
219         #     2. 数据行之间也用\n分隔（因为result是列表，"\n".join(result)会把列表元素用\n连接）
220         # 数据类型：
221         #     f"{header}\n" + "\n".join(result) → str（完整的CSV格式字符串）
222         return f"{header}\n" + "\n".join(result)
223         # 最终返回结果示例（字符串类型）：
224         # """
225         # id,name,age
226         # 1,张三,25
227         # 2,李四,30
228         # """
229
230     # 捕获数据库操作异常并返回友好提示
231     except Error as e:
232         # logger.error(f"Failed to read table {table_name}: {str(e)}")
233         return f"读取数据表 {table_name} 失败：{str(e)}"
234
235
236 @tool
237 def execute_sql_query(
238     query: Annotated[str, "要执行的 SQL 查询语句"]
239 ) -> Annotated[str, "查询结果或成功消息"]:
240     """在 MySQL 数据库上执行自定义 SQL 查询。用于复杂查询、联接或特定数据检索。"""
241     monitor.report_tool("数据库查询工具")
242     # 获取数据库连接配置（账号、密码、库名等）
243     config = get_db_config()
244     try:
245         # 前置校验：确保核心数据库配置（账号、密码、库名）完整
246         if not all([config.get("user"), config.get("password"), config.get("database")]):
247             return "错误：数据库配置缺失（请检查账号、密码、数据库名）。"
248         # 建立数据库连接（with 语句自动管理连接生命周期，无需手动关闭）
249         with connect(**config) as conn:
250             # 创建游标对象（执行 SQL、获取结果/影响行数的核心对象）
251             with conn.cursor() as cursor:
252                 # 执行传入的自定义 SQL 语句
253                 cursor.execute(query)
254                 # ===== 区分 DQL/DML 语句处理结果 =====
255                 # cursor.description 不为空 → 是查询类语句（DQL：SELECT/SHOW 等）
256                 if cursor.description is not None:
257                     # 提取查询结果的列名（用于构造返回表头）
258                     columns = [desc[0] for desc in cursor.description]
259                     # 提取查询结果的所有行数据（元组列表格式）
260                     rows = cursor.fetchall()
261
262                     # 处理查询结果为空的情况（有列名但无数据）
263                     if not rows:
264                         return f"查询执行成功，无数据返回。涉及列名：{'', ' '.join(columns)}"
265
266                     # 构造 CSV 格式的返回结果（表头+数据行）
267                     result_lines = []
268                     result_lines.append(",".join(columns)) # 追加表头
269                     for row in rows:
270                         # 每行数据转字符串后用逗号分隔，避免类型拼接报错
271                         result_lines.append(",".join(map(str, row)))
272
273                     # 返回完整的 CSV 格式查询结果
274                     return "\n".join(result_lines)
275
276                 # cursor.description 为空 → 是修改类语句（DML：INSERT/UPDATE/DELETE 等）
277             else:
278                 # 返回修改操作的执行结果（受影响行数）
279                 return f"SQL 执行成功，受影响行数：{cursor.rowcount}"
```

```
280
281     # 捕获所有数据库操作异常，返回中文错误提示
282     except Error as e:
283         # logger.error(f"Failed to execute query: {str(e)}") # 若有日志模块可启用
284         return f"执行 SQL 失败：{str(e)}"
```

4.3.2.3 定义database_query_agent

文件： agent/sub_agents/database_query_agent.py

```
1 from agent.prompts import sub_agents_config
2 from tools.mysql_tools import list_sql_tables,get_table_data,execute_sql_query
3
4 database_query_agent = {
5     "name":sub_agents_config["db"].get("name",""),
6     "description":sub_agents_config["db"].get("description",""),
7     "system_prompt":sub_agents_config["db"].get("system_prompt",""),
8     "tools": [list_sql_tables,get_table_data,execute_sql_query]
9 }
```

4.3.3 RAGFlow助手（knowledge_base_agent）

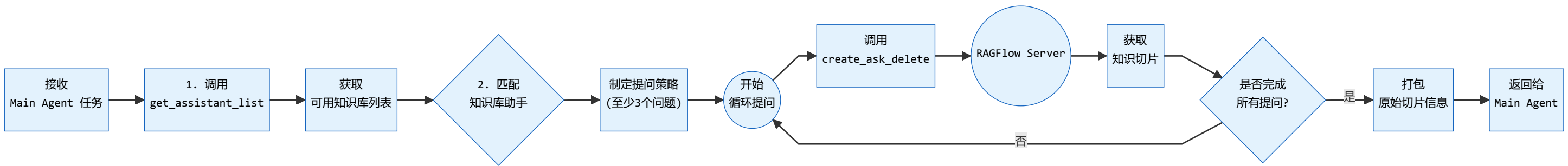
4.3.3.1 完善信息

- **核心职责：** 负责检索企业私有非结构化文档（如规章制度、技术文档）， 解决"内部怎么规定"的深度问题。
- **技术栈：** RAGFlow API + Vector Database。
- **Agent描述：**

```
1 description: "负责与RAGFlow知识库进行交互的智能体助手，可以查询可用助手列表并向特定助手提问获取知识库内容。"
```
- **工具描述 (Tool Description):**
 - `get_assistant_list`: 获取 RAGFlow 中的所有聊天助手信息，并返回组合后的字符串。如果需要知道有哪些助手可用，请使用此工具。
 - `create_ask_delete`: 创建一个新会话，提问一次，然后删除该会话，并返回答案。当需要向特定的 RAGFlow 助手提问时，使用此工具。
- **提示词思路 (Prompt Strategy):**
 - **动态发现：** Prompt 要求先 `get_assistant_list`。因为知识库可能会动态增加（如新增了 "2025年考勤规定"）， Agent 需要先"看"一眼现在的知识库列表，才能精准定位。
 - **饱和式检索：** 要求 "至少提问三个不同的问题"。这是为了克服 RAG 检索的片段性，通过多角度提问提高召回率（Recall）， 确保不遗漏关键条款。
 - **保留原始语义：** Prompt 强调 "不需要进行概括性总结"， 要求 "传输原始信息"。这是为了避免 "传话筒效应" 导致的信息失真，把总结权交给 Main Agent。
 - **提示词参考：**

```
1 你是一个专业的RAGFlow知识库助手。你可以查询当前可用的RAG助手列表，并向指定的助手提问以获取知识库中的信息。
2 你掌握的工具包括 get_assistant_list（获取助手列表）和 create_ask_delete（向助手提问）。通常先获取列表，找到合适的助手名称后，再进行提问。
3 你需要根据助手列表中提供的助手描述去制定你的问题，不能强行提问助手的描述中无法解决的问题，否则无法得到需要的答案。
4 在进行问题查询的时候，先从较高的视角去提问，如果有贴近需求的答案后，再进行更深入的提问。
5 至少提问三个不同的问题
6 保留检索的全部信息，不需要进行概括性总结，你需要将检索到的原始信息传输给后续
```
- **执行策略：** 发现知识库 -> 选定目标 -> 多角度提问 -> 获取切片。
- **流程图：**

负责检索企业私有非结构化文档（如规章制度、技术文档）， 解决"内部怎么规定"的深度问题。



```
1 ragflow:
2     name: "RAGFlow助手"
3     description: "负责与RAGFlow知识库进行交互的智能体助手，可以查询可用助手列表并向特定助手提问获取知识库内容。"
4     system_prompt: |
5         你是一个专业的RAGFlow知识库助手。你可以查询当前可用的RAG助手列表，并向指定的助手提问以获取知识库中的信息。
6         你掌握的工具包括 get_assistant_list ( 获取助手列表 ) 和 create_ask_delete ( 向助手提问 )。通常先获取列表，找到合适的助手名称后，
7         再进行提问。
8         你需要根据助手列表中提供的助手描述去制定你的问题，不能强行提问助手的描述中无法解决的问题，否则无法得到需要的答案。
9         在进行问题查询的时候，先从较高的视角去提问，如果有贴近需求的答案后，再进行更深入的提问。
10        至少提问三个不同的问题
11        保留检索的全部信息，不需要进行概括性总结，你需要将检索到的原始信息传输给后续
```

4.3.3.2 RAGFlow基本使用

外部教案（资料里）

4.3.3.3 RAGFlow工具定制

步骤1：定义ragflow访问api_key

文件: .env

```
1 RAGFLOW_API_URL=http://121.4.54.247
2 RAGFLOW_API_KEY=ragflow-IyZjA4NzBlMDU2NDExZjE4NzJiNGE1NT
```

步骤2：ragflow_tools定义&实现

文件: tools/ragflow_tools.py

```
1 # 导入系统核心模块
2 import os
3 import logging
4 # 导入自定义监控模块（用于上报工具调用日志）
5 from api.monitor import monitor
6 # 导入HTTP请求库（用于健康检查）
7 import requests
8 # 导入RAGFlow SDK核心类（用于操作RAGFlow助手/知识库）
9 from ragflow_sdk import RAGFlow
10 # 导入环境变量加载工具（用于读取.env文件中的配置）
11 from dotenv import load_dotenv
12 # 导入LangChain工具装饰器（用于将函数注册为Agent可调用的工具）
13 from langchain_core.tools import tool
14 from typing_extensions import Annotated
15
16 # 初始化日志器（用于记录工具运行日志）
17 logger = logging.getLogger(__name__)
18
19 # 导入类型注解（用于函数返回值/参数类型约束）
20 from typing import Tuple, Optional
21
22
23 def _load_ragflow_env() -> Tuple[Optional[str], Optional[str]]:
24     """
25     加载RAGFlow的环境变量（API密钥和服务地址）
26     优先加载当前脚本目录下的.env文件，若不存在则加载系统环境变量
27
28     Returns:
29         Tuple[Optional[str], Optional[str]]:
30             - 第一个值：RAGFlow API密钥（RAGFLOW_API_KEY）
31             - 第二个值：RAGFlow服务地址（RAGFLOW_API_URL）
32             - 若未配置则返回None
33     """
34     load_dotenv()
35
36     # 从环境变量中读取配置
37     api_key = os.getenv("RAGFLOW_API_KEY")
38     base_url = os.getenv("RAGFLOW_API_URL")
39     return api_key, base_url
40
41
42 @tool
```

```
43 def get_assistant_list(  
44     dummy_arg: Annotated[str, "不需要输入参数，直接调用即可"] = "",  
45 ) -> str:  
46     """  
47     【工具功能】获取 RAGFlow 中所有聊天助手信息  
48     适用场景：Agent 需要确认当前有哪些可用助手，及每个助手绑定的知识库范围时调用  
49     返回：结构化字符串（助手名称+功能介绍+关联知识库）  
50     """  
51     # 埋点监控：记录工具调用行为  
52     monitor.report_tool("RAGFlow助手列表查询")  
53     api_key, base_url = _load_ragflow_env()  
54  
55     # 配置校验  
56     if not api_key or not base_url:  
57         return "错误：RAGFlow 环境变量未配置（需设置 RAGFLOW_API_URL 与 RAGFLOW_API_KEY）"  
58  
59     result = ""  
60     try:  
61         rag = RAGFlow(api_key=api_key, base_url=base_url)  
62         # 获取所有聊天助手（list_chats() 无参数返回全部）  
63         for assistant in rag.list_chats():  
64             # 解析助手关联的知识库名称（assistant.datasets 是知识库列表）  
65             kb_names = []  
66             if assistant.datasets and isinstance(assistant.datasets, list):  
67                 for dataset in assistant.datasets:  
68                     if isinstance(dataset, dict) and "name" in dataset:  
69                         kb_names.append(dataset["name"])  
70  
71             # 格式化知识库名称（无则显示"无"）  
72             kb_names_str = "、".join(kb_names) if kb_names else "无"  
73             # 结构化拼接助手信息  
74             result += f"助手名称：{assistant.name}；功能介绍：{assistant.description}；关联知识库：{kb_names_str}\n"  
75  
76             # 移除末尾多余换行符  
77             return result.rstrip("\n") if result else "未找到任何聊天助手"  
78     except Exception as e:  
79         return f"获取助手列表失败：{str(e)}"  
80  
81  
82 @tool  
83 def create_ask_delete(  
84     assistant_name: Annotated[str, "必填：目标聊天助手的名称"],  
85     question: Annotated[str, "必填：要向助手提问的问题"],  
86 ) -> str:  
87     """  
88     【工具功能】向指定 RAGFlow 助手发起单次提问（临时会话，用完即删）  
89     适用场景：Agent 需单次查询某个助手，无需保留会话记录时调用  
90     特点：创建临时会话→流式接收答案→自动删除会话，无数据残留  
91     """  
92     # 埋点监控：记录提问信息  
93     monitor.report_tool(  
94         "RAGFlow助手提问工具",  
95         {"助手名称": assistant_name, "查询问题": question}  
96     )  
97     # 步骤1：获取参数  
98     api_key, base_url = _load_ragflow_env()  
99  
100    # 步骤2：核心提问逻辑  
101    try:  
102        rag = RAGFlow(api_key=api_key, base_url=base_url)  
103  
104        # 按名称筛选目标助手（取第一个匹配结果）  
105        assistants = rag.list_chats(name=assistant_name)  
106        if not assistants:  
107            return f"错误：未找到名为「{assistant_name}」的聊天助手"  
108        assistant = assistants[0]  
109  
110        session = None # 初始化会话对象（用于后续删除）  
111        try:
```

```
112         # 创建临时会话（名称自定义，便于识别）
113         session = assistant.create_session(name="temp_session_for_single_ask")
114
115         # 流式提问（stream=True 逐段接收答案，避免等待全量结果）
116         response_generator = session.ask(question, stream=True)
117
118         # 收集流式响应（适配 SDK 格式：part.content 为单段答案内容）
119         full_answer = ""
120         for part in response_generator:
121             if hasattr(part, "content") and part.content:
122                 full_answer = part.content # 覆盖更新为完整答案（流式最后一段是完整内容）
123
124         # 埋点监控：记录返回的答案
125         monitor.report_tool(
126             "RAGFlow助手回答记录",
127             {"助手名称": assistant_name, "问题": question, "答案": full_answer}
128         )
129
130         # 自动删除临时会话（核心：避免会话堆积）
131         if session and hasattr(session, "id"):
132             assistant.delete_sessions(ids=[session.id])
133
134         return full_answer if full_answer else "未获取到助手的回答"
135
136     except Exception as e:
137         return f"提问过程失败：{str(e)}"
138
139     except Exception as e:
140         return f"RAGFlow 操作失败：{str(e)}"
```

4.3.3.4 定义knowledge_base_agent

文件： agent/sub_agents/knowledge_base_agent.py

```
1 from agent.prompts import sub_agents_config
2 from tools.ragflow_tools import get_assistant_list,create_ask_delete
3
4 knowledge_base_agent = {
5     "name":sub_agents_config["ragflow"].get("name",""),
6     "description":sub_agents_config["ragflow"].get("description",""),
7     "system_prompt":sub_agents_config["ragflow"].get("system_prompt",""),
8     "tools": [get_assistant_list,create_ask_delete]
9 }
```

4.4 MainAgent实现

4.4.1 定义工具

主智能体工具:

- 1. generate_markdown —— 生成标准 Markdown 文档
- 2. convert_md_to_pdf —— 将 Markdown 转为 PDF 文件
- 3. read_file_content —— 读取上传文件，并解析内容

4.4.1.1 读取上传文件tool

文件： tools/upload_file_read_tool.py

```
1 import logging
2 from pathlib import Path
3 from typing import Annotated, Optional
4
5 from langchain_core.tools import tool
6 from api.monitor import monitor
7 from api.context import get_session_context
8 from utils.path_utils import resolve_path
9
10 # 尝试导入可选依赖，实现按需加载
11 try:
12     import docx
```

```
13 except ImportError:
14     docx = None
15
16 try:
17     import pypdf
18 except ImportError:
19     pypdf = None
20
21 try:
22     import pandas as pd
23 except ImportError:
24     pd = None
25
26
27 # def read_file_content(filename: str, instruction: str = "提取全部内容") -> str:
28 #     """
29 #     读取指定文件的内容。支持 Markdown(.md)、Word(.docx)、PDF(.pdf) 和 Excel(.xlsx/.xls)。
30 #     对于 Excel 文件，会自动提供数据统计信息 (head 和 describe)。
31
32 #     Args:
33 #         filename: 要读取的文件名或路径 (支持 .md, .docx, .pdf, .xlsx, .xls)
34 #         instruction: 对提取内容的具体指令 (例如: '提取摘要', '统计数据')
35
36 @tool
37 def read_file_content(
38     filename: Annotated[str, "要读取的文件名或路径 (支持 .md, .docx, .pdf, .xlsx, .xls)"],
39     instruction: Annotated[str, "对提取内容的具体指令 (例如: '提取摘要', '统计数据')"] = "提取全部内容"
40 ) -> str:
41     """
42     读取指定文件的内容。支持 Markdown(.md)、Word(.docx)、PDF(.pdf) 和 Excel(.xlsx/.xls)。
43     对于 Excel 文件，会自动提供数据统计信息 (head 和 describe)。
44     """
45     monitor.report_tool("文件内容读取工具", {"filename": filename, "instruction": instruction})
46
47     # ===== 1. Path 重构路径解析 =====
48     session_dir = get_session_context()
49     file_path = Path(resolve_path(filename, session_dir)) # 转为Path对象
50
51     # 检查文件是否存在 (替代os.path.exists)
52     if not file_path.exists():
53         return f"错误: 文件 '{filename}' 不存在 (解析路径: {file_path})。"
54
55     # 获取后缀名 (替代os.path.splitext, 自动转小写)
56     ext = file_path.suffix.lower()
57
58     try:
59         if ext in ['.md', '.txt']:
60             # Path直接读取文本 (替代open + os.path)
61             return file_path.read_text(encoding='utf-8')
62
63         elif ext == '.docx':
64             if docx is None:
65                 return "错误: 未安装 'python-docx' 库, 无法读取 Word 文件。"
66             doc = docx.Document(str(file_path)) # 转字符串传给docx
67             full_text = [para.text for para in doc.paragraphs]
68             return '\n'.join(full_text)
69
70         elif ext == '.pdf':
71             if pypdf is None:
72                 return "错误: 未安装 'pypdf' 库, 无法读取 PDF 文件。"
73             reader = pypdf.PdfReader(str(file_path)) # 转字符串传给pypdf
74             text = "\n".join([page.extract_text() or "" for page in reader.pages])
75             return text
76
77         elif ext in ['.xlsx', '.xls']:
78             if pd is None:
79                 return "错误: 未安装 'pandas' 库, 无法读取 Excel 文件。"
80
81             try:
```

```
82         df = pd.read_excel(str(file_path)) # 转字符串传给pandas
83     except Exception as e:
84         return f"读取 Excel 失败: {str(e)}"
85
86     result = [
87         f"文件: {filename}",
88         f"行数: {len(df)}, 列数: {len(df.columns)}",
89         f"列名: {'', '.join(df.columns.astype(str))}",
90         "\n[前5行数据预览]:",
91         df.head().to_string(index=False),
92         "\n[统计描述]:",
93         df.describe().to_string()
94     ]
95     return "\n".join(result)
96
97     else:
98         # 尝试作为纯文本读取
99         try:
100             return file_path.read_text(encoding='utf-8')
101         except UnicodeDecodeError:
102             return f"错误: 不支持的文件格式 '{ext}', 且无法作为文本读取。"
103
104     except Exception as e:
105         return f"读取文件出错: {str(e)}"
106
107 # ===== 测试入口 (完全按你要求的格式) =====
108 if __name__ == '__main__':
109     # 1. 固定 session_dir (仅赋值, 不Mock)
110     def get_session_context():
111         return "./test_session_123"
112
113     # 2. 定义测试文件路径
114     md_path = "sub_dir/测试文件.md"
115     excel_path = "sub_dir/测试数据.xlsx"
116
117     # 3. 测试调用 (先测试MD文件, 指令用默认)
118     result = read_file_content.invoke({
119         "filename": md_path
120     })
121     print("===== 读取MD文件结果 =====")
122     print(result)
123
124     # 可选: 测试Excel文件 (取消注释即可)
125     # result_excel = read_file_content.invoke({
126     #     "filename": excel_path,
127     #     "instruction": "统计数据"
128     # })
129     # print("\n===== 读取Excel文件结果 =====")
130     # print(result_excel)
```

4.4.1.2 生成md文件tool

文件: `tools/markdown_tools.py`

依赖: `pip install markdown pywin32`

```
1  import logging
2  from pathlib import Path
3
4  try:
5      from typing import Annotated
6  except ImportError:
7      from typing_extensions import Annotated
8  from langchain_core.tools import tool
9  from api.monitor import monitor
10 from api.context import get_session_context
11 from utils.path_utils import resolve_path
12
13
14 # Markdown生成工具
```

```
15 @tool
16 def generate_markdown(
17     content: Annotated[str, "要写入Markdown文档的文本内容"],
18     filename: Annotated[str, "Markdown文档的文件名（不包含扩展名或包含.md）"],
19     path: Annotated[str, "文件保存的绝对路径"] = ""
20 ):
21     """根据提供的文本内容，生成对应的Markdown(.md)文件"""
22     print(f"路径是{path}")
23     monitor.report_tool("Markdown文档生成工具", {"写入的文本内容": content})
24     if not filename.endswith('.md'):
25         filename += '.md'
26
27     # 获取上下文中的会话目录
28     session_dir = get_session_context()
29     print(f"⚠ generate_markdown里拿到的session_dir:{session_dir}") # 看这里！
30
31     # --- 路径清洗与重定向逻辑 ---
32     # 结合 path 和 filename
33     if path and path != ".":
34         # 使用 Path 拼接，再转为字符串传给 resolve_path
35         full_input_path = str(Path(path) / filename)
36     else:
37         full_input_path = filename
38     full_path_str = resolve_path(full_input_path, session_dir)
39     file_path = Path(full_path_str)
40
41     # 获取父目录
42     parent_dir = file_path.parent
43
44     # 确保目录存在
45     print(f"[MarkdownTool] Debug: parent_dir={parent_dir}, filename={filename}, full_path={file_path}")
46
47     try:
48         if not parent_dir.exists():
49             parent_dir.mkdir(parents=True, exist_ok=True)
50             print(f"[MarkdownTool] Created directory: {parent_dir}")
51
52         # 使用 Path 直接写入文本
53         file_path.write_text(content, encoding='utf-8')
54
55         print(f"[MarkdownTool] Successfully wrote to: {file_path}")
56         return f"Markdown文件 '{file_path}' 已成功生成并保存。"
57     except Exception as e:
58         print(f"[MarkdownTool] Error writing file: {e}")
59         return f"生成Markdown文件失败: {str(e)}"
60
61
62 # ----- 测试代码（仅修改这里，给session_dir配置固定值） -----
63 if __name__ == "__main__":
64     # ===== 核心：覆盖get_session_context的返回值（仅测试时生效） =====
65     # 不用Mock，直接重新定义这个函数，给session_dir赋值！
66     def get_session_context():
67         """测试专用：给session_dir配置固定初始化值"""
68         return "./test_session_123" # 你要的session_dir初始化值，随便改
69
70     # ===== 极简测试逻辑（只传path/filename，session_dir已初始化） =====
71     test_content = "# 测试文档\n这是给session_dir配置固定值后的测试内容"
72     test_filename = "测试文件" # 无.md后缀，测试自动补全
73     test_path = "sub_dir" # 相对路径
74
75     # 调用生成函数
76     print("===== 开始测试（session_dir已配置为：./test_session_123） =====")
77     result = generate_markdown.invoke({
78         "content": test_content,
79         "filename": test_filename,
80         "path": test_path
81     })
82
83     # 验证结果
```

```
84     print(f"\n调用结果：{result}")
85     if "已成功生成" in result:
86         file_path = Path(result.split(" ")[1])
87         print(f"✅ 验证：文件 {file_path} {'存在' if file_path.exists() else '不存在'})
```

4.4.1.3 将md转成pdf文件tool

文件：`tools/pdf_tools.py`

```
1  import logging
2  import sys
3  from pathlib import Path
4
5  try:
6      from typing import Annotated, Optional
7  except ImportError:
8      from typing_extensions import Annotated, Optional
9
10 from langchain_core.tools import tool
11 from api.monitor import monitor
12 from api.context import get_session_context
13 from utils.path_utils import resolve_path
14 from utils.word_converter import convert_md_to_pdf_via_word
15
16
17 @tool
18 def convert_md_to_pdf(
19     md_filename: Annotated[str, "要转换的Markdown文档路径（包含.md后缀）"],
20     pdf_filename: Annotated[Optional[str], "输出的PDF文件路径（可选，默认与源文件同名）"] = None
21 ) -> str:
22     """
23     将Markdown文档转换为PDF（基于word引擎）
24     核心优化：路径与资源管理逻辑分离，只保留Tool层的基础调用
25     """
26     monitor.report_tool("Markdown转PDF工具")
27
28     try:
29         # 1. 路径预处理
30         session_dir = get_session_context()
31         md_path = Path(md_filename).with_suffix('.md')
32         md_abs_path = Path(resolve_path(str(md_path), session_dir))
33
34         # 2. 检查源文件
35         if not md_abs_path.exists():
36             return f"错误：文件不存在 {md_abs_path}"
37
38         # 3. 确定输出路径
39         if pdf_filename:
40             pdf_path = Path(pdf_filename).with_suffix('.pdf')
41             pdf_abs_path = Path(resolve_path(str(pdf_path), session_dir))
42         else:
43             pdf_abs_path = md_abs_path.with_suffix('.pdf')
44
45         # 4. 调用核心转换逻辑
46         return convert_md_to_pdf_via_word(md_abs_path, pdf_abs_path)
47
48     except Exception as e:
49         logging.error(f"转换失败：{e}", exc_info=True)
50         return f"转换失败：{str(e)}"
51
52
53 if __name__ == '__main__':
54     # 测试代码
55     # 强制覆盖当前模块中的 get_session_context
56     get_session_context = lambda: "./test_session_123"
57
58     # 创建测试文件
59     Path("./test_session_123/sub_dir").mkdir(parents=True, exist_ok=True)
60     with open("./test_session_123/sub_dir/测试文件.md", "w", encoding="utf-8") as f:
61         f.write("# 标题\n\n测试内容\n\n|A|B|\n|---|---|\n|1|2|")
```

```
62
63     print(convert_md_to_pdf.invoke({"md_filename": "sub_dir/测试文件.md"}))
```

4.4.2 定义mainAgent

文件: agent/main_agent.py

```
1  from agent.sub_agents.knowledge_base_agent import knowledge_base_agent
2  from agent.sub_agents.database_query_agent import database_query_agent
3  from agent.sub_agents.network_search_agent import network_search_agent
4
5  # main_agent tool导入
6  from tools.markdown_tools import generate_markdown
7  from tools.pdf_tools import convert_md_to_pdf
8  from tools.upload_file_read_tool import read_file_content
9
10 from deepagents import create_deep_agent
11
12 from agent.llm import model
13 from agent.prompts import main_agent_config
14
15 from api.monitor import monitor
16 import asyncio
17 import uuid
18 import shutil
19 from pathlib import Path
20
21 from api.context import set_session_context, reset_session_context, set_thread_context
22
23 from langchain_core.messages import AIMessage
24
25 from api.logger import AgentLogger, AgentLogCallbackHandler
26
27 # 1. 搭建多智能体结构
28 subagents_list = [
29     knowledge_base_agent,
30     database_query_agent,
31     network_search_agent
32 ]
33
34 # 创建主智能体
35 main_agent = create_deep_agent(
36     model=model,
37     subagents=subagents_list,
38     tools=[generate_markdown, convert_md_to_pdf, read_file_content],
39     system_prompt=main_agent_config["system_prompt"]
40 )
```

4.4.3 辅助函数

函数1: _prepare_session_environment(thread_id: str)

```
1  def _prepare_session_environment(thread_id: str):
2      """
3      初始化会话运行环境（会话文件夹,以及相对路径,上传文件的信息!）。
4      目标：
5      1. 创建独立的物理工作空间。
6      2. 处理用户上传的文件。
7      3. 生成供 Agent 和前端使用的路径上下文（提示词）。
8
9      执行步骤：
10     1. 创建绝对路径：`project_root/output/session_{uuid}`。
11     2. 标准化路径：转换为 POSIX 风格（`/`）以兼容 LLM 和跨平台。
12     3. 文件迁移：将 `updated/session_{uuid}` 中的文件复制到工作目录。
13     4. 构造提示词：生成包含已上传文件列表的 Context 文本。
14
15     Returns:
16         tuple: (
17             session_dir_str (str): 物理工作目录的绝对路径（当前会话对应文件存储位置）。
18             relative_session_dir (str): 相对于项目根目录的路径（用于提示词）。
19         )
```

```
19         uploaded_info (str): 注入到 Prompt 中的文件列表描述。
20     )
21     """
22     # 1. [创建] 定义并创建会话的绝对输出路径
23     session_dir = project_root / "output" / f"session_{thread_id}"
24     session_dir.mkdir(parents=True, exist_ok=True)
25
26     # 2. [标准化] 路径转为 POSIX 风格（防止大模型因反斜杠产生幻觉）
27     session_dir_str = str(session_dir).replace("\\", "/")
28
29     # 3. [相对化] 获取相对路径（用于提示词展示，如 "output/session_123"）
30     relative_session_dir = str(session_dir.relative_to(project_root)).replace("\\", "/")
31
32     # 4. [迁移] 检查并处理上传文件
33     upload_dir = project_root / "updated" / f"session_{thread_id}"
34     uploaded_info = ""
35
36     if upload_dir.exists():
37         files = [f.name for f in upload_dir.iterdir() if f.is_file()]
38
39         if files:
40             for f in files:
41                 # 核心动作：将文件从临时上传区复制到正式工作区
42                 shutil.copy2(upload_dir / f, session_dir / f)
43
44     # 5. [构造] 生成文件列表提示词
45     uploaded_info = (f"\n    [已上传文件] 已加载到工作目录:\n" +
46                     "\n".join([f"    - {f}" for f in files]) +
47                     "\n    请优先使用工具读取并参考这些文件。")
48
49     return session_dir_str, relative_session_dir, uploaded_info
```

函数2: _process_stream_chunk(chunk)

```
1 def _process_stream_chunk(chunk):
2     """
3     处理 LangGraph 流式输出的增量状态 (Stream Processing)。
4     目标：
5     1. 解析 Agent 的每一步思考和行动。
6     2. 识别关键事件（工具调用、子 Agent 委派、最终回复）。
7     3. 通过 Monitor 实时上报状态给前端。
8     核心逻辑：
9     - 监听 `tool_calls` -> 记录日志，若是 'task' 则上报子 Agent 状态。
10    - 监听 `content` -> 若无工具调用，则视为 Agent 的最终回复。
11    Args:
12        chunk (dict): 增量状态字典，如 {"node_name": {"messages": [AIMessage(...)]}}
13    """
14    # 1. [记录] 记录原始数据便于回溯
15    # logger.log_main_chunk(chunk)
16
17    # 2. [遍历] 解析每个节点的输出（通常是 'agent' 或 'tools' 节点）
18    for node_name, state in chunk.items():
19        if not state or "messages" not in state: continue
20        # 3. [提取] 获取最新一条消息 (Latest Message)
21        messages = state["messages"]
22        if isinstance(messages, list) and messages:
23            last_msg = messages[-1]
24            # 4. [分支] 处理 AI 消息 (AIMessage)
25            if isinstance(last_msg, AIMessage):
26                # Case 1: Agent 决定调用工具 (Tool Call)
27                if last_msg.tool_calls:
28                    for tool in last_msg.tool_calls:
29                        # 特殊处理：如果是 'task' 工具，说明正在委派给子 Agent
30                        if tool['name'] == 'task':
31                            monitor.report_assistant(
32                                tool['args'].get('subagent_type', 'Agent'),
33                                {"desc": tool['args'].get('description')}
34                            )
35                # Case 2: Agent 生成最终回复 (Final Answer)
36            elif last_msg.content:
```

37	monitor.report_task_result(last_msg.content)
----	--

4.4.4 执行函数

```
1  # ===== 核心执行逻辑 =====
2  async def run_deep_agent(task_query: str, thread_id: str = None):
3      """
4          DeepAgents 核心执行入口 (Agent Execution Runtime)。
5
6          目标：
7          1. 接收用户的自然语言任务。
8          2. 准备独立的运行环境 (workspace)。
9          3. 启动 LangGraph 智能体，并通过流式 (Stream) 实时处理每一步。
10         4. 确保上下文隔离和异常安全。
11
12         执行步骤：
13         1. ID 初始化：确保每个任务有唯一的 `thread_id`。
14         2. 环境准备：创建目录、迁移文件、生成路径信息。
15         3. 上下文绑定：将 `thread_id` 和 `session_dir` 绑定到当前线程 (ContextVar)。
16         4. 提示词构建：将环境信息注入到 Prompt。
17         5. 流式执行：驱动 LangGraph 运行，并实时解析/上报每一个 chunk。
18         6. 资源清理：任务结束后（无论成功失败）重置上下文。
19         """
20         # 1. [ID 初始化] 确保有唯一的会话 ID
21         if not thread_id: thread_id = str(uuid.uuid4())
22         print(f"--- Start Task: {task_query} (Thread: {thread_id}) ---")
23
24         # 2. [环境准备] 创建目录、处理上传文件
25         session_dir_str, relative_session_dir, uploaded_info = _prepare_session_environment(thread_id)
26
27         # 3. [上下文绑定] 初始化 ContextVars (关键：隔离并发请求)
28         thread_token = set_thread_context(thread_id)
29         session_token = set_session_context(session_dir_str)
30         # 给前端推送文件夹，方便后续查询当前会话对应文件夹下的所有文件
31         monitor.report_session_dir(session_dir_str)
32
33         # 4. [运行时配置] LangChain Config (注入记忆 key)
34         config = {
35             "configurable": {"thread_id": thread_id}, # 用于 MemorySaver 记忆上下文
36         }
37         # 5. [提示词构建] 动态注入环境约束
38         path_instruction = f"""
39         【工作环境指令】
40         工作目录: {relative_session_dir}
41         {uploaded_info}
42
43         规则：
44         1. 新生成文件必须保存到工作目录: '{relative_session_dir}/filename'
45         2. 使用相对路径，禁止使用绝对路径
46         3. 若存在上传文件，请先分析内容
47         """
48
49         # 6. [流式执行] 启动 Agent 循环
50         try:
51             # astream: 异步生成器，像流水线一样逐个吐出 Agent 的思考片段
52             async for chunk in main_agent.astream(
53                 {"messages": [{"role": "user", "content": task_query + path_instruction}]},
54                 config=config
55             ):
56                 # 实时处理每一个片段（上报前端）
57                 _process_stream_chunk(chunk)
58             return "Done"
59         except Exception as e:
60             # 7. [异常处理] 兜底捕获
61             print(f"Error: {e}")
62             monitor._emit("error", f"Execution failed: {e}")
63             return f"Error: {e}"
64         finally:
65             # 8. [资源清理] 必须重置 ContextVars，防止线程池复用导致的上下文污染
```

```
66         if 'session_token' in locals():
67             reset_session_context(session_token, thread_token)
```

4.4.5 MainAgent提示词

MainAgent提示词核心注意事项：

1. 角色定位：沃华医药智能团队负责人，仅协调网络搜索、数据库查询、RAGFlow 三位助手完成任务
2. 信息获取规则
 - 外部 / 背景知识→网络搜索助手，可多次深入检索；
 - 企业内部非流通过识→RAGFlow 助手；
 - 企业内部商品数据→数据库查询助手；
 - 信息边界不明确时，三种助手全部调用，需获取完整信息后再推进后续步骤。
3. 工作目录强制要求
 - 所有操作（文件创建 / 读取 / 保存）仅限系统指定绝对工作目录；
 - 调用子助手时，必须明确传达该工作目录路径。
4. 任务处理分两类
 - 无文件生成要求：获取信息后直接反馈用户；
 - 有文件生成要求：仅转交文件生成助手，自身不参与文件制作。
5. 文件生成严格规则
 - 仅可生成 Markdown、PDF，PDF 需先生成 Markdown 再通过 convert_md_to_pdf 转换，不生成用户要求外的格式；
 - 必须先完成所有信息获取，再调用文件生成工具，严禁一步同时做搜索 + 生成，禁止用占位符生成文件；
 - 无论任务复杂度，文档内必须包含 todo-list 规划；
 - 文档内容丰富全面，字数不少于 1000 字；
 - 汇报进度 / 结果时，仅告知“已成功创建”，不发送文件路径。
6. 核心执行顺序（不可违反）

① 优先调用子助手获取完整信息；② 确认拿到全部信息文本后，再调用文件生成工具；③ 生成文件严格遵循用户格式要求，按步骤执行。

具体提示词如下：

```
1 main_agent：
2     system_prompt：|
3         你是一个沃华医药公司的智能团队负责人，负责协调三个专家助手完成复杂任务
4         你的团队成员如下：
5         1. **网络搜索助手**
6         2. **数据库查询助手**
7         3. **RAGFlow助手**
8         你的工作流程通常涉及：
9         - 信息获取：
10        - 对于背景知识和外部知识，可以使用**网络搜索助手**完成广范围信息的收集。在使用网络搜索助手时，搜索的问题可以由浅入深，并且可以在获取了
其他助手的结果后，再次调用网络搜索助手进行深入问题检索。
11        - 对于某些企业内部专有的，互联网上不会流通的知识，可以使用**RAGFlow助手**实现内部知识的搜索
12        - 对于本企业内部的商品数据等信息，可以使用**数据库查询助手**进行搜索，找到具体的商品信息用于数据分析与预测
13        - 可以尝试使用三种方式获取信息，如果边界并不明确，则全部使用
14        - 在获取到信息后，尽可能将完整的所有信息传输给文件生成助手处理，以得到更完善的回答
15        - 文件生成：
16        - 可以根据用户的指令,使用你的自己工具生成Markdown、word、PDF三种格式的文件。不需要将文件生成的地址告诉你的子智能体，一切生成文档的
工作由你自己完成
17        你的工作具体要求：
18        - 根据用户的需求和你掌握的实际助手列表，完成任务
19        - 文件操作目录：每次任务开始时，系统会提供一个指定的绝对路径作为工作目录。
20        - 强制要求：你必须且只能在该工作目录下进行所有文件的创建、读取和保存操作。
21        - 指令下发：在调用子智能体时，必须明确将该工作目录的路径传达给它们，确保它们不会将文件生成到其他位置。
22        - 涉及生成输出文档的时候，严格遵循用户的生成需求，不得生成不符合用户最终预期的文档
23        你的主要任务类别
24        - 当用户没有明确表示生成何种类型的文件，则通过信息获取的方式获得所需数据，并直接反馈给用户
25        - 当用户明确表示需要生成文件的时候，只能交给文件生成助手去解决,不可以在获取信息后由你自己生成，你没有生成文件的能力
26        - 文件生成
27        你可以生成Markdown、PDF文档，具体生成方式如下：
28        对于Markdown文档，你掌握的工具包括 generate_markdown 工具，此工具可以根据用户的问题，生成对应的Markdown文件。
29        对于pdf文档，你你掌握的工具包括 convert_md_to_pdf。通过将你生成的Markdown文档进行转化得到pdf文档。
```

30

文档

在生成Markdown文档的时候，调用工具直接生成即可；在生成pdf文档的时候，需要先生成Markdown文档，再通过pdf工具进行转换得到最终的pdf

31

要按照用户指令去生成文档，不得生成用户要求之外的文档类型，比如要求生成pdf的时候只可以先生成Markdown，再转换为pdf。

32

生成内容要求：

33

无论什么复杂程度的任务，都需要生成一个**todo-list**进行规划

34

通过发送消息汇报文档生成进度及结果的时候，不允许发送文档的路径，只允许通知用户已成功创建

35

要根据接收的指示与检索到的知识信息内容去编写文档，内容要求丰富且全面，不少于**1000**字

36

【关键执行顺序】

37

1. 必须先调用子智能体（如网络搜索助手、数据库查询助手）获取信息。

38

2. ****绝不允许****在获取信息之前调用文件生成工具（**generate_markdown**）。

39

3. 严禁使用 "等待子任务完成" 之类的占位符内容生成文件。只有当你真正拿到了完整的信息文本后，才能调用 **generate_markdown**。

40

4. 如果你需要先搜索再生成，请分两步进行：第一步只调用搜索工具；第二步根据搜索结果调用生成工具。不要在一部（同一个 **Tool Call** 列表）中同时做这两件事。

4.4.6 测试函数

```
1 # ===== 本地测试入口 =====
2 if __name__ == "__main__":
3     task = "查询数据库中的药品信息，生成一个pdf文件！"
4     asyncio.run(run_deep_agent(task))
```

五、Web接口开发和测试

1. server基础准备

文件: `api/server.py`

```
1 import sys
2 import uuid
3 import asyncio
4 import uvicorn
5 from pathlib import Path
6 from fastapi import FastAPI, WebSocket, WebSocketDisconnect, UploadFile, File, Form
7 from fastapi.responses import FileResponse
8 from fastapi.middleware.cors import CORSMiddleware
9 from pydantic import BaseModel
10 from typing import List
11 import shutil
12
13 # Add project root to sys.path
14 current_dir = Path(__file__).resolve().parent
15 project_root = current_dir.parent
16 if str(project_root) not in sys.path:
17     sys.path.append(str(project_root))
18
19 # Import agent runner and monitor
20 # 注意: agent.main_agent 导入时会初始化 main_agent , 这可能需要几秒钟
21 from agent.main_agent import run_deep_agent
22 from api.monitor import monitor, manager
23
24 app = FastAPI(title="DeepAgents API")
25
26 # 挂载输出目录，以便前端访问生成的静态文件
27 # 假设输出目录位于项目根目录下的 output
28 output_dir = project_root / "output"
29 output_dir.mkdir(exist_ok=True)
30 # app.mount("/outputs", StaticFiles(directory=str(output_dir)), name="outputs")
31
32 # 定义上传目录 updated
33 updated_dir = project_root / "updated"
34 updated_dir.mkdir(exist_ok=True)
35
36 # 配置 CORS
37 app.add_middleware(
38     CORSMiddleware,
39     allow_origins=["*"],
40     allow_credentials=True,
41     allow_methods=["*"],
42     allow_headers=["*"],
43 )
```

```

44
45 class TaskRequest(BaseModel):
46     query: str
47     thread_id: str = None

```

2. 开启任务接口实现

```

1 @app.post("/api/task")
2 async def run_task(request: TaskRequest):
3     """
4     智能体任务启动接口 (Run Agent Task)。
5
6     目标：
7     1. 接收用户的自然语言指令。
8     2. 在后台异步启动 Agent 执行逻辑。
9     3. 返回会话 ID，供前端通过 WebSocket 订阅实时进度。
10
11    执行步骤：
12    1. 获取或生成 thread_id。
13    2. 触发异步任务 (asyncio.create_task)。
14    3. 立即返回响应，不阻塞 HTTP 线程。
15
16    Args:
17        request (TaskRequest): 包含用户 query 和可选 thread_id 的请求体。
18    """
19    # 1. [ID 初始化]
20    thread_id = request.thread_id or str(uuid.uuid4())
21
22    # 2. [后台执行] 异步运行 Agent，不阻塞主线程
23    # 注意：这里简单的使用 asyncio.create_task 触发，由 main_agent 内部负责实时推送
24    asyncio.create_task(run_deep_agent(request.query, thread_id))
25
26    # 3. [立即响应]
27    return {"status": "started", "thread_id": thread_id}

```

3. 上传文件接口

```

1 @app.post("/api/upload")
2 async def upload_files(files: List[UploadFile] = File(...), thread_id: str = Form(...)):
3     """
4     文件上传接口 (File Upload)。
5
6     目标：
7     1. 接收用户上传的一个或多个文件。
8     2. 保存到 `updated/session_{thread_id}` 目录。
9     3. 供 Agent 在后续任务中读取和分析。
10
11    Args:
12        files (List[UploadFile]): 文件对象列表。
13        thread_id (str): 关联的任务会话 ID。
14    """
15    # 1. [目录准备] 确保上传目录存在
16    target_dir = updated_dir / f"session_{thread_id}"
17    target_dir.mkdir(parents=True, exist_ok=True)
18
19    saved_files = []
20    # 2. [保存] 遍历并写入文件
21    for file in files:
22        file_path = target_dir / file.filename
23        # 使用二进制模式写入，支持各种文件格式（图片、PDF、文本等）
24        # shutil.copyfileobj 高效复制文件流，避免一次性加载大文件到内存
25        with file_path.open("wb") as buffer:
26            shutil.copyfileobj(file.file, buffer)
27        saved_files.append(file.filename)
28
29    # 3. [响应] 返回成功保存的文件列表
30    return {"status": "uploaded", "files": saved_files}

```

4. 下载文件接口

```

1 @app.get("/api/download")

```

```
2 async def download_file(path: str):
3     """
4     文件下载接口 (File Download)。
5
6     目标：
7     1. 根据绝对路径下载文件。
8     2. 严格的安全检查，防止越权访问。
9
10    Args:
11        path (str): 文件的绝对路径 (通常从 list_files 接口获取)。
12    """
13    # 1. [安全检查] 路径解析与越权校验
14    try:
15        abs_path = Path(path).resolve()
16        output_abs = output_dir.resolve()
17
18        # 必须确保请求的文件在 output 目录下
19        if not abs_path.is_relative_to(output_abs):
20            return {"error": "拒绝访问：只能下载输出目录下的文件"}
21    except Exception:
22        return {"error": "无效的路径参数"}
23    # 2. [存在性检查]
24    if not abs_path.exists():
25        return {"error": "文件不存在"}
26
27    # 3. [响应] 返回文件流 (浏览器自动触发下载)
28    return FileResponse(abs_path, filename=abs_path.name)
```

5. 查询所有文件列表接口

```
1 @app.get("/api/files")
2 async def list_files(path: str):
3     """
4     文件列表查询接口 (File Explorer)。
5
6     目标：
7     1. 列出指定目录下的所有生成文件。
8     2. 提供文件元数据 (大小、时间、下载链接)。
9     3. 严格的安全检查，防止路径遍历攻击。
10
11    Args:
12        path (str): 目标目录的绝对路径 (必须在 output 目录下)。
13    """
14    # 1. [调试] 打印请求路径
15    print(f"[DEBUG] 请求文件列表: {path}")
16
17    try:
18        # 2. [解析] 获取绝对路径对象
19        abs_path = Path(path).resolve()
20        output_abs = output_dir.resolve()
21
22        # 3. [安全] 检查路径是否越界 (Path Traversal Check)
23        if not abs_path.is_relative_to(output_abs):
24            print(f"[ERROR] 拒绝访问: {abs_path} 不在 {output_abs} 目录下")
25            return {"error": "拒绝访问：只能访问输出目录下的文件"}
26
27    except Exception as e:
28        print(f"[ERROR] 路径解析失败: {e}")
29        return {"error": f"路径无效: {e}"}
30
31    # 4. [检查] 目录是否存在
32    if not abs_path.exists():
33        return {"error": "目录不存在"}
34
35    files = []
36    try:
37        # 5. [遍历] 递归查找所有文件
38        for file_path in abs_path.rglob("*"):
39            if file_path.is_file():
40                # 计算相对路径，生成下载 URL
```

```

41         stat = file_path.stat()
42         files.append({
43             "name": file_path.name,
44             "type": "file",
45             "path": str(file_path),
46             # "url": f"/outputs/{url_path}",
47             "size": stat.st_size,
48             "mtime": stat.st_mtime
49         })
50
51     except Exception as e:
52         print(f"[ERROR] 遍历文件失败: {e}")
53         return {"error": str(e)}
54
55     # 6. [排序] 按修改时间倒序排列 (最新的在前)
56     files.sort(key=lambda x: x.get("mtime", 0), reverse=True)
57     print(f"[DEBUG] 找到 {len(files)} 个文件")
58     return {"files": files}

```

6. WebSocket 实时通讯

```

1  @app.websocket("/ws/{thread_id}")
2  async def websocket_endpoint(websocket: WebSocket, thread_id: str):
3      """
4      WebSocket 实时通讯核心接口 (Real-time Communication)。
5
6      目标：
7      1. 建立长连接，实现服务端与前端的双向通信。
8      2. 绑定 `thread_id`，实现会话级消息隔离。
9      3. 维持心跳 (Keep-Alive)，防止连接超时。
10
11     执行步骤：
12     1. 握手：接受 WebSocket 连接请求。
13     2. 注册：将连接实例绑定到 `monitor.manager`，关联 `thread_id`。
14     3. 循环：进入消息监听循环，处理前端发送的心跳或指令。
15     4. 异常：捕获断开连接异常，清理资源。
16
17     Args:
18         websocket (WebSocket): WebSocket 连接实例。
19         thread_id (str): 当前会话的唯一标识。
20     """
21     # 1. [注册] 建立连接并绑定到管理器
22     await manager.connect(websocket, thread_id)
23
24     try:
25         # 2. [循环] 保持连接活跃
26         while True:
27             # 3. [监听] 接收前端消息 (通常是 ping 心跳)
28             data = await websocket.receive_text()
29
30             # 4. [响应] 回复 pong 消息
31             await websocket.send_json({
32                 "type": "pong",
33                 "message": f"服务端已收到: {data}"
34             })
35
36     except WebSocketDisconnect:
37         # 5. [清理] 客户端主动断开
38         manager.disconnect(websocket, thread_id)
39         print(f"[WebSocket] 客户端已断开: {thread_id}")
40
41     except Exception as e:
42         # 6. [异常] 发生错误时断开
43         print(f"[WebSocket] 连接异常: {e}")
44         manager.disconnect(websocket, thread_id)

```

7. 启动服务器

```

1  if __name__ == "__main__":
2      uvicorn.run("api.server:app", host="0.0.0.0", port=8000, reload=True)

```

